SKRIPTA · MODERNI C++ (C++17 / C++20)

C++

od temelja do modernog koda

Svi važni koncepti objašnjeni polako i do kraja — s primjerima, čestim greškama, zadacima za vježbu i rješenjima.

```
class Student {
    std::string ime;
    std::vector<int> ocjene;
public:
    double prosjek() const;
};
```

Rujan 2026 · Verzija 1.0

Sadržaj

Kako koristiti ovu skriptu 4
- Kako je skripta organizirana 4
- Oznake u tekstu 4
- Što ti treba 5

DIO I — TEMELJI

1. Kako C++ zapravo radi 7
- 1.1 Što je C++ i zašto ga učiti 7
- 1.2 Od izvornog koda do programa 7
- 1.3 Prvi program, liniju po liniju 8
- 1.4 Kako program izvršava naredbe 9
- 1.5 Vrste grešaka 9
- 1.6 Komentari 10

2. Varijable, tipovi podataka i operatori 12
- 2.1 Što je zapravo varijabla 12
- 2.2 Deklaracija i inicijalizacija 12
- 2.3 Osnovni tipovi podataka 13
- 2.4 Konstante: const i constexpr 14
- 2.5 Ključna riječ auto 14
- 2.6 Aritmetički operatori 15
- 2.7 Operatori pridruživanja, ++ i -- 16
- 2.8 Pretvorbe tipova 16
- 2.9 Preljev i preciznost 17

3. Ulaz i izlaz 20
- 3.1 Tokovi: kako razmišljati o cin i cout 20
- 3.2 Ispis: cout, endl i posebni znakovi 20
- 3.3 Unos: cin >> 21
- 3.4 Što ako korisnik unese nešto krivo? 21
- 3.5 Čitanje cijelog reda: getline 22
- 3.6 Formatiranje ispisa 22

4. Grananje: if, switch i logika 25
- 4.1 Naredba if 25
- 4.2 Logički operatori 25
- 4.3 Zamke kod grananja 26
- 4.4 Naredba switch 27
- 4.5 Ternarni operator ? : 27
- 4.6 Blokovi i doseg varijabli 28

5. Petlje 30
- 5.1 Petlja while 30
- 5.2 Petlja do-while 30
- 5.3 Petlja for 30
- 5.4 break i continue 31
- 5.5 Ugniježđene petlje 32
- 5.6 Tipični obrasci 33
- 5.7 Česte greške s petljama 34

6. Funkcije 37
- 6.1 Anatomija funkcije 37
- 6.2 Deklaracija i definicija 37
- 6.3 Prijenos po vrijednosti 38
- 6.4 Prijenos po referenci 38
- 6.5 Zadani argumenti i preopterećenje 40
- 6.6 Doseg i životni vijek varijabli 40
- 6.7 Rekurzija 41

7. Polja, vektori i osnovni algoritmi 44
- 7.1 Klasično polje 44
- 7.2 Polja i funkcije 45
- 7.3 Dvodimenzionalna polja (matrice) 45
- 7.4 std::vector — polje koje raste 46
- 7.5 Osnovni algoritmi nad nizovima 47

8. Stringovi 50
- 8.1 Osnove 50
- 8.2 Prolazak kroz znakove 50
- 8.3 Najvažnije metode 51
- 8.4 Pretvorbe između brojeva i stringova 52
- 8.5 stringstream — string kao tok 52
- 8.6 C-stringovi (ukratko) 53

DIO II — MEMORIJA

9. Pokazivači i reference 56
- 9.1 Adrese i operator & 56
- 9.2 Pokazivač: varijabla koja čuva adresu 56
- 9.3 nullptr i opasni pokazivači 57
- 9.4 Pokazivači i reference — usporedba 57
- 9.5 Pokazivači i polja 58
- 9.6 const i pokazivači 59

10. Stog, gomila i dinamička memorija 61
- 10.1 Dva područja memorije: stog i gomila 61
- 10.2 new i delete 61
- 10.3 Dinamička polja: new[] i delete[] 62
- 10.4 Četiri klasične greške 62
- 10.5 Dinamička matrica 63
- 10.6 Kako raste vector? (pogled ispod haube) 63
- 10.7 Poruka za moderni C++ 64

11. Strukture i enumeracije 66
- 11.1 Definicija i korištenje strukture 66
- 11.2 Strukture i funkcije 66
- 11.3 Vektor struktura 67
- 11.4 Pokazivači na strukture i operator -> 68
- 11.5 Ugniježđene strukture 68
- 11.6 Enumeracije: enum i enum class 68

DIO III — OBJEKTNO ORIJENTIRANO PROGRAMIRANJE

12. Klase i objekti 71
- 12.1 Od strukture do klase: problem koji rješavamo 71
- 12.2 Prva klasa 71
- 12.3 Konstruktori 72
- 12.4 Destruktor 73
- 12.5 Pokazivač this, const metode i static članovi 74
- 12.6 struct ili class? 75
- 12.7 Klasa u više datoteka 75

13. Kopiranje, RAII i pravilo tri/pet/nule 77
- 13.1 RAII: resurs je vezan uz život objekta 77
- 13.2 Problem: plitka kopija 77
- 13.3 Četiri posebne funkcije za upravljanje resursom 78
- 13.4 Semantika premještanja (move) 79
- 13.5 Pravilo tri, pravilo pet i pravilo nule 80

14. Preopterećenje operatora 82
14.1 Ideja: operator je samo funkcija s posebnim imenom 82
14.2 Potpun primjer: klasa Razlomak 82
14.3 Metoda ili slobodna funkcija? 83
14.4 Prefiks i postfiks ++ 84
14.5 Operator indeksiranja [] 84

15. Nasljeđivanje 86
15.1 Osnovna ideja: odnos „je“ (is-a) 86
15.2 Sintaksa i protected 86
15.3 Redoslijed konstrukcije i destrukcije 87
15.4 Vrste nasljeđivanja 87
15.5 Izvedeni objekt kao bazni 88
15.6 Kompozicija ili nasljeđivanje? 88

16. Polimorfizam i virtualne funkcije 90
16.1 Statičko i dinamičko vezanje 90
16.2 Apstraktne klase i čiste virtualne funkcije 91
16.3 Kako to radi: tablica virtualnih funkcija 92
16.4 Virtualni destruktor 92
16.5 Još nekoliko alata 93

DIO IV — MODERNI C++ I STL

17. Predlošci (templates) 96
17.1 Predlošci funkcija 96
17.2 Predlošci klasa 97
17.3 Predlošci idu u zaglavlja 98
17.4 Ograničenja predložaka: koncepti (C++20) 98

18. STL kontejneri 100
18.1 Pregled 100
18.2 vector u dubinu 100
18.3 deque i list 101
18.4 Adapteri: stack, queue, priority_queue 101
18.5 set — skup jedinstvenih, sortiranih vrijednosti 102
18.6 map — rječnik ključ → vrijednost 102
18.7 Neuređeni kontejneri: unordered_map i unordered_set 103
18.8 Kako odabrati kontejner 104

19. Iteratori, algoritmi i lambda-izrazi 106
19.1 Iteratori 106
19.2 Lambda-izrazi 106
19.3 Najvažniji algoritmi 107
19.4 Rasponi (ranges, C++20) 109

20. Pametni pokazivači 111
20.1 Pojam vlasništva 111
20.2 unique_ptr — jedini vlasnik 111
20.3 shared_ptr — zajedničko vlasništvo 112
20.4 weak_ptr i problem ciklusa 113

21. Iznimke (exceptions) 115
21.1 throw, try i catch 115
21.2 Standardne iznimke 115
21.3 Vlastite iznimke 116
21.4 Odmatanje stoga i RAII 117
21.5 Kada koristiti iznimke 118

22. Rad s datotekama 120
22.1 Tokovi za datoteke 120
22.2 Čitanje red po red i dodavanje u datoteku 121
22.3 Primjer: CSV datoteka u vektor struktura 121

23. Moderni C++, dobre prakse i debugiranje 124
23.1 Stari stil → moderni stil 124
23.2 Nekoliko modernih alata na jednom mjestu 124
23.3 Česte greške — kontrolni popis 125
23.4 Kako pronaći grešku 126
23.5 Stil i navike koje se isplate 127
23.6 Kamo dalje 127

DIO V — VJEŽBA

24. Završni projekti 129
Projekt 1: Knjižnica ★★☆ 129
Projekt 2: Bankovni sustav s polimorfizmom ★★★ 130
Projekt 3: Analiza teksta ★★☆ 130
Projekt 4: Igra života ★★☆ 130

25. Tipična ispitna pitanja 132
25.1 Teorijska pitanja 132
25.2 Što ispisuje program? 134

26. Rješenja zadataka 138

Kako koristiti ovu skriptu

Ova skripta nastala je iz jednog jednostavnog iskustva: studenti rijetko padaju C++ zato što je „pretežak“. Padaju zato što su neki koncept naučili napamet, bez razumijevanja, pa se na prvom malo drugačijem zadatku sve raspadne. Zato ovdje nećeš naći samo „ovo se piše ovako“ — uz svaku važnu stvar objašnjeno je i **zašto** je tako.

Pretpostavljam da znaš ono najosnovnije: što je varijabla, kako se piše `if` i kako ispisati nešto s `cout`. Ipak, i te stvari ćemo proći ponovno, jer se u njima kriju detalji koji kasnije rade probleme. Nakon toga idemo redom: od petlji i funkcija, preko pokazivača i memorije (dio gdje većina ljudi zapne), do objektno orijentiranog programiranja i modernog C++-a kakav se danas piše u industriji.

Kako je skripta organizirana

Skripta ima pet dijelova. Svaki se oslanja na prethodni, pa ih je najbolje čitati redom:

1. **Temelji** — kako C++ radi, tipovi, ulaz/izlaz, grananje, petlje, funkcije, polja, vektori i stringovi.
2. **Memorija** — pokazivači, reference, stog i gomila (stack i heap), dinamička memorija, strukture.
3. **Objektno orijentirano programiranje** — klase, konstruktori, kopiranje i RAII, operatori, nasljeđivanje, polimorfizam.
4. **Moderni C++ i STL** — predlošci, kontejneri, algoritmi i lambde, pametni pokazivači, iznimke, datoteke.
5. **Vježba** — završni projekti, tipična ispitna pitanja i rješenja svih zadataka.

Oznake u tekstu

Kroz skriptu ćeš vidjeti obojene okvire. Svaki ima svoju ulogu:

Okvir	Što znači
Savjet	Praktično iskustvo — kako se to radi „u stvarnom svijetu“ i kako si olakšati posao.
Pazi!	Česte greške. Na ovima studenti gube najviše bodova i sati debugiranja.
Zašto?	Objašnjenje što se događa „ispod haube“. Ovo je dio koji gradi pravo razumijevanje.
Analogija	Usporedba sa svakodnevnim životom koja pomaže da apstraktna stvar „sjedne“.
Ukratko	Sažetak poglavlja — ono što moraš znati i kad te probude u 3 ujutro.
Na ispitu	Tipovi pitanja i zamke koje se redovito pojavljuju na kolokvijima i ispitima.

Kod je prikazan u sivim okvirima s plavom crtom. Okvir s crvenom crtom i oznakom **✗ Neispravno** prikazuje kod koji namjerno sadrži grešku — da je naučiš prepoznati. Ispod mnogih programa nalazi se i okvir **Izlaz programa** koji pokazuje što program ispisuje. Svi cjeloviti primjeri provjereni su prevoditeljem (g++ 13, standard C++20), a prikazani ispisi uspoređeni su sa stvarnim ispisom programa.

Zadaci

Na kraju svakog poglavlja nalaze se zadaci označeni težinom:

- ★☆☆ — lagano, provjera jesi li razumio osnovu;
- ★★☆ — srednje, treba spojiti nekoliko stvari;
- ★★★ — teže, razina ispitnog zadatka ili malo iznad.

Rješenja su na kraju skripte. Molim te, ne gledaj ih prije nego što se stvarno potrudiš. Programiranje se uči isto kao plivanje — gledanjem nitko nije naučio. Ako zapneš, ostavi zadatak na pola sata, pa se vrati. Tek onda pogledaj rješenje, zatvori ga i napiši svoje od nule.

> **SAVJET PROFESORA: KAKO UČITI PROGRAMIRANJE**
>
> Svaki primjer iz skripte **pretipkaj sam** (ne kopiraj!) i pokreni. Zatim ga namjerno pokvari: promijeni broj, obriši točku-zarez, zamijeni `<` s `<=`. Pogledaj što se dogodi. Deset minuta takvog „čačkanja“ vrijedi više od sat vremena čitanja.

Što ti treba

Treba ti prevoditelj (kompajler) za C++. Najjednostavnije opcije:

- **Windows:** Visual Studio Community (besplatan, sve dolazi u paketu) ili VS Code + MinGW (g++).
- **macOS:** u terminalu upiši `xcode-select --install` i dobit ćeš `clang++`.
- **Linux:** `sudo apt install g++`.
- **Bez instalacije:** online prevoditelji poput *Compiler Explorer* (godbolt.org) ili *OnlineGDB* — odlični za brzo isprobavanje.

Kad koristiš g++ iz terminala, preporučujem ovakav poziv, koji uključuje moderni standard i upozorenja:

```
g++ -std=c++20 -Wall -Wextra program.cpp -o program
./program
```

DIO I

Temelji

Kako C++ program nastaje i izvršava se, tipovi podataka, ulaz i izlaz, grananje, petlje, funkcije, polja, vektori i stringovi. Sve ono na čemu se gradi ostatak jezika.

1. Kako C++ zapravo radi

Prije nego što napišemo ijednu petlju, moramo razumjeti što se uopće događa kad „pokreneš program“. Studenti koji ovo preskoče kasnije ne razumiju poruke o greškama, ne znaju zašto im nešto „radi na jednom računalu, a na drugom ne“ i općenito imaju osjećaj da je C++ crna kutija. Nije. Potrošit ćemo nekoliko stranica da je otvorimo.

1.1 Što je C++ i zašto ga učiti

C++ je napravio Bjarne Stroustrup početkom 1980-ih kao nadogradnju jezika C. Ideja je bila jednostavna: zadržati brzinu i kontrolu koju daje C, a dodati alate za organizaciju velikih programa — prije svega **klase** (zato se isprva zvao „C s klasama“).

Danas se C++ koristi svugdje gdje je važna brzina i kontrola nad računalom: igre i pogoni za igre (Unreal Engine), web-preglednici (Chrome, Firefox), operacijski sustavi, baze podataka, financijski sustavi koji trguju u mikrosekundama, ugradbeni sustavi u automobilima, znanstvene simulacije. Kad netko kaže „ovo mora biti brzo“, vrlo često na kraju piše C++.

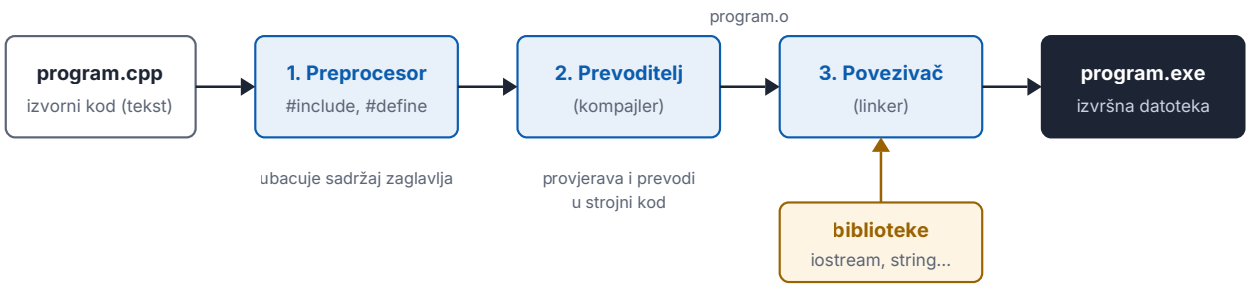
Jezik se razvija kroz **standarde**. Najvažnije je znati sljedeće:

Standard	Što je donio
C++98/03	„Klasični“ C++: klase, predlošci, STL.
C++11	Revolucija — `auto`, lambde, pametni pokazivači, move semantika, `nullptr`. Odavde počinje „moderni C++“.
C++14, C++17	Dorade: `std::optional`, strukturirano vezanje, `std::string_view` …
C++20, C++23	Koncepti, `ranges`, `std::format`, moduli…

Ova skripta uči **moderni C++** (C++17/20), ali objašnjava i starije konstrukcije, jer ćeš ih sretati u postojećem kodu i na ispitima.

1.2 Od izvornog koda do programa

Procesor ne razumije C++. Razumije samo **strojni kod** — nizove bitova koji znače stvari poput „zbroji ova dva registra“. Tvoj `.cpp` dokument je običan tekst, i netko ga mora prevesti. Taj proces ima tri koraka:



Slika 1.1 — Put od izvornog koda do izvršne datoteke

1. **Preprocesor** obrađuje sve linije koje počinju s `#`. Kad napišeš `#include <iostream>`, preprocesor doslovno *kopira* sadržaj datoteke `iostream` na to mjesto. Ništa pametnije od „kopiraj-zalijepi“.

2. **Prevoditelj (kompajler)** čita dobiveni tekst, provjerava je li sve ispravno napisano (sintaksa, tipovi) i prevodi ga u strojni kod. Rezultat je **objektna datoteka** (`.o` ili `.obj`). Ona još nije gotov program, jer se u njoj spominju funkcije koje su negdje drugdje (npr. kod za ispis na ekran).
3. **Povezivač (linker)** spaja tvoju objektnu datoteku s bibliotekama i drugim objektnim datotekama u jednu **izvršnu datoteku** (`.exe` na Windowsima).

Kad u Visual Studiju klikneš „Run“, sva tri koraka dogode se automatski, pa izgleda kao jedan. Ali greške dolaze iz različitih koraka i zato ih je dobro razlikovati.

ANALOGIJA: PREVOĐENJE KNJIGE

Zamisli da pišeš knjigu na hrvatskom, a čitatelj zna samo japanski. Preprocesor je asistent koji u tvoj rukopis ubaci poglavlja koja si citirao iz drugih knjiga. Prevoditelj prevodi tekst na japanski — i odbije posao ako nađe gramatičku grešku. Povezivač je knjigovežar koji sve prevedene dijelove uveže u jednu knjigu. Tek tada je čitatelj (procesor) može čitati — i to jako brzo, jer je sve već prevedeno.

ZAŠTO JE C++ BRZ?

Jezici poput Pythona prevode kod *dok se program izvršava* (interpretiraju ga), liniju po liniju. C++ sav posao prevođenja obavi *unaprijed*. Kad se program pokrene, procesor izvršava gotov strojni kod bez posrednika. Zato je C++ program često 10–100 puta brži od istog programa u Pythonu.

1.3 Prvi program, liniju po liniju

```cpp
#include <iostream>     // uključujemo biblioteku za ulaz i izlaz
using namespace std;    // da ne moramo pisati std:: ispred cout

int main() {            // ovdje program počinje
    cout << "Pozdrav, svijete!" << endl;
    cout << "2 + 3 = " << 2 + 3 << "\n";
    return 0;           // 0 znači: "sve je prošlo u redu"
}
```

IZLAZ PROGRAMA

```
Pozdrav, svijete!
2 + 3 = 5
```

Idemo redom, jer svaka linija ima razlog postojanja:

- `#include <iostream>` — naredba preprocesoru. Bez nje prevoditelj ne bi znao što je `cout`. Ime dolazi od *input/output stream* (tok ulaza i izlaza).
- `using namespace std;` — sve iz standardne biblioteke živi u **imenskom prostoru** (*namespace*) koji se zove `std`. Pravo ime od `cout` je `std::cout`. Ova linija kaže: „kad napišem `cout`, misli na `std::cout`“.
- `int main()` — **glavna funkcija**. Svaki C++ program ima točno jednu funkciju `main` i izvršavanje *uvijek* počinje od nje. `int` znači da funkcija na kraju vraća cijeli broj operacijskom sustavu.
- `{ ... }` — vitičaste zagrade omeđuju **blok** — tijelo funkcije.
- `cout << "..."` — šalje tekst na standardni izlaz (ekran). Operator `<<` zamisli kao strelicu: podaci „teku“ prema `cout`.
- `endl` i `"\n"` — prelazak u novi red (razliku ćemo objasniti u poglavlju 3).
- `return 0;` — kraj funkcije `main`; vrijednost 0 po dogovoru znači uspjeh. (U `main` je smiješ izostaviti — prevoditelj će sam dodati `return 0`.)
- `;` — **točka-zarez završava naredbu**. To je C++-ova „točka na kraju rečenice“.

> **SAVJET: USING NAMESPACE STD — DA ILI NE?**
>
> U malim programima i na ispitima `using namespace std;` je sasvim u redu i štedi tipkanje. U velikim projektima i u datotekama zaglavlja (`.h`) izbjegava se, jer „izlije“ tisuće imena u tvoj kod pa može doći do sudara imena. Profesionalni kod obično piše `std::cout`, `std::string`, `std::vector`. U ovoj skripti koristit ćemo oba stila, pa se navikni na oba.

1.4 Kako program izvršava naredbe

Program se izvršava **redom, odozgo prema dolje**, naredbu po naredbu, počevši od prve linije unutar `main`. Taj redoslijed mijenjaju samo tri stvari: **grananje** (`if`, `switch`), **petlje** (`for`, `while`) i **pozivi funkcija**. To je cijela „magija“ upravljanja tokom programa i njome se bave sljedeća poglavlja.

Razmaci, tabulatori i novi redovi prevoditelju uglavnom nisu važni. Ovo je, tehnički, ispravan program:

```
#include <iostream>
int main(){std::cout<<"Radi, ali je ruzno.\n";return 0;}
```

Ali nitko ga ne želi čitati. Kod se piše za ljude — prevoditelju je svejedno. Zato **uvlačimo** (*indent*) sadržaj svakog bloka za 4 razmaka i pišemo jednu naredbu po liniji.

1.5 Vrste grešaka

Greške su normalan dio programiranja. Iskusni programer ne radi manje grešaka — on ih samo brže prepoznaje. Postoje četiri vrste:

1. Greške pri prevođenju (compile-time). Prevoditelj odbije prevesti kod jer nešto nije napisano po pravilima jezika: nedostaje `;`, zagrada nije zatvorena, varijabla nije deklarirana, tipovi se ne slažu.

✗ NEISPRAVNO

```
#include <iostream>
using namespace std;

int main() {
    cout << "Bok" << endl     // nedostaje ;
    return 0;
}
```

Prevoditelj javlja nešto poput: `error: expected ';' before 'return'`. Primijeti da greška pokazuje na *sljedeću* liniju — prevoditelj je tek tamo shvatio da nešto ne štima.

2. Greške povezivanja (link-time). Kod je ispravno napisan, ali povezivač ne može pronaći neku funkciju — npr. deklarirao si funkciju, ali je nikad nisi napisao. Poruka obično sadrži riječi `undefined reference to …` ili `unresolved external symbol`.

3. Greške pri izvođenju (runtime). Program se prevede i pokrene, ali se sruši ili ponaša čudno: dijeljenje nulom, pristup elementu polja koji ne postoji, rad s memorijom koja nije tvoja.

4. Logičke greške. Najpodmuklije. Program radi i ništa se ne ruši, ali rezultat je pogrešan — npr. napisao si `<` umjesto `<=` pa petlja prođe jednom premalo. Prevoditelj ti tu ne može pomoći; pomažu samo testiranje i razmišljanje.

> **SAVJET PROFESORA: ČITAJ PRVU GREŠKU**
>
> Kad prevoditelj izbaci 40 grešaka, ne paničari. Vrlo često je samo **prva** prava greška, a ostalih 39 su posljedica zbunjenosti prevoditelja. Popravi prvu, prevedi ponovno — i često nestanu sve.

NEDEFINIRANO PONAŠANJE (UNDEFINED BEHAVIOR)

C++ ima pojam koji drugi jezici nemaju u tolikoj mjeri: **nedefinirano ponašanje (UB)**. Ako napraviš nešto što standard zabranjuje (npr. čitaš izvan granica polja), jezik ne garantira *ništa* — program može raditi ispravno, može se srušiti, može ispisati smeće, može raditi na tvom računalu, a na profesorovom ne. UB je razlog zašto ćemo u skripti stalno naglašavati „ovo nikad ne radi".

1.6 Komentari

Komentari su tekst koji prevoditelj ignorira. Pišu se za ljude — uključujući tebe za tri mjeseca.

```
// Jednolinijski komentar: sve do kraja reda se ignorira.

/* Višelinijski komentar:
   može se protezati kroz
   više redova. */

int brojStudenata = 30; // komentar može stajati i iza naredbe
```

Dobar komentar objašnjava **zašto**, a ne **što**. Komentar `i++; // povećaj i za 1` je beskoristan — to se vidi iz koda. Komentar `// preskačemo zaglavlje CSV datoteke` je koristan.

UKRATKO

- C++ je **prevođeni** jezik: preprocesor → prevoditelj → povezivač → izvršna datoteka.
- Izvršavanje uvijek počinje u funkciji `main`, i ide redom odozgo prema dolje.
- Naredbe završavaju s `;`, blokovi su omeđeni s `{ }`.
- Greške: pri prevođenju, pri povezivanju, pri izvođenju i logičke. Čitaj prvu poruku o grešci.
- Nedefinirano ponašanje znači da program može učiniti bilo što — izbjegavaj ga.

Zadaci

Zadatak 1.1 ★☆☆

Napiši program koji u tri reda ispisuje tvoje ime, ime fakulteta (ili škole) i omiljeni broj. Omiljeni broj ispiši kao izraz (npr. `6 * 7`), a ne kao gotov broj.

Zadatak 1.2 ★☆☆

Pronađi sve greške u sljedećem programu (ima ih četiri) i za svaku reci je li greška pri prevođenju ili logička greška. Program bi trebao ispisati površinu pravokutnika sa stranicama 4 i 5.

✗ NEISPRAVNO

```
#include <iostream>
using namespace std

int Main() {
    cout << "Povrsina: " << 4 + 5 << endl;
    return 0
}
```

Zadatak 1.3 ★☆☆

Napiši program koji pomoću `cout` ispisuje ovaj oblik (pazi na razmake):

```
   *
  ***
 *****
*******
  |
```

Zadatak 1.4 ★★☆

Objasni vlastitim riječima (3–4 rečenice): što se događa s linijom `#include <iostream>` prije nego što prevoditelj uopće počne s radom? Zašto program bez te linije ne može koristiti `cout`? Koja faza prevođenja javlja grešku ako je izostaviš?

2. Varijable, tipovi podataka i operatori

Ovo poglavlje možda izgleda kao nešto što već znaš. Ipak, pročitaj ga pažljivo — upravo ovdje se kriju zamke poput cjelobrojnog dijeljenja, preljeva i usporedbe decimalnih brojeva, na kojima pada iznenađujuće puno ljudi, i to ne samo početnika.

2.1 Što je zapravo varijabla

Memorija računala (RAM) je ogroman niz malih ćelija od po jedan **bajt** (8 bitova). Svaka ćelija ima svoj redni broj — **adresu**. Kad napišeš `int godine = 21;`, događa se sljedeće:

1. prevoditelj rezervira komad memorije dovoljno velik za jedan `int` (obično 4 bajta);
2. tom komadu memorije daje ime `godine` (ime postoji samo u tvom kodu — procesor vidi samo adresu);
3. u njega upisuje vrijednost 21.



Slika 2.1 — Varijable u memoriji: svaka ima ime, tip, vrijednost i adresu

Dakle, varijabla ima četiri svojstva: **ime**, **tip**, **vrijednost** i **adresu**. Tip je najvažniji, jer određuje tri stvari:

- **koliko memorije** varijabla zauzima;
- **kako se bitovi tumače** — isti niz bitova može biti broj 65 ili slovo `'A'`, ovisno o tipu;
- **koje su operacije dopuštene** — brojeve možeš množiti, tekst ne.

C++ je **statički tipiziran** jezik: tip varijable određuješ jednom, pri deklaraciji, i on se više nikad ne mijenja. To je razlika u odnosu na Python, gdje ista varijabla može danas biti broj, a sutra tekst.

2.2 Deklaracija i inicijalizacija

Deklaracija uvodi varijablu (ime i tip). **Inicijalizacija** joj daje početnu vrijednost. U C++-u postoji nekoliko načina:

```
int a;          // samo deklaracija: vrijednost je NEPOZNATA (smeće)
int b = 5;      // klasična inicijalizacija
int c(5);       // "konstruktorski" oblik
int d{5};       // vitičasta (uniformna) inicijalizacija, C++11
int e{};        // vitičaste bez vrijednosti: postavlja na 0
```

NEINICIJALIZIRANE VARIJABLE

Lokalna varijabla koju nisi inicijalizirao sadrži ono što je slučajno ostalo u toj memoriji. Čitanje takve vrijednosti je nedefinirano ponašanje. Klasičan primjer: `int suma;` pa u petlji `suma += x;` — rezultat je smeće. **Uvijek inicijaliziraj varijable**: `int suma = 0;`.

Vitičasta inicijalizacija ima jednu korisnu osobinu: **ne dopušta sužavanje** (*narrowing*) — tihi gubitak podataka:

```
int x = 3.7;    // prevede se, x postaje 3 (decimale tiho nestanu)
int y{3.7};     // GREŠKA pri prevođenju: sužavanje double -> int
```

Zato mnogi moderni stilovi preporučuju vitičaste zagrade. U praksi ćeš viđati oba oblika; važno je razumjeti razliku.

Pravila imenovanja. Ime smije sadržavati slova, znamenke i podvlaku `_`, ne smije počinjati znamenkom i ne smije biti ključna riječ (`int`, `for`, `class`...). C++ razlikuje velika i mala slova: `broj`, `Broj` i `BROJ` su tri različite varijable. Dogovori za višeriječna imena su `brojStudenata` (*camelCase*) ili `broj_studenata` (*snake_case*) — odaberi jedan i drži ga se.

> **SAVJET: IMENA SU DOKUMENTACIJA**
>
> `int x, y, z;` ne govori ništa. `int brojBodova, maxBodova, brojPokusaja;` govori sve. Kratka imena poput `i`, `j`, `n` u redu su samo za brojače petlji i vrlo kratke blokove.

2.3 Osnovni tipovi podataka

Tip	Tipična veličina	Raspon / preciznost	Primjer
`bool`	1 bajt	`true` ili `false`	`bool polozio = true;`
`char`	1 bajt	jedan znak (−128 do 127)	`char ocjena = 'A';`
`int`	4 bajta	oko ±2,1 milijarde	`int godine = 21;`
`long long`	8 bajtova	oko $\pm 9{,}2 \cdot 10^{18}$	`long long populacija = 8000000000LL;`
`unsigned int`	4 bajta	0 do oko 4,29 milijarde	`unsigned int brojac = 0;`
`float`	4 bajta	oko 7 značajnih znamenki	`float t = 36.6f;`
`double`	8 bajtova	oko 15–16 značajnih znamenki	`double pi = 3.14159;`
`std::string`	—	tekst proizvoljne duljine	`string ime = "Ana";`

Veličine su „tipične“ jer ih standard ne propisuje točno (propisuje samo minimume). Na današnjim računalima gotovo su uvijek ove iz tablice. Operator `sizeof` kaže koliko bajtova zauzima neki tip ili varijabla:

```
#include <iostream>
using namespace std;

int main() {
    cout << "bool: " << sizeof(bool) << "\n";
    cout << "char: " << sizeof(char) << "\n";
    cout << "int: " << sizeof(int) << "\n";
    cout << "long long: " << sizeof(long long) << "\n";
    cout << "double: " << sizeof(double) << "\n";
}
```

IZLAZ PROGRAMA

```
bool: 1
char: 1
int: 4
long long: 8
double: 8
```

`char` je zapravo broj. Svaki znak ima svoj brojčani kod (ASCII tablica): `'A'` je 65, `'B'` je 66, `'a'` je 97, `'0'` je 48. Zato sa znakovima možeš računati:

```cpp
#include <iostream>
using namespace std;

int main() {
    char c = 'A';
    cout << c << "\n";              // ispisuje znak
    cout << int(c) << "\n";         // ispisuje kod znaka
    cout << c + 1 << "\n";          // char + int = int!
    cout << char(c + 1) << "\n";    // vraćamo u char
    cout << '7' - '0' << "\n";      // znamenka kao znak -> broj
}
```

IZLAZ PROGRAMA

```
A
65
66
B
7
```

Posljednji redak je trik koji ćeš često koristiti: znamenke `'0'` – `'9'` u ASCII tablici stoje jedna iza druge, pa `'7' - '0'` daje broj 7.

`bool` može biti `true` ili `false`. Ispisuje se kao `1` ili `0`, osim ako uključiš `boolalpha`. Svaki broj različit od nule pretvara se u `true`, a nula u `false`.

SAVJET: KOJI TIP ODABRATI?

Za cijele brojeve koristi `int`, a ako vrijednosti mogu prijeći dvije milijarde — `long long`. Za decimalne brojeve koristi `double`. Tip `float` koristi samo kad imaš jako puno brojeva i štediš memoriju (npr. grafika). `unsigned` izbjegavaj za obične brojeve — razlog ćeš vidjeti u odjeljku 2.9.

2.4 Konstante: `const` i `constexpr`

Neke vrijednosti se ne smiju mijenjati. Označavamo ih s `const`:

```cpp
const double PDV = 0.25;
const int MAX_STUDENATA = 30;

PDV = 0.13;   // GREŠKA pri prevođenju: PDV je konstanta
```

`constexpr` je jača verzija: vrijednost mora biti poznata već **pri prevođenju** (npr. `constexpr int VELICINA = 100;`). Koristi se, među ostalim, za veličine polja.

ZAŠTO KONSTANTE UMJESTO „MAGIČNIH BROJEVA“?

Usporedi `cijena * 1.25` i `cijena * (1 + PDV)`. Drugi oblik se čita sam od sebe, a kad se stopa PDV-a promijeni, mijenjaš je na jednom mjestu umjesto da tražiš broj 1.25 po cijelom programu (i slučajno promijeniš neki 1.25 koji nema veze s PDV-om).

2.5 Ključna riječ `auto`

Od C++11 prevoditelj može sam zaključiti tip iz vrijednosti kojom inicijaliziraš varijablu:

```cpp
auto a = 5;         // int
auto b = 5.0;       // double
auto c = 'x';       // char
auto d = 5LL;       // long long
```

`auto` je najkorisniji kod dugih imena tipova (vidjet ćeš ga s iteratorima u poglavlju 19). Za obične brojeve jasnije je napisati tip.

ZAMKA: AUTO I TEKST

`auto s = "Ana";` **nije** `std::string`, nego `const char*` — pokazivač na znakove u C-stilu (poglavlje 8). Ako želiš string, piši `string s = "Ana";`.

2.6 Aritmetički operatori

Operator	Značenje	Primjer	Rezultat
`+` `-` `*`	zbrajanje, oduzimanje, množenje	`7 * 3`	`21`
`/`	dijeljenje	`7 / 2`	`3` (!)
`%`	ostatak dijeljenja (modulo)	`7 % 2`	`1`

Cjelobrojno dijeljenje je izvor greške broj jedan u početnim programima. Kad su **oba** operanda cijeli brojevi, rezultat je cijeli broj — decimale se jednostavno odbacuju (reže se prema nuli, ne zaokružuje):

```
#include <iostream>
using namespace std;

int main() {
    cout << 7 / 2 << "\n";      // oba int -> cjelobrojno dijeljenje
    cout << 7.0 / 2 << "\n";    // jedan je double -> pravo dijeljenje
    cout << 7 % 2 << "\n";
    cout << -7 / 2 << "\n";     // reže se prema nuli
    cout << -7 % 2 << "\n";     // predznak ostatka prati djeljenika
}
```

IZLAZ PROGRAMA

```
3
3.5
1
-3
-1
```

Klasična greška izgleda ovako:

```
#include <iostream>
using namespace std;

int main() {
    int zbroj = 17, n = 4;
    double krivo = zbroj / n;                       // 17/4 = 4, pa tek onda u double
    double tocno = static_cast<double>(zbroj) / n;  // 17.0/4 = 4.25
    cout << krivo << " " << tocno << "\n";
}
```

IZLAZ PROGRAMA

```
4 4.25
```

ZAŠTO KRIVO IZNOSI 4, A NE 4.25?

C++ izraz računa **iznutra prema van**, ne gledajući kamo će se rezultat spremiti. Prvo izračuna `zbroj / n` — dva `int`-a, dakle cjelobrojno dijeljenje, rezultat 4. Tek *nakon toga* taj 4 pretvara u `double` da bi ga spremio u `krivo`. Varijabla s lijeve strane ne utječe na način računanja desne strane.

Operator `%` (ostatak) koristi se nevjerojatno često:

- `n % 2 == 0` — je li broj paran;
- `n % 10` — zadnja znamenka broja; `n / 10` — broj bez zadnje znamenke;
- `n % k == 0` — je li `n` djeljiv s `k`;
- `(i + 1) % 7` — „kružno“ brojanje (dani u tjednu, indeksi u krug).

`%` radi samo s cijelim brojevima. Za `double` postoji funkcija `fmod` iz `<cmath>`.

Prioritet operatora je kao u matematici: `*`, `/`, `%` imaju prednost pred `+` i `-`, a operatori istog prioriteta računaju se slijeva nadesno. Kad nisi siguran — stavi zagrade. `(a + b) / 2` je čitljivije i sigurnije od razmišljanja o pravilima.

2.7 Operatori pridruživanja, ++ i --

Operator `=` je **pridruživanje**, ne jednakost: `x = x + 1` znači „izračunaj `x + 1` i spremi to u `x`“. Za česte operacije postoje skraćeni oblici: `x += 5` (isto što i `x = x + 5`), zatim `-=`, `*=`, `/=`, `%=`.

Operatori `++` i `--` povećavaju ili smanjuju vrijednost za 1, a postoje u dva oblika:

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int b = a++;    // POSTFIKS: b dobiva staru vrijednost (5), pa se a poveća
    cout << "a=" << a << ", b=" << b << "\n";
    int c = ++a;    // PREFIKS: a se prvo poveća (na 7), pa c dobiva novu vrijednost
    cout << "a=" << a << ", c=" << c << "\n";
}
```

IZLAZ PROGRAMA

```
a=6, b=5
a=7, c=7
```

Pravilo za pamćenje: **gdje je `++`, tamo je vrijeme**. `++a` — prvo povećaj, onda koristi. `a++` — prvo koristi, onda povećaj. Kad `++` stoji sam u naredbi (`i++;`), razlike nema.

> **NE MIJENJAJ ISTU VARIJABLU DVAPUT U JEDNOM IZRAZU**
>
> Izrazi poput `i = i++ + ++i;` ili `cout << i << i++;` (u starijim standardima) nemaju jasno definiran rezultat. Pojavljuju se na nekim ispitima kao „trik pitanja“, ali u stvarnom kodu im nije mjesto. Jedna promjena varijable po izrazu — i nema problema.

2.8 Pretvorbe tipova

Kad u izrazu miješaš tipove, C++ ih automatski (implicitno) pretvara. Pravilo je da se „manji“ tip pretvara u „veći“: `int + double` postaje `double + double`, a rezultat je `double`.

Pretvorba iz `double` u `int` **odsijeca** decimale — ne zaokružuje! Za zaokruživanje postoji `round` iz `<cmath>`. Eksplicitnu pretvorbu radiš operatorom `static_cast`:

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double x = 3.99;
    cout << static_cast<int>(x) << "\n";    // odsijecanje
    cout << round(x) << "\n";               // zaokruživanje
    cout << static_cast<int>(-3.99) << "\n";
    cout << (int)x << "\n";                 // stari C-stil, radi isto
}
```

IZLAZ PROGRAMA

```
3
4
-3
3
```

SAVJET: STATIC_CAST UMJESTO (INT)

Stari C-oblik `(int)x` radi, ali je „tup alat“ — može učiniti i opasne pretvorbe bez upozorenja, i teško ga je pronaći pretraživanjem. `static_cast<int>(x)` je dulji, ali eksplicitno kaže što želiš. U modernom kodu koristi `static_cast`.

2.9 Preljev i preciznost

Tipovi imaju ograničen raspon. Što se dogodi kad ga prijeđeš?

Preljev (overflow) predznačenih tipova (`int`, `long long`) je nedefinirano ponašanje. U praksi obično dobiješ „namotavanje“ na negativan broj, ali jezik to ne garantira.

✗ NEISPRAVNO

```cpp
int a = 50000;
long long rez = a * a;   // PAZI: a * a se računa u int-u i preljeva se,
                         // tek se onda (pogrešan) rezultat sprema u long long
```

Ispravno je pretvoriti barem jedan operand prije računanja: `1LL * a * a` ili `static_cast<long long>(a) * a`. Isti princip kao kod cjelobrojnog dijeljenja: izraz se računa u tipu operanada, a ne u tipu varijable u koju spremaš.

Nepredznačeni tipovi (`unsigned`) se ne preljevaju nedefinirano — oni se namotavaju po definiciji:

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    cout << INT_MAX << "\n";          // najveći int
    cout << 50000LL * 50000 << "\n";  // računamo u long long-u
    unsigned int u = 0;
    u = u - 1;                        // 0 - 1 se "namota" na najveći unsigned
    cout << u << "\n";
}
```

IZLAZ PROGRAMA

```
2147483647
2500000000
4294967295
```

Upravo zbog toga je `unsigned` opasan za obične brojeve: izraz `u - 1 < 0` za `unsigned` nikad nije istinit, a petlja `for (unsigned i = n - 1; i >= 0; i--)` nikad ne završava.

Decimalni brojevi nisu točni. `double` pamti broj u binarnom zapisu, a mnogi decimalni brojevi (npr. 0.1) u binarnom sustavu imaju beskonačan zapis — kao što 1/3 u dekadskom ima 0.333... Zato:

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double x = 0.1 + 0.2;
    if (x == 0.3) cout << "jednako\n";
    else          cout << "NIJE jednako!\n";

    const double EPS = 1e-9;          // tolerancija
    if (fabs(x - 0.3) < EPS) cout << "priblizno jednako\n";
}
```

IZLAZ PROGRAMA

```
NIJE jednako!
priblizno jednako
```

PRAVILO: DECIMALNE BROJEVE NE USPOREĐUJ S ==

Umjesto `a == b` provjeri je li razlika dovoljno mala: `fabs(a - b) < 1e-9`. Iz istog razloga novac se u ozbiljnim sustavima ne čuva u `double`, nego u cijelim brojevima (u centima).

UKRATKO

- Varijabla = ime + tip + vrijednost + adresa. Tip određuje veličinu, tumačenje i dopuštene operacije.
- Uvijek inicijaliziraj varijable. Vitičasta inicijalizacija `{}` sprječava tiho sužavanje.
- `int / int` je cjelobrojno dijeljenje. Za pravo dijeljenje barem jedan operand mora biti `double`.
- Izraz se računa u tipu operanada, a ne u tipu odredišne varijable (vrijedi i za preljev).
- `++a` prvo povećava pa koristi; `a++` prvo koristi pa povećava.
- `double` u `int` odsijeca decimale. Za eksplicitne pretvorbe koristi `static_cast`.
- Decimalne brojeve uspoređuj s tolerancijom, nikad s `==`.

NA ISPITU

Gotovo svaki ispit ima zadatak tipa „što ispisuje ovaj kod“ s kombinacijom `/`, `%`, pretvorbi i `++`. Rješavaj ih polako, operaciju po operaciju, i za svaki međurezultat zapiši i **tip** (`int` ili `double`) — ne samo vrijednost.

Zadaci

Zadatak 2.1 ★☆☆

Napiši program koji zamjenjuje vrijednosti dviju varijabli `a = 3` i `b = 8` pomoću treće, pomoćne varijable. Ispiši vrijednosti prije i poslije zamjene.

Zadatak 2.2 ★☆☆

Bez pokretanja programa odredi vrijednost i tip svakog izraza. Zatim provjeri na računalu.

`17 / 5`, `17 % 5`, `17 / 5.0`, `(double)(17 / 5)`, `17 / 5 * 5 + 17 % 5`, `'a' + 1`, `char('a' + 1)`, `10 / 4 * 2.0`, `-17 % 5`

Zadatak 2.3 ★★☆

Korisnik unosi broj sekundi (npr. 7384). Ispiši koliko je to sati, minuta i sekundi u obliku `2 h 3 min 4 s`. Koristi samo `/` i `%`.

Zadatak 2.4 ★★☆

Za uneseni troznamenkasti broj ispiši zbroj njegovih znamenki i broj zapisan unatrag (za 472 ispiši `13` i `274`).

Zadatak 2.5 ★★☆

Korisnik unosi tri cijela broja (ocjene). Ispiši njihov prosjek kao decimalni broj. Zatim ispiši i prosjek zaokružen na najbliži cijeli broj. Pazi na cjelobrojno dijeljenje!

Zadatak 2.6 ★★★

Zamijeni vrijednosti dviju cjelobrojnih varijabli **bez** pomoćne varijable, koristeći samo zbrajanje i oduzimanje. Zatim objasni u kojoj situaciji ovaj trik može dati pogrešan rezultat (razmisli o odjeljku 2.9).

3. Ulaz i izlaz

Program koji ne komunicira s korisnikom ne može puno. U ovom poglavlju temeljito ćemo proći `cout` i `cin`, objasniti jedan problem na kojem zapne gotovo svaki student (miješanje `cin >>` i `getline`) i naučiti kako uredno formatirati ispis.

3.1 Tokovi: kako razmišljati o `cin` i `cout`

C++ ulaz i izlaz promatra kao **tokove** (*streams*) — nizove znakova koji teku od izvora do odredišta. `cout` je izlazni tok prema ekranu, a `cin` ulazni tok s tipkovnice. Operatori to lijepo prikazuju:

- `cout << x` — podatak **teče u** `cout` (strelica pokazuje prema toku);
- `cin >> x` — podatak **teče iz** `cin` u varijablu `x`.

Operatore možemo nizati jer svaki od njih vraća sam tok, pa se sljedeći `<<` nastavlja na njega: `cout << "a=" << a << "\n";` zapravo je `((cout << "a=") << a) << "\n";`.

3.2 Ispis: `cout`, `endl` i posebni znakovi

Za prelazak u novi red imamo dvije mogućnosti: `endl` i `"\n"`. Razlika je u tome što `endl` osim novog reda radi i **pražnjenje međuspremnika** (*flush*) — tjera da se sve odmah zapiše na ekran. Ispis inače ide u međuspremnik i šalje se u većim komadima, jer je to puno brže.

> **SAVJET: KORISTI \"\N\"**
>
> U običnom ispisu koristi `"\n"`. `endl` u petlji koja ispisuje sto tisuća redova može usporiti program nekoliko puta. `endl` ima smisla kad želiš da se ispis *sigurno* pojavi odmah (npr. prije nečega što bi moglo srušiti program).

Neke znakove ne možemo napisati izravno u navodnicima, pa za njih postoje **escape sekvence** koje počinju obrnutom kosom crtom:

Sekvenca	Značenje	Sekvenca	Značenje
`\n`	novi red	`\\`	obrnuta kosa crta
`\t`	tabulator	`\"`	navodnik
`\'`	apostrof	`\0`	nul-znak (kraj C-stringa)

> **HRVATSKI ZNAKOVI U KONZOLI**
>
> Na Windowsima konzola često krivo prikazuje slova č, ć, ž, š, đ. Zato u primjerima u navodnicima pišemo bez dijakritika (`"Unesi broj: "`). U komentarima dijakritici ne smetaju.

3.3 Unos: `cin >>`

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Unesi dva broja: ";
    cin >> a >> b;
    cout << "Zbroj je " << a + b << "\n";
}
```

IZLAZ PROGRAMA

```
Unesi dva broja: 3 4
Zbroj je 7
```

Važno je razumjeti kako `>>` radi, jer odatle dolaze sve kasnije zabune:

1. **preskače sve bjeline** (razmake, tabulatore, nove redove) prije podatka;
2. čita znakove **dok god odgovaraju tipu varijable** — za `int` to su znamenke (i predznak), za `string` sve do prve bjeline;
3. staje i **ostavlja ostatak ulaza u toku** — uključujući i znak novog reda `'\n'` koji je nastao kad je korisnik pritisnuo Enter.

Zato je svejedno unosi li korisnik `3 4` u jednom redu ili `3` Enter `4` u dva reda — bjeline između se preskaču. A zato `cin >> ime` za unos „Ana Marić“ pročita samo „Ana“.

3.4 Što ako korisnik unese nešto krivo?

Ako program očekuje broj, a korisnik upiše `abc`, `cin` ne može ništa pročitati. Tada se tok postavlja u **stanje greške** (*fail state*): varijabla dobiva vrijednost 0, a *svi sljedeći* unosi se preskaču dok grešku ne očistiš. Uspjeh unosa provjerava se ovako:

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main() {
    int godine;
    cout << "Unesi godine: ";
    while (!(cin >> godine)) {          // ako unos NIJE uspio...
        cout << "To nije broj. Pokusaj ponovno: ";
        cin.clear();                    // 1) očisti stanje greške
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // 2) odbaci ostatak reda
    }
    cout << "Imas " << godine << " godina.\n";
}
```

IZLAZ PROGRAMA

```
Unesi godine: abc
To nije broj. Pokusaj ponovno: 21
Imas 21 godina.
```

Izraz `cin >> godine` vraća sam `cin`, a tok se u uvjetu ponaša kao `bool`: `true` ako je sve u redu, `false` ako je nastala greška. Ovaj obrazac — provjera, `clear`, `ignore` — vrijedi zapamtiti.

ČITANJE DO KRAJA ULAZA

Isti trik koristi se za čitanje nepoznatog broja podataka: `while (cin >> x) { ... }` čita brojeve sve dok ih ima. Petlja staje kad unos završi (Ctrl+Z na Windowsima, Ctrl+D na Linuxu/macOS-u) ili kad naiđe nešto što nije broj.

3.5 Čitanje cijelog reda: `getline`

Kad trebaš pročitati tekst s razmacima (ime i prezime, adresu, rečenicu), koristiš `getline`:

```
string imePrezime;
getline(cin, imePrezime);   // čita SVE do Entera (Enter se odbacuje)
```

A sada problem koji muči generacije studenata:

✗ NEISPRAVNO

```
int godine;
string ime;
cout << "Godine: ";
cin >> godine;          // korisnik upiše 21 i pritisne Enter
cout << "Ime i prezime: ";
getline(cin, ime);      // NE ČEKA unos — ime je prazan string!
```

ZAŠTO GETLINE „PRESKOČI" UNOS?

Korisnik je upisao `21⏎`. `cin >> godine` pročita `21` i **ostavi `'\n'` u toku** (točka 3 iz odjeljka 3.3). Zatim dolazi `getline`, čiji je posao „čitaj do prvog `'\n'`". On odmah naiđe na zaostali `'\n'`, pročita prazan red i završi. Program nije pogriješio — radi točno ono što smo mu rekli.

Rješenje je odbaciti zaostali kraj reda prije `getline`:

```
cin >> godine;
cin.ignore(numeric_limits<streamsize>::max(), '\n');  // odbaci ostatak reda
getline(cin, ime);

// ili kraće: ws "pojede" sve bjeline prije teksta
cin >> godine;
getline(cin >> ws, ime);
```

3.6 Formatiranje ispisa

Biblioteka `<iomanip>` sadrži **manipulatore** koji uređuju ispis. Najvažniji su:

Manipulator	Učinak	Traje
`setprecision(n)`	broj značajnih znamenki (ili decimala uz `fixed`)	dok ga ne promijeniš
`fixed`	ispis s fiksnim brojem decimala	dok ga ne promijeniš
`setw(n)`	širina polja za **sljedeći** podatak	samo jedan ispis!
`left` / `right`	poravnanje unutar polja	dok ga ne promijeniš
`setfill(c)`	znak za popunu (umjesto razmaka)	dok ga ne promijeniš

Razlika između `setprecision` sa i bez `fixed` često zbunjuje, pa je pogledajmo:

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double x = 3.14159265;
    cout << x << "\n";                              // zadano: 6 značajnih znamenki
    cout << setprecision(3) << x << "\n";           // 3 ZNAČAJNE znamenke
    cout << fixed << setprecision(3) << x << "\n";  // 3 DECIMALE
    cout << setprecision(2) << 12.5 << "\n";        // fixed i dalje vrijedi
}
```

IZLAZ PROGRAMA

```
3.14159
3.14
3.142
12.50
```

Za ispis tablica koristi se `setw`. Pazi: on vrijedi samo za **prvi sljedeći** podatak, pa ga ponavljaš za svaki stupac.

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << left << setw(10) << "Ime" << right << setw(8) << "Bodovi" << "\n";
    cout << left << setw(10) << "Ana" << right << setw(8) << 87.5 << "\n";
    cout << left << setw(10) << "Marko" << right << setw(8) << 92 << "\n";
    cout << setfill('0') << setw(2) << 7 << ":" << setw(2) << 5 << "\n";
}
```

IZLAZ PROGRAMA

```
Ime         Bodovi
Ana           87.5
Marko           92
07:05
```

MODERNI NAČIN: STD::FORMAT (C++20)

C++20 donosi `std::format` iz zaglavlja `<format>`, sličan Pythonovom formatiranju: `cout << format("{:<10}{:>8.2f}\n", "Ana", 87.5);` Čitljiviji je od manipulatora, a podržavaju ga novije verzije prevoditelja (GCC 13+, Visual Studio 2019+). Na ispitima se još uvijek češće traži `<iomanip>`.

UKRATKO

- `cout << x` ispisuje, `cin >> x` čita. Operatori se mogu nizati.
- Za novi red koristi `"\n"`; `endl` dodatno prazni međuspremnik i sporiji je.
- `cin >>` preskače bjeline, čita jedan „komad“ i **ostavlja `'\n'` u toku**.
- Nakon neuspjelog unosa tok je u stanju greške: `cin.clear()` + `cin.ignore(...)`.
- `getline(cin, s)` čita cijeli red. Nakon `cin >>` prvo odbaci zaostali `'\n'` (`cin.ignore` ili `cin >> ws`).
- `fixed` + `setprecision(n)` = n decimala. `setw` vrijedi samo za jedan podatak.

Zadaci

Zadatak 3.1 ★☆☆

Napiši program koji pita korisnika ime (jedna riječ) i godinu rođenja, a zatim ispiše: `Bok, Ana! Ove godine punis 21.` (Trenutnu godinu zapiši kao konstantu.)

Zadatak 3.2 ★☆☆

Korisnik unosi cijenu proizvoda bez PDV-a. Ispiši iznos PDV-a (25 %) i ukupnu cijenu, oboje na točno dvije decimale.

Zadatak 3.3 ★★☆

Korisnik unosi broj godina, a zatim puno ime i prezime (s razmakom). Ispiši `Ime: ...` i `Godine: ...`. Program mora ispravno raditi i kad korisnik prvo unese broj pa pritisne Enter.

Zadatak 3.4 ★★☆

Napiši program koji ispisuje tablicu kvadrata i kubova brojeva od 1 do 5 s poravnatim stupcima širine 6 znakova (koristi `setw`; ako još ne znaš petlje, napiši pet redova ručno).

Zadatak 3.5 ★★★

Napiši program koji traži cijeli broj između 1 i 10. Ako korisnik upiše nešto što nije broj ili je broj izvan raspona, program ispisuje poruku i traži ponovni unos, sve dok ne dobije ispravan broj.

4. Grananje: `if`, `switch` i logika

Do sada su se naši programi izvršavali ravno, od prve do zadnje linije. Grananje omogućuje programu da **donosi odluke**: ovisno o uvjetu, izvršava se jedan ili drugi dio koda. Samo grananje nije teško — teški su uvjeti. Zato ćemo dobar dio poglavlja posvetiti logici.

4.1 Naredba `if`

```
if (uvjet) {
    // izvršava se ako je uvjet istinit
} else {
    // izvršava se ako uvjet nije istinit
}
```

Uvjet je bilo koji izraz koji se može protumačiti kao `bool`. Najčešće ga gradimo **relacijskim operatorima**:

Operator	Značenje	Operator	Značenje
`==`	jednako	`!=`	različito
`<`	manje	`<=`	manje ili jednako
`>`	veće	`>=`	veće ili jednako

Kad treba izabrati između više mogućnosti, nižemo `else if`. Uvjeti se provjeravaju **redom**, izvrši se blok **prvog** istinitog, a ostali se preskaču:

```
#include <iostream>
using namespace std;

int main() {
    int bodovi = 73;
    int ocjena;
    if (bodovi >= 90)      ocjena = 5;
    else if (bodovi >= 75) ocjena = 4;
    else if (bodovi >= 60) ocjena = 3;
    else if (bodovi >= 50) ocjena = 2;
    else                   ocjena = 1;
    cout << "Ocjena: " << ocjena << "\n";
}
```

IZLAZ PROGRAMA

```
Ocjena: 3
```

Primijeti da u drugom uvjetu ne pišemo `bodovi >= 75 && bodovi < 90`. Ne treba: ako smo stigli do drugog uvjeta, prvi *sigurno* nije bio istinit, pa znamo da je `bodovi < 90`. Redoslijed uvjeta je dio logike.

4.2 Logički operatori

Složene uvjete gradimo logičkim operatorima:

Operator	Naziv	Istinit je kada...
`&&`	I (konjunkcija)	su **oba** uvjeta istinita
`||`	ILI (disjunkcija)	je **barem jedan** uvjet istinit
`!`	NE (negacija)	je uvjet neistinit

Prioritet: `!` je najjači, zatim `&&`, pa tek onda `||`. Dakle `a || b && c` znači `a || (b && c)`. Kad kombiniraš `&&` i `||`, **uvijek stavi zagrade** — i zbog sebe i zbog onoga tko čita.

Kratko spajanje (short-circuit evaluation). C++ računa logičke izraze slijeva nadesno i staje čim zna rezultat. Kod `A && B`, ako je `A` neistinit, `B` se *uopće ne računa* (rezultat je ionako `false`). Kod `A || B`, ako je `A` istinit, `B` se preskače. To je jako korisno:

```
if (n != 0 && zbroj / n > 10) { ... }   // sigurno: dijeljenje se ne dogodi ako je n == 0
if (i < v.size() && v[i] == x) { ... }  // sigurno: v[i] se čita samo ako i postoji
```

Da smo zamijenili redoslijed uvjeta, program bi se u rubnom slučaju srušio. Redoslijed u `&&` nije svejedno!

De Morganovi zakoni pomažu kad treba negirati složen uvjet:

- `!(A && B)` je isto što i `!A || !B`
- `!(A || B)` je isto što i `!A && !B`

Primjer: „broj **nije** u intervalu [1, 10]“ je `!(x >= 1 && x <= 10)`, što je isto što i `x < 1 || x > 10`. Drugi oblik je čitljiviji.

4.3 Zamke kod grananja

Ove četiri greške vidio sam na stotinama ispita. Nauči ih prepoznati.

Zamka 1: `=` umjesto `==`.

✗ NEISPRAVNO

```
int x = 5;
if (x = 0) {      // PRIDRUŽIVANJE, ne usporedba! x postaje 0,
    ...           // a uvjet je 0 -> false. Blok se nikad ne izvrši.
}
```

Prevoditelj s uključenim upozorenjima (`-Wall`) upozorit će te na ovo. Još jedan razlog da ih uvijek uključiš.

Zamka 2: točka-zarez iza uvjeta.

✗ NEISPRAVNO

```
if (x > 10);            // ; je prazna naredba — ona je "tijelo" if-a!
{
    cout << "Veliko\n"; // ovaj blok se izvršava UVIJEK
}
```

Zamka 3: nedostaju vitičaste zagrade. Bez zagrada, `if` obuhvaća **samo jednu** sljedeću naredbu. Uvlačenje prevoditelju ne znači ništa:

✗ NEISPRAVNO

```
if (bodovi < 50)
    cout << "Pao si.\n";
    cout << "Dodi na popravni.\n";   // izvršava se UVIJEK, bez obzira na uvlačenje
```

Zamka 4: matematička notacija. U matematici pišeš `0 < x < 10`. U C++-u se to prevede, ali znači nešto sasvim drugo: prvo se izračuna `0 < x` (rezultat `true` ili `false`, tj. 1 ili 0), a zatim se provjerava je li *taj* rezultat manji od 10 — što je uvijek istina. Ispravno: `0 < x && x < 10`.

SAVJET PROFESORA: UVIJEK PIŠI VITIČASTE ZAGRADE

Čak i kad blok ima jednu naredbu, piši `{ }`. Tako zamka 3 jednostavno ne može nastati kad kasnije dodaš drugu naredbu. Većina profesionalnih stilova kodiranja to i propisuje.

4.4 Naredba `switch`

Kad se jedna vrijednost uspoređuje s nizom konkretnih konstanti, `switch` je pregledniji od lanca `else if`:

```cpp
#include <iostream>
using namespace std;

int main() {
    double a = 12, b = 4;
    char op = '/';
    switch (op) {
        case '+': cout << a + b << "\n"; break;
        case '-': cout << a - b << "\n"; break;
        case '*': cout << a * b << "\n"; break;
        case '/':
            if (b != 0) cout << a / b << "\n";
            else        cout << "Dijeljenje s nulom!\n";
            break;
        default:
            cout << "Nepoznat operator.\n";
    }
}
```

IZLAZ PROGRAMA

```
3
```

Pravila za `switch`:

- vrijednost mora biti **cjelobrojnog tipa** (`int`, `char`, `enum`) — `switch` ne radi s `double` ni sa `string`;
- iza `case` mora stajati **konstanta**, ne varijabla ni raspon;
- `default` se izvršava kad nijedan `case` ne odgovara (ne mora biti zadnji, ali po dogovoru jest);
- `break` izlazi iz `switch`-a.

PROPADANJE (FALL-THROUGH)

Ako zaboraviš `break`, izvršavanje **nastavlja u sljedeći `case`**, bez provjere! To je čest izvor grešaka — ali ponekad i namjerno koristimo, kad više slučajeva treba isti kod:

```cpp
char c = 'e';
switch (c) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
        cout << "samoglasnik\n";
        break;
    default:
        cout << "suglasnik ili nesto drugo\n";
}
```

4.5 Ternarni operator `?` `:`

Kratki oblik za „ako–onda–inače“ koji **vraća vrijednost**:

```
int veci = (a > b) ? a : b;                 // ako a > b, onda a, inače b
cout << (n % 2 == 0 ? "paran" : "neparan");
```

Koristi ga za jednostavne izbore vrijednosti. Ugniježđeni ternarni operatori (`a ? b : c ? d : e`) brzo postaju nečitljivi — tada je `if` bolji izbor.

4.6 Blokovi i doseg varijabli

Varijabla deklarirana unutar bloka `{ }` postoji **samo unutar tog bloka**. Kad izvršavanje izađe iz bloka, varijabla nestaje:

✗ NEISPRAVNO

```
if (x > 0) {
    int kvadrat = x * x;
}
cout << kvadrat;   // GREŠKA: kvadrat ovdje više ne postoji
```

Ako ti varijabla treba i nakon bloka, deklariraj je **prije** njega. Ovo pravilo o dosegu (*scope*) vrijedi za sve blokove — u petljama, funkcijama, bilo gdje — i vratit ćemo mu se u poglavlju o funkcijama.

UKRATKO

- `if / else if / else` — provjera ide redom, izvrši se prvi istinit blok.
- `&&` i `||` računaju se kratko: drugi uvjet se ne računa ako je rezultat već poznat. Koristi to za zaštitne provjere (`n != 0 && ...`).
- Zamke: `=` umjesto `==`, `;` iza `if`, izostavljene `{ }`, `0 < x < 10`.
- `switch` radi s cjelobrojnim vrijednostima i konstantama; ne zaboravi `break`.
- Varijabla živi samo unutar bloka u kojem je deklarirana.

Zadaci

Zadatak 4.1 ★☆☆

Korisnik unosi cijeli broj. Ispiši je li broj pozitivan, negativan ili nula, te je li paran ili neparan.

Zadatak 4.2 ★☆☆

Godina je prijestupna ako je djeljiva s 4, ali ne sa 100 — osim ako je djeljiva s 400. Napiši program koji za unesenu godinu ispisuje je li prijestupna, **koristeći jedan jedini uvjet** u `if`. Provjeri: 2024 da, 1900 ne, 2000 da, 2023 ne.

Zadatak 4.3 ★★☆

Korisnik unosi tri broja. Ispiši najveći od njih, a zatim ih ispiši sortirane od najmanjeg do najvećeg (bez petlji i bez gotovih funkcija za sortiranje).

Zadatak 4.4 ★★☆

Napiši kalkulator: korisnik unosi izraz u obliku `broj operator broj` (npr. `7.5 * 2`). Koristi `switch`. Program mora obraditi dijeljenje s nulom i nepoznat operator.

Zadatak 4.5 ★★☆

Za unesene duljine triju stranica provjeri mogu li činiti trokut (zbroj svake dvije stranice veći je od treće). Ako mogu, ispiši je li trokut jednakostraničan, jednakokračan ili raznostraničan.

Zadatak 4.6 ★★★

Napiši program za rješavanje kvadratne jednadžbe `ax² + bx + c = 0`. Obradi sve slučajeve: dva realna rješenja, jedno (dvostruko) rješenje, kompleksna rješenja (ispiši ih u obliku `re ± im i`) i slučaj `a = 0` (tada je jednadžba linearna — i ona ima svoje posebne slučajeve!).

5. Petlje

Petlje su razlog zašto su računala korisna: ponavljaju isti posao milijun puta bez umora i bez greške. U ovom poglavlju naučit ćemo tri vrste petlji, a zatim — što je važnije — **tipične obrasce** koji se ponavljaju u gotovo svakom zadatku. Kad ih jednom prepoznaš, polovica ispitnih zadataka svodi se na njihovo kombiniranje.

5.1 Petlja `while`

`while` ponavlja blok **dok god je uvjet istinit**. Uvjet se provjerava **prije** svakog prolaza — ako nije istinit već na početku, tijelo se ne izvrši nijednom.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    while (n > 0) {
        cout << n << " ";
        n--;                 // bez ovoga petlja nikad ne bi završila
    }
    cout << "Start!\n";
}
```

IZLAZ PROGRAMA

```
5 4 3 2 1 Start!
```

Svaka petlja mora imati nešto što je **približava kraju**. Ovdje je to `n--`. Ako to zaboraviš, dobiješ **beskonačnu petlju** — program „visi“ i moraš ga prekinuti (Ctrl+C).

`while` koristimo kad **unaprijed ne znamo** koliko će biti ponavljanja: „čitaj dok korisnik ne unese 0“, „dijeli dok broj ne postane 0“, „ponavljaj dok greška ne padne ispod 0.001“.

5.2 Petlja `do-while`

Isto kao `while`, ali se uvjet provjerava **nakon** tijela. Zato se tijelo izvrši **barem jednom**. Idealna je za izbornike i provjeru unosa, gdje prvo moraš nešto pitati da bi znao treba li ponoviti:

```
int izbor;
do {
    cout << "1) Novi unos  2) Ispis  0) Kraj\nIzbor: ";
    cin >> izbor;
    // ... obrada izbora ...
} while (izbor != 0);   // pazi na točku-zarez na kraju!
```

5.3 Petlja `for`

`for` je petlja za situacije kad **znamo koliko puta** nešto treba ponoviti (ili prolazimo kroz niz vrijednosti). Sve što upravlja petljom stoji na jednom mjestu:

```
for (inicijalizacija; uvjet; korak) {
    tijelo;
}
```

Redoslijed izvršavanja je važan i često se pita na ispitu:

1. **inicijalizacija** — izvrši se *samo jednom*, na početku;
2. provjeri se **uvjet** — ako nije istinit, petlja završava;
3. izvrši se **tijelo**;
4. izvrši se **korak**, pa se vraćamo na točku 2.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 10, zbroj = 0;
    for (int i = 1; i <= n; i++) {
        zbroj += i;
    }
    cout << "Zbroj 1.." << n << " = " << zbroj << "\n";

    for (int i = 10; i > 0; i -= 3) cout << i << " ";   // korak može biti bilo što
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
Zbroj 1..10 = 55
10 7 4 1
```

Varijabla `i` deklarirana u zaglavlju `for`-a postoji samo unutar petlje. To je dobro — ne „curi“ van i ne može se slučajno upotrijebiti kasnije.

Petlja	Koristi kad…	Tipičan primjer
`for`	znaš broj ponavljanja ili prolaziš kroz niz	`for (int i = 0; i < n; i++)`
`while`	ponavljaš dok ne nastupi neki uvjet	dijeljenje broja dok ne postane 0
`do-while`	tijelo se mora izvršiti barem jednom	izbornik, ponovni unos

Tehnički se svaka od njih može zamijeniti drugom. Ali pravi izbor čini kod čitljivijim — čitatelj odmah vidi *vrstu* ponavljanja.

5.4 `break` i `continue`

- `break` odmah **prekida** petlju i nastavlja iza nje.
- `continue` **preskače ostatak tijela** i ide na sljedeći prolaz (u `for`-u se korak ipak izvrši).

```cpp
#include <iostream>
using namespace std;

int main() {
    // prvi broj veći od 50 djeljiv sa 7
    for (int i = 51; ; i++) {         // bez uvjeta = beskonačna petlja...
        if (i % 7 == 0) {
            cout << "Prvi: " << i << "\n";
            break;                    // ...koju prekida break
        }
    }
    // ispiši brojeve 1..10 osim onih djeljivih s 3
    for (int i = 1; i <= 10; i++) {
        if (i % 3 == 0) continue;
        cout << i << " ";
    }
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
Prvi: 56
1 2 4 5 7 8 10
```

`break` u ugniježđenim petljama prekida **samo najunutarnju** petlju u kojoj se nalazi.

5.5 Ugniježđene petlje

Petlja unutar petlje: za **svaki** prolaz vanjske petlje, unutarnja se izvrši **cijela**. Ako vanjska ima `n`, a unutarnja `m` prolaza, tijelo se izvrši `n · m` puta.

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    for (int red = 1; red <= 4; red++) {
        for (int stup = 1; stup <= 5; stup++) {
            cout << setw(4) << red * stup;
        }
        cout << "\n";                 // novi red nakon svakog reda tablice
    }
    // trokut: u redu i ispisujemo i zvjezdica
    for (int i = 1; i <= 4; i++) {
        for (int j = 1; j <= i; j++) cout << '*';
        cout << "\n";
    }
}
```

IZLAZ PROGRAMA

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
*
**
***
****
```

> **SAVJET: KAKO RJEŠAVATI ZADATKE S UZORCIMA**
>
> Zadaci tipa „ispiši trokut/romb/šahovnicu“ uvijek se rješavaju isto: vanjska petlja ide po **redovima**, a za svaki red zapitaš se *koliko čega* treba ispisati — razmaka, zvjezdica — **kao funkcija broja reda**. Napiši mali primjer na papiru (red 1: 3 razmaka i 1 zvjezdica, red 2: 2 razmaka i 3 zvjezdice...), pronađi formulu, i tek onda piši kod.

5.6 Tipični obrasci

Ovo je najvažniji dio poglavlja. Sljedeći obrasci pojavljuju se stalno — nauči ih kao „riječi“ od kojih gradiš rješenja.

Akumulator i brojač. Varijabla koja skuplja rezultat kroz petlju. Mora biti inicijalizirana **prije** petlje: zbroj na 0, umnožak na 1, brojač na 0.

Minimum i maksimum. Početnu vrijednost uzmi iz **prvog podatka** (ili `INT_MAX` za minimum, odnosno `INT_MIN` za maksimum), nikako 0 — jer što ako su svi brojevi negativni?

```cpp
#include <iostream>
using namespace std;

int main() {
    // čita brojeve do 0; ispisuje min, max i prosjek
    int x, n = 0, zbroj = 0, mn = 0, mx = 0;
    while (cin >> x && x != 0) {
        if (n == 0 || x < mn) mn = x;    // prvi broj postaje početni min/max
        if (n == 0 || x > mx) mx = x;
        zbroj += x;
        n++;
    }
    if (n > 0)
        cout << "min=" << mn << " max=" << mx
             << " prosjek=" << static_cast<double>(zbroj) / n << "\n";
}
```

IZLAZ PROGRAMA

```
min=-2 max=9 prosjek=3.75
```

Rastavljanje broja na znamenke. `n % 10` daje zadnju znamenku, a `n /= 10` je „odreže“. Ponavljaj dok broj ne postane 0:

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 90417;
    int zbroj = 0, brojZnamenki = 0, obrnuto = 0;
    int k = n;                          // radimo na kopiji, n ostaje sačuvan
    while (k > 0) {
        int z = k % 10;                 // zadnja znamenka
        zbroj += z;
        obrnuto = obrnuto * 10 + z;     // "dodaj znamenku na kraj"
        brojZnamenki++;
        k /= 10;                        // odreži zadnju znamenku
    }
    cout << zbroj << " " << brojZnamenki << " " << obrnuto << "\n";
}
```

IZLAZ PROGRAMA

```
21 5 71409
```

Pazi na rubni slučaj: za `n = 0` petlja se ne izvrši nijednom, pa bi broj znamenki bio 0 umjesto 1. Rubni slučajevi (0, negativni brojevi, prazan unos) su ono što razlikuje „radi na primjeru“ od „radi“.

Je li broj prost? Dovoljno je provjeriti djelitelje do √n: ako `n` ima djelitelj veći od √n, onda ima i odgovarajući manji. Uvjet `d * d <= n` izbjegava rad s korijenom (i decimalnim brojevima):

```cpp
#include <iostream>
using namespace std;

int main() {
    for (int n = 1; n <= 30; n++) {
        bool prost = (n >= 2);          // 0 i 1 nisu prosti
        for (int d = 2; d * d <= n; d++) {
            if (n % d == 0) { prost = false; break; }
        }
        if (prost) cout << n << " ";
    }
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
2 3 5 7 11 13 17 19 23 29
```

Primijeti obrazac „**zastavica**“ (*flag*): pretpostavimo da je broj prost (`prost = true`), a petlja traži protuprimjer. Čim ga nađe, spušta zastavicu i izlazi.

Najveći zajednički djelitelj (Euklidov algoritam). Jedan od najstarijih algoritama (oko 300. g. pr. Kr.): `NZD(a, b) = NZD(b, a % b)`, a kad `b` postane 0, rezultat je `a`.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 48, b = 18;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    cout << "NZD = " << a << "\n";
}
```

IZLAZ PROGRAMA

```
NZD = 6
```

Koraci: (48, 18) → (18, 12) → (12, 6) → (6, 0) → rezultat 6. Najmanji zajednički višekratnik dobiva se iz NZD-a: `NZV(a, b) = a / NZD(a, b) * b`.

5.7 Česte greške s petljama

- **Pogreška za jedan (off-by-one).** `for (int i = 0; i <= n; i++)` prolazi `n + 1` puta, a ne `n`. Pitaj se uvijek: koja je *prva* i koja *zadnja* vrijednost brojača? Za n ponavljanja idiomatski se piše `for (int i = 0; i < n; i++)`.
- **Točka-zarez iza zaglavlja:** `for (int i = 0; i < n; i++);` — tijelo petlje je prazna naredba, a blok ispod izvrši se jednom.
- **Mijenjanje brojača u tijelu `for`-a.** Tehnički dopušteno, ali gotovo uvijek znak zbrkane logike.
- **Preljev u akumulatoru.** Faktorijel od 13 već ne stane u `int`. Za velike umnoške koristi `long long`.
- **Neinicijaliziran akumulator** (`int suma;`) — vidi poglavlje 2.

UKRATKO

- `while` — provjera prije tijela; `do-while` — tijelo barem jednom; `for` — poznat broj ponavljanja.
- `for`: inicijalizacija (jednom) → uvjet → tijelo → korak → uvjet → ...
- `break` prekida petlju, `continue` preskače na sljedeći prolaz.
- Obrasci: akumulator, brojač, min/max (početna vrijednost iz prvog podatka!), znamenke (`%10`, `/10`), zastavica, Euklid.
- Uvijek provjeri rubne slučajeve: 0, 1, negativne brojeve, prazan unos.

Zadaci

Zadatak 5.1 ★☆☆

Korisnik unosi `n`. Ispiši zbroj svih parnih brojeva od 1 do `n` i broj brojeva djeljivih s 3 u tom rasponu.

Zadatak 5.2 ★☆☆

Izračunaj faktorijel unesenog broja `n` ($0 \le n \le 20$). Koji tip varijable trebaš i zašto? Što se dogodi za `n = 21`?

Zadatak 5.3 ★★☆

Korisnik unosi pozitivan cijeli broj. Provjeri je li **palindrom** (čita se jednako s obje strane, npr. 12321) — bez pretvaranja u string.

Zadatak 5.4 ★★☆

Ispiši sve savršene brojeve manje od 10000. Broj je savršen ako je jednak zbroju svojih pravih djelitelja (npr. 6 = 1 + 2 + 3).

Zadatak 5.5 ★★☆

Igra pogađanja: program „zamisli“ broj od 1 do 100 (za početak ga zapiši kao konstantu), a korisnik pogađa. Nakon svakog pokušaja program kaže „vise“ ili „manje“. Na kraju ispiši broj pokušaja. Koju petlju je najprirodnije koristiti?

Zadatak 5.6 ★★★

Za unesen neparan `n` ispiši romb od zvjezdica visine `n`. Za `n = 5`:

```
  *
 ***
*****
 ***
  *
```

Zadatak 5.7 ★★★

Collatzov niz: kreni od broja `n`; ako je paran, podijeli ga s 2, inače ga pomnoži s 3 i dodaj 1. Ponavljaj dok ne dođeš do 1. Pronađi broj do 10000 koji ima **najdulji** takav niz i ispiši ga zajedno s duljinom niza. (Pazi: međurezultati mogu prijeći raspon `int`-a.)

6. Funkcije

Funkcija je imenovani komad koda koji obavlja jedan posao. Zvuči jednostavno, ali funkcije su možda **najvažniji alat za razmišljanje** u programiranju: omogućuju da veliki problem razbiješ na male, od kojih svaki možeš razumjeti, napisati i testirati zasebno. U ovom poglavlju posebnu pažnju posvetit ćemo **prijenosu parametara** (po vrijednosti i po referenci) — to je temelj za sve što slijedi.

6.1 Anatomija funkcije

```cpp
#include <iostream>
using namespace std;

//  povratni tip   ime      parametri
    double         povrsinaKruga(double r) {
        const double PI = 3.14159265358979;
        return r * r * PI;        // vraća vrijednost pozivatelju
    }

int main() {
    double p = povrsinaKruga(2.0);           // poziv; 2.0 je argument
    cout << p << "\n";
    cout << povrsinaKruga(1.0) + povrsinaKruga(3.0) << "\n";
}
```

IZLAZ PROGRAMA

```
12.5664
31.4159
```

- **Povratni tip** — tip vrijednosti koju funkcija vraća. Ako ne vraća ništa, piše se `void`.
- **Ime** — glagol ili opis onoga što radi: `izracunajProsjek`, `jeProst`, `ispisiIzbornik`.
- **Parametri** — varijable koje primaju vrijednosti izvana. **Argumenti** su konkretne vrijednosti proslijeđene pri pozivu.
- `return` — završava funkciju *odmah* i vraća vrijednost. Funkcija može imati više `return` naredbi.

ZAŠTO PISATI FUNKCIJE?

1) **Ne ponavljaš se** — kod napisan jednom koristiš sto puta, a grešku ispravljaš na jednom mjestu. 2) **Apstrakcija** — kad jednom napišeš `jeProst(n)`, više ne razmišljaš *kako* radi, nego samo *što* radi. 3) **Čitljivost** — `main` postaje popis koraka umjesto zida koda. 4) **Testiranje** — malu funkciju lako je provjeriti zasebno. Pravilo iz prakse: funkcija bi trebala raditi **jednu stvar** i stati na jedan ekran.

6.2 Deklaracija i definicija

Prevoditelj čita datoteku odozgo prema dolje i funkciju mora „znati“ prije nego što je pozoveš. Ako želiš `main` na vrhu, a funkcije ispod, napiši gore **deklaraciju** (prototip) — samo zaglavlje sa `;`:

```cpp
#include <iostream>
using namespace std;

bool jeProst(int n);            // DEKLARACIJA: "ova funkcija postoji"

int main() {
    for (int i = 1; i <= 20; i++)
        if (jeProst(i)) cout << i << " ";
    cout << "\n";
}

bool jeProst(int n) {           // DEFINICIJA: tijelo funkcije
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return false;   // čim nađemo djelitelj — gotovo
    return true;
}
```

IZLAZ PROGRAMA

```
2 3 5 7 11 13 17 19
```

Primijeti kako `return` usred petlje elegantno zamjenjuje zastavicu iz prošlog poglavlja. Deklaracije su ključne i kod podjele programa u više datoteka (poglavlje 12): deklaracije idu u `.h`, definicije u `.cpp`.

6.3 Prijenos po vrijednosti

Po zadanom, C++ parametre prenosi **po vrijednosti**: funkcija dobiva **kopiju** argumenta. Što god radila s tom kopijom, original se ne mijenja.

```cpp
#include <iostream>
using namespace std;

void zamijeni(int a, int b) {    // a i b su KOPIJE
    int t = a; a = b; b = t;
}

int main() {
    int x = 1, y = 2;
    zamijeni(x, y);
    cout << x << " " << y << "\n";   // ništa se nije promijenilo!
}
```

IZLAZ PROGRAMA

```
1 2
```

Funkcija je zamijenila svoje kopije, koje su nestale čim je završila. `x` i `y` u `main`-u nitko nije ni dotaknuo.

6.4 Prijenos po referenci

Referenca je drugo ime (nadimak, *alias*) za postojeću varijablu. Označava se znakom `&` iza tipa. Parametar-referenca nije kopija — to je **ista varijabla** pod drugim imenom:

```cpp
#include <iostream>
using namespace std;

void zamijeni(int& a, int& b) {  // a i b su DRUGA IMENA za x i y
    int t = a; a = b; b = t;
}

int main() {
    int x = 1, y = 2;
    zamijeni(x, y);
    cout << x << " " << y << "\n";
}
```

IZLAZ PROGRAMA

```
2 1
```

ANALOGIJA: FOTOKOPIJA I ORIGINAL

Prijenos po vrijednosti je kao da nekome daš **fotokopiju** dokumenta — može po njoj šarati koliko hoće, tvoj original ostaje čist. Prijenos po referenci je kao da mu daš **original**: sve što napiše, vidjet ćeš i ti.

Konstantna referenca (`const T&`). Kopiranje `int`-a je jeftino, ali kopiranje stringa od milijun znakova ili vektora s milijun elemenata nije. Kad funkcija samo *čita* velik objekt, prenosi ga po `const` referenci: nema kopije (brzo), a `const` jamči da ga funkcija neće promijeniti (sigurno).

```cpp
// loše: kopira cijeli vektor pri svakom pozivu
double prosjek(vector<double> v);
// dobro: bez kopije, i obećanje da ga ne mijenjamo
double prosjek(const vector<double>& v);
```

Način	Sintaksa	Kad koristiti
po vrijednosti	`void f(int x)`	mali tipovi (`int`, `double`, `char`, `bool`) koje samo čitaš
po referenci	`void f(int& x)`	funkcija **treba promijeniti** argument
po const referenci	`void f(const string& s)`	veliki objekti (`string`, `vector`, klase) koje samo čitaš

REFERENCU NE MOŽEŠ VEZATI NA PRIVREMENU VRIJEDNOST

Ako je parametar `int& x`, poziv `f(5)` ili `f(a + b)` neće se prevesti — ne postoji varijabla koju bi funkcija mogla mijenjati. S `const int&` to je dopušteno. I još nešto: **nikad ne vraćaj referencu na lokalnu varijablu** funkcije — ona nestaje kad funkcija završi, pa bi referenca pokazivala u prazno.

Vraćanje više vrijednosti. Funkcija vraća samo jednu vrijednost preko `return`. Ako treba vratiti više, možeš koristiti reference kao „izlazne“ parametre, ili (moderniji način) vratiti strukturu ili `pair` — o tome u poglavljima 11 i 18.

```cpp
#include <iostream>
using namespace std;

void minMax(int a, int b, int c, int& mn, int& mx) {
    mn = min(a, min(b, c));
    mx = max(a, max(b, c));
}

int main() {
    int najmanji, najveci;
    minMax(7, 2, 9, najmanji, najveci);
    cout << najmanji << " " << najveci << "\n";
}
```

IZLAZ PROGRAMA

```
2 9
```

6.5 Zadani argumenti i preopterećenje

Zadani argumenti (*default arguments*) daju parametru vrijednost ako je pozivatelj izostavi. Moraju biti zdesna — iza parametra sa zadanom vrijednošću ne smije doći parametar bez nje:

```cpp
void ispisiLiniju(int duljina = 20, char znak = '-');

ispisiLiniju();          // 20 crtica
ispisiLiniju(5);         // 5 crtica
ispisiLiniju(5, '*');    // 5 zvjezdica
```

Preopterećenje funkcija (*overloading*): više funkcija smije imati **isto ime** ako se razlikuju po **parametrima** (broju ili tipovima). Prevoditelj bira pravu prema argumentima:

```cpp
#include <iostream>
using namespace std;

double povrsina(double r)             { return 3.14159 * r * r; }  // krug
double povrsina(double a, double b)   { return a * b; }            // pravokutnik
int    povrsina(int a)                { return a * a; }            // kvadrat

int main() {
    cout << povrsina(1.0) << " " << povrsina(2.0, 3.0) << " " << povrsina(4) << "\n";
}
```

IZLAZ PROGRAMA

```
3.14159 6 16
```

Funkcije se **ne mogu** razlikovati samo po povratnom tipu — iz poziva `f(5)` prevoditelj ne bi znao koju želiš.

6.6 Doseg i životni vijek varijabli

- **Lokalne varijable** (deklarirane u funkciji ili bloku) nastaju kad izvršavanje uđe u blok i nestaju kad iz njega izađe. Svaki poziv funkcije dobiva *nove* lokalne varijable.
- **Globalne varijable** (izvan svih funkcija) žive cijelo vrijeme i vidljive su svugdje. Zvuči praktično, ali to je problem: bilo koja funkcija ih može promijeniti, pa je teško pratiti tko je što pokvario. **Izbjegavaj globalne varijable** (globalne *konstante* su u redu).
- **Statičke lokalne varijable** (`static int brojac = 0;`) vidljive su samo u funkciji, ali **pamte vrijednost između poziva** — inicijaliziraju se samo jednom.

```cpp
#include <iostream>
using namespace std;

void pozdrav() {
    static int brojPoziva = 0;    // inicijalizira se samo pri PRVOM pozivu
    brojPoziva++;
    cout << "Poziv broj " << brojPoziva << "\n";
}

int main() {
    pozdrav(); pozdrav(); pozdrav();
}
```

IZLAZ PROGRAMA

```
Poziv broj 1
Poziv broj 2
Poziv broj 3
```

6.7 Rekurzija

Funkcija je **rekurzivna** ako poziva samu sebe. Svaka rekurzija mora imati dva dijela:

1. **bazni slučaj** — dovoljno jednostavan da se riješi izravno, bez novog poziva;
2. **rekurzivni korak** — problem se svodi na *manji* problem iste vrste.

```cpp
#include <iostream>
using namespace std;

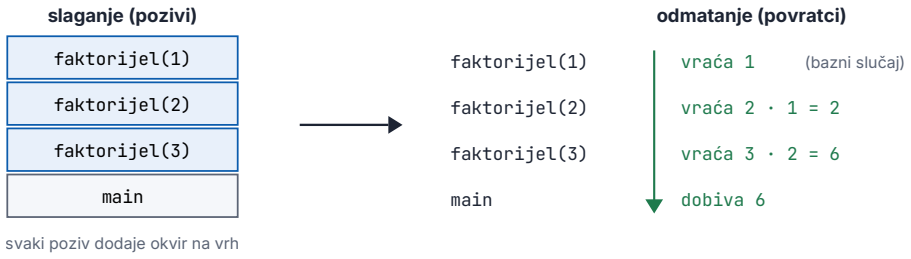
long long faktorijel(int n) {
    if (n <= 1) return 1;              // bazni slučaj
    return n * faktorijel(n - 1);      // rekurzivni korak: n! = n * (n-1)!
}

int main() {
    cout << faktorijel(5) << " " << faktorijel(20) << "\n";
}
```

IZLAZ PROGRAMA

```
120 2432902008176640000
```

Kako to računalo zapravo izvršava? Svaki poziv funkcije dobiva svoj **okvir** (*stack frame*) na **stogu poziva** — komadu memorije u kojem žive njegovi parametri i lokalne varijable. Okviri se slažu jedan na drugi, a kad poziv završi, njegov okvir se skida:



Slika 6.1 — Stog poziva za faktorijel(3): pozivi se slažu, a rezultati se vraćaju odozgo prema dolje

Ako rekurzija nema bazni slučaj (ili ga nikad ne dosegne), okviri se slažu dok stog ne ponestane — program se sruši s greškom **stack overflow**.

REKURZIJA NIJE UVIJEK DOBRA IDEJA

Naivni Fibonacci `fib(n) = fib(n-1) + fib(n-2)` izgleda elegantno, ali isti broj računa iznova i iznova — broj poziva raste eksponencijalno pa `fib(50)` traje minutama. Petlja to izračuna trenutačno. Rekurzija je prirodna za probleme koji su *sami po sebi* rekurzivni: stabla, podjela na pola (binarno pretraživanje, *merge sort*), pretraživanje s povratkom. Za jednostavna brojanja koristi petlje.

UKRATKO

- Funkcija: povratni tip, ime, parametri, tijelo. `void` = ne vraća ništa. `return` odmah završava funkciju.
- Funkcija mora biti deklarirana prije poziva (prototip gore, definicija dolje).
- **Po vrijednosti** = kopija, original netaknut. **Po referenci** (`&`) = isti objekt, promjene se vide vani. `const&` = bez kopije i bez promjena — za velike objekte.
- Preopterećenje: isto ime, različiti parametri. Zadani argumenti idu zdesna.
- Lokalne varijable žive dok traje poziv; `static` lokalne pamte vrijednost između poziva. Globalne varijable izbjegavaj.
- Rekurzija: bazni slučaj + svođenje na manji problem. Svaki poziv ima svoj okvir na stogu.

Zadaci

Zadatak 6.1 ★☆☆

Napiši funkciju `int zbrojZnamenki(int n)` i funkciju `bool jeParan(int n)`. U `main`-u ispiši sve brojeve od 1 do 100 kojima je zbroj znamenki paran.

Zadatak 6.2 ★☆☆

Napiši funkciju `void udvostruci(int& x)` koja udvostručuje vrijednost argumenta. Zatim napiši verziju `int udvostruceno(int x)` koja vraća rezultat. Objasni razliku u načinu korištenja.

Zadatak 6.3 ★★☆

Napiši funkcije `int nzd(int a, int b)` i `int nzv(int a, int b)`, a zatim funkciju `void skratiRazlomak(int& brojnik, int& nazivnik)` koja skraćuje razlomak do kraja. Za 84/36 treba dobiti 7/3.

Zadatak 6.4 ★★☆

Napiši **rekurzivne** funkcije: (a) zbroj znamenki broja, (b) potencija `x^n` za cijeli `n ≥ 0`, (c) ispis broja unatrag (za 1234 ispisuje 4321).

Zadatak 6.5 ★★☆

Što ispisuje sljedeći program? Odgovori bez pokretanja, pa provjeri.

```
int f(int a, int& b) { a += 10; b += 10; return a + b; }

int main() {
    int x = 1, y = 2;
    int z = f(x, y);
    cout << x << " " << y << " " << z << "\n";
}
```

Zadatak 6.6 ★★★

Napiši brzu rekurzivnu funkciju za potenciranje koja koristi pravilo `x^n = (x^(n/2))²` za paran `n`, odnosno `x · x^(n-1)` za neparan. Koliko otprilike poziva treba za `n = 1 000 000` u usporedbi s naivnom verzijom iz 6.4?

Zadatak 6.7 ★★★

Hanojski tornjevi: napiši rekurzivnu funkciju `void hanoi(int n, char s, char c, char p)` koja ispisuje poteze za premještanje `n` diskova sa štapa `s` na štap `c` uz pomoćni štap `p`. Koliko poteza treba za `n` diskova?

7. Polja, vektori i osnovni algoritmi

Do sada je svaka varijabla čuvala jednu vrijednost. Ali što ako trebamo ocjene 30 studenata? Trideset varijabli `ocjena1`, `ocjena2`... nije rješenje. Trebamo **niz** vrijednosti istog tipa pod jednim imenom. C++ za to ima klasično **polje** (*array*, naslijeđeno iz C-a) i `std::vector` — moderni, sigurniji i fleksibilniji niz koji ćeš u praksi koristiti najčešće.

7.1 Klasično polje

```
int ocjene[5];                      // 5 int-ova, vrijednosti neinicijalizirane
int a[5] = {4, 2, 5, 3, 5};         // inicijalizacija
int b[5] = {1, 2};                  // ostatak se puni nulama: {1, 2, 0, 0, 0}
int c[5] = {};                      // sve nule
int d[] = {7, 8, 9};                // veličinu (3) određuje prevoditelj
```

Elementi polja leže u memoriji **jedan do drugoga**, bez razmaka. Pristupa im se **indeksom** u uglatim zagradama, a indeksi kreću **od nule**: prvi element je `a[0]`, a zadnji `a[4]` — ne `a[5]`!



Slika 7.1 — Polje od 5 elemenata: indeksi od 0 do 4

ZAŠTO INDEKSI POČINJU OD NULE?

Indeks nije „redni broj“, nego **pomak od početka polja**. Element `a[i]` nalazi se na adresi `početak + i · veličina_elementa`. Prvi element je na samom početku, dakle pomak je 0. Kad ovo shvatiš, indeksiranje od nule postaje logično — i razumjet ćeš pokazivače u poglavlju 9.

Polje se gotovo uvijek obrađuje petljom:

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int a[N] = {4, 2, 5, 3, 5};
    int zbroj = 0;
    for (int i = 0; i < N; i++) zbroj += a[i];
    cout << "Prosjek: " << static_cast<double>(zbroj) / N << "\n";

    for (int x : a) cout << x << " ";     // range-for: "za svaki x u a"
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
Prosjek: 3.8
4 2 5 3 5
```

Petlja **range-for** (`for (int x : a)`) prolazi kroz sve elemente bez indeksa — kraća je i nema mjesta za pogrešku za jedan. Pazi: `x` je *kopija* elementa. Ako elemente želiš mijenjati, piši `for (int& x : a)`.

IZLAZAK IZVAN GRANICA POLJA

C++ **ne provjerava** indekse. `a[5]` ili `a[-1]` na polju od 5 elemenata prevest će se bez greške i čitati (ili, još gore, pisati po) memoriji koja pripada nečemu drugom. To je nedefinirano ponašanje — program se može srušiti, ali može i tiho pokvariti neku drugu varijablu. Klasičan uzrok je `for (int i = 0; i <= N; i++)`. Uvijek: `i < N`.

Veličina klasičnog polja mora biti poznata **pri prevođenju** (konstanta) i ne može se mijenjati. Ako ne znaš unaprijed koliko će biti podataka — trebaš `vector`.

7.2 Polja i funkcije

Kad polje predaš funkciji, **ne kopira se**: funkcija dobiva adresu prvog elementa (detalje ćemo vidjeti kod pokazivača). Dvije važne posljedice:

1. funkcija **može mijenjati** elemente originalnog polja;
2. funkcija **ne zna veličinu** polja — moraš je poslati kao zaseban parametar.

```cpp
#include <iostream>
using namespace std;

void udvostruci(int a[], int n) {       // int a[] je zapravo int* a
    for (int i = 0; i < n; i++) a[i] *= 2;
}

int main() {
    int p[4] = {1, 2, 3, 4};
    udvostruci(p, 4);
    for (int x : p) cout << x << " ";    // promjena se vidi u main-u
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
2 4 6 8
```

7.3 Dvodimenzionalna polja (matrice)

```cpp
#include <iostream>
using namespace std;

int main() {
    const int R = 3, S = 3;
    int m[R][S] = { {1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9} };
    int trag = 0;
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < S; j++) cout << m[i][j] << " ";
        cout << "\n";
        trag += m[i][i];                 // glavna dijagonala: i == j
    }
    cout << "Trag: " << trag << "\n";
}
```

IZLAZ PROGRAMA

```
1 2 3
4 5 6
7 8 9
Trag: 15
```

Prvi indeks je **redak**, drugi **stupac**. U memoriji su reci zapisani jedan za drugim (prvo cijeli redak 0, pa redak 1…). Kod matrica se često pitaju dijagonale: glavna ima `i == j`, sporedna `i + j == n - 1`; elementi iznad glavne dijagonale imaju `j > i`.

7.4 `std::vector` — polje koje raste

`vector` je iz zaglavlja `<vector>` i predstavlja **dinamičko polje**: zna svoju veličinu, može rasti i smanjivati se, sam upravlja memorijom, i ponaša se kao normalna vrijednost (može se kopirati, vratiti iz funkcije, usporediti). U modernom C++-u to je **zadani izbor za niz podataka**.

```cpp
#include <iostream>
#include <vector>
using namespace std;

double prosjek(const vector<int>& v) {        // const& — bez kopije
    if (v.empty()) return 0;
    int zbroj = 0;
    for (int x : v) zbroj += x;
    return static_cast<double>(zbroj) / v.size();
}

int main() {
    vector<int> ocjene;                  // prazan vektor
    ocjene.push_back(5);                 // dodaj na kraj
    ocjene.push_back(3);
    ocjene.push_back(4);
    cout << "Broj ocjena: " << ocjene.size() << "\n";
    cout << "Prva: " << ocjene[0] << ", zadnja: " << ocjene.back() << "\n";
    cout << "Prosjek: " << prosjek(ocjene) << "\n";

    vector<int> nule(5, 0);              // 5 elemenata, svi 0
    vector<int> v = {3, 1, 4, 1, 5};     // inicijalizacijska lista
    v.pop_back();                        // ukloni zadnji element
    cout << "v ima " << v.size() << " elementa\n";
}
```

IZLAZ PROGRAMA

```
Broj ocjena: 3
Prva: 5, zadnja: 4
Prosjek: 4
v ima 4 elementa
```

Operacija	Značenje
`v.push_back(x)`	dodaj `x` na kraj
`v.pop_back()`	ukloni zadnji element
`v.size()`, `v.empty()`	broj elemenata; je li prazan
`v[i]`	pristup bez provjere granica (brzo)
`v.at(i)`	pristup **s provjerom** — baca iznimku za krivi indeks
`v.front()`, `v.back()`	prvi i zadnji element
`v.clear()`	ukloni sve elemente
`v.resize(n)`	promijeni veličinu na `n`

Matrica s vektorima: `vector<vector<int>> m(R, vector<int>(S, 0));` — vektor od `R` redaka, svaki s `S` nula. Pristup je isti kao kod polja: `m[i][j]`.

SIZE() VRAĆA UNSIGNED TIP

`v.size()` vraća `size_t`, nepredznačen tip. Zato je `v.size() - 1` za prazan vektor ogroman broj (a ne −1), pa petlja `for (int i = 0; i < v.size() - 1; i++)` na praznom vektoru „poludi“. Sigurnije je pisati `i + 1 < v.size()`. Prevoditelj s `-Wall` upozorava i na usporedbu `int` i `size_t` — to upozorenje shvati ozbiljno.

Vektor je detaljno obrađen u poglavlju 18 (kapacitet, umetanje, brisanje). Za sada je dovoljno: **kad ne znaš broj elemenata unaprijed, koristi `vector` i `push_back`.**

7.5 Osnovni algoritmi nad nizovima

Linearno pretraživanje — prođi redom dok ne nađeš traženi element. Radi na bilo kakvom nizu. Za `n` elemenata treba najviše `n` usporedbi.

Binarno pretraživanje — radi samo na **sortiranom** nizu. Pogledaj srednji element: ako je traženi manji, nastavi u lijevoj polovici, inače u desnoj. Svakim korakom prepolovi se područje, pa za milijun elemenata treba samo oko 20 koraka!

```
#include <iostream>
#include <vector>
using namespace std;

int linearno(const vector<int>& v, int x) {
    for (size_t i = 0; i < v.size(); i++)
        if (v[i] == x) return i;
    return -1;                              // nije pronađen
}

int binarno(const vector<int>& v, int x) {   // v MORA biti sortiran
    int lo = 0, hi = v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;        // sprječava preljev kod (lo+hi)
        if (v[mid] == x) return mid;
        if (v[mid] < x) lo = mid + 1;        // traži desno
        else            hi = mid - 1;        // traži lijevo
    }
    return -1;
}

int main() {
    vector<int> v = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    cout << linearno(v, 23) << " " << binarno(v, 23) << " " << binarno(v, 7) << "\n";
}
```

IZLAZ PROGRAMA

```
5 5 -1
```

Sortiranje mjehurićem (bubble sort) — prolazi kroz niz i zamjenjuje susjedne elemente koji su u krivom redoslijedu. Nakon prvog prolaza najveći element „ispliva“ na kraj, nakon drugog drugi najveći, itd. **Sortiranje izborom (selection sort)** — u svakom koraku pronađi najmanji element u nesortiranom dijelu i stavi ga na početak tog dijela.

```
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& v) {
    int n = v.size();
    for (int prolaz = 0; prolaz < n - 1; prolaz++) {
        bool zamjena = false;
```

```
        for (int j = 0; j < n - 1 - prolaz; j++) {  // zadnjih 'prolaz' je već složeno
            if (v[j] > v[j + 1]) {
                swap(v[j], v[j + 1]);
                zamjena = true;
            }
        }
        if (!zamjena) break;     // nije bilo zamjena -> niz je već sortiran
    }
}

void selectionSort(vector<int>& v) {
    int n = v.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (v[j] < v[minIdx]) minIdx = j;
        swap(v[i], v[minIdx]);
    }
}

int main() {
    vector<int> a = {5, 1, 4, 2, 8}, b = a;
    bubbleSort(a);
    selectionSort(b);
    for (int x : a) cout << x << " ";
    cout << "| ";
    for (int x : b) cout << x << " ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
1 2 4 5 8 | 1 2 4 5 8
```

Složenost. Koliko posla algoritam radi kad niz naraste? To opisujemo „velikim O“:

Algoritam	Složenost	Za n = 1 000 000 otprilike
binarno pretraživanje	O(log n)	20 koraka
linearno pretraživanje	O(n)	1 000 000 koraka
dobro sortiranje (`std::sort`)	O(n log n)	20 000 000 koraka
bubble / selection sort	$O(n^2)$	1 000 000 000 000 koraka (!)

SAVJET: U PRAKSI KORISTI STD::SORT

Bubble i selection sort učimo jer su jednostavni i dobra su vježba petlji i indeksa — i jer se traže na ispitima. U stvarnom kodu pišeš `sort(v.begin(), v.end());` iz `<algorithm>` (poglavlje 19), koji je višestruko brži i provjeren.

UKRATKO

- Polje: niz elemenata istog tipa, jedan do drugog u memoriji. Indeksi od `0` do `n-1`. Izlazak izvan granica = nedefinirano ponašanje.
- Polje u funkciji nije kopija i ne zna svoju veličinu — veličinu šalješ zasebno.
- `vector` raste po potrebi i zna svoju veličinu — zadani izbor u modernom C++-u. Prenosi se po `const&`.
- `for (int x : v)` — kopija; `for (int& x : v)` — mijenjanje elemenata.
- Binarno pretraživanje: samo na sortiranom nizu, O(log n). Bubble/selection sort: $O(n^2)$.

Zadaci

Zadatak 7.1 ★☆☆

Korisnik unosi `n`, a zatim `n` cijelih brojeva u vektor. Ispiši ih obrnutim redoslijedom, a zatim ispiši sve brojeve veće od prosjeka.

Zadatak 7.2 ★★☆

Napiši funkciju koja vraća **drugi najveći** element vektora u samo **jednom** prolazu kroz vektor. Što ako su svi elementi jednaki?

Zadatak 7.3 ★★☆

Napiši funkciju `void rotiraj(vector<int>& v, int k)` koja rotira vektor udesno za `k` mjesta: `{1,2,3,4,5}` za `k = 2` postaje `{4,5,1,2,3}`. Radi li ispravno i za `k` veći od veličine vektora?

Zadatak 7.4 ★★☆

Za matricu `n × n` unesenu s tipkovnice ispiši: zbroj svakog retka, zbroj svakog stupca, zbroj sporedne dijagonale i je li matrica simetrična (`m[i][j] == m[j][i]` za sve `i`, `j`).

Zadatak 7.5 ★★☆

Izbaci duplikate iz **sortiranog** vektora (npr. `{1,1,2,3,3,3,7}` → `{1,2,3,7}`) bez korištenja pomoćnog vektora.

Zadatak 7.6 ★★★

Napiši funkciju koja **spaja dva sortirana vektora** u jedan sortirani vektor u linearnom vremenu (bez ponovnog sortiranja). Ovo je srce algoritma *merge sort*.

8. Stringovi

Tekst je posvuda: imena, poruke, datoteke, podaci s weba. C++ za rad s tekstom ima klasu `std::string` iz zaglavlja `<string>`. Ona je, u biti, pametan niz znakova (`char`-ova) koji sam upravlja svojom memorijom. Ako razumiješ vektore, razumiješ i stringove — a ovdje ćemo proći metode koje se najčešće koriste.

8.1 Osnove

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string ime = "Ana";
    string prezime("Horvat");
    string puno = ime + " " + prezime;     // spajanje (konkatenacija)
    puno += "!";                           // dodavanje na kraj

    cout << puno << "\n";
    cout << "Duljina: " << puno.length() << "\n";   // isto što i size()
    cout << "Prvi znak: " << puno[0] << "\n";
    cout << "Zadnji znak: " << puno.back() << "\n";

    puno[0] = 'E';                         // stringovi su promjenjivi
    cout << puno << "\n";
    cout << (ime == "Ana") << " " << (ime < "Ivan") << "\n";  // usporedbe
}
```

IZLAZ PROGRAMA

```
Ana Horvat!
Duljina: 11
Prvi znak: A
Zadnji znak: !
Ena Horvat!
1 1
```

Stringovi se uspoređuju operatorima `==`, `!=`, `<`, `>`… **leksikografski** — kao u rječniku, znak po znak, prema ASCII kodovima. Zato je `"Ana" < "Ivan"` istina (A dolazi prije I), ali i `"Zebra" < "auto"` — jer su sva velika slova u ASCII tablici prije malih.

SPAJANJE DVA LITERALA

`string s = "Bok" + " svima";` se **ne prevodi**. Literali u navodnicima nisu `string` objekti, nego C-nizovi znakova (odjeljak 8.6), a njih se ne može zbrajati. Barem jedan operand mora biti `string`: `string("Bok") + " svima"` ili `ime + " svima"`.

8.2 Prolazak kroz znakove

String je niz znakova, pa se obrađuje petljom — s indeksom ili range-for. Funkcije iz `<cctype>` pomažu s klasifikacijom znakova:

Funkcija	Provjerava / radi	Funkcija	Provjerava / radi
`isdigit(c)`	znamenka 0–9	`isalpha(c)`	slovo
`isupper(c)`	veliko slovo	`islower(c)`	malo slovo
`isspace(c)`	bjelina	`isalnum(c)`	slovo ili znamenka
`toupper(c)`	u veliko slovo	`tolower(c)`	u malo slovo

```cpp
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string s = "Programiranje u C++ 2026";
    int slova = 0, znamenke = 0, samoglasnici = 0;
    for (char c : s) {
        if (isalpha(c)) slova++;
        if (isdigit(c)) znamenke++;
        char m = tolower(c);
        if (m == 'a' || m == 'e' || m == 'i' || m == 'o' || m == 'u') samoglasnici++;
    }
    cout << slova << " " << znamenke << " " << samoglasnici << "\n";

    for (char& c : s) c = toupper(c);   // & -> mijenjamo originalne znakove
    cout << s << "\n";
}
```

IZLAZ PROGRAMA

```
15 4 6
PROGRAMIRANJE U C++ 2026
```

8.3 Najvažnije metode

Metoda	Što radi	Primjer (`s = "banana"`)
`s.substr(poz, n)`	podniz od pozicije `poz`, duljine `n`	`s.substr(1, 3)` → `"ana"`
`s.find(t)`	pozicija prvog pojavljivanja ili `string::npos`	`s.find("na")` → `2`
`s.find(t, od)`	traži od pozicije `od` nadalje	`s.find("na", 3)` → `4`
`s.rfind(t)`	pozicija zadnjeg pojavljivanja	`s.rfind('a')` → `5`
`s.replace(poz, n, t)`	zamijeni `n` znakova od `poz` s `t`	`s.replace(0, 1, "B")` → `"Banana"`
`s.insert(poz, t)`	umetni `t` na poziciju `poz`	`s.insert(0, "a")` → `"abanana"`
`s.erase(poz, n)`	obriši `n` znakova od `poz`	`s.erase(0, 2)` → `"nana"`
`s.empty()`, `s.clear()`	je li prazan; isprazni	

Najčešći obrazac s `find` je petlja koja pronalazi **sva** pojavljivanja:

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string tekst = "ana voli banane i ananas";
    string trazi = "an";
    int broj = 0;
    size_t poz = tekst.find(trazi);
    while (poz != string::npos) {          // npos = "nije pronađeno"
        broj++;
        poz = tekst.find(trazi, poz + 1);  // nastavi iza pronađenog
    }
    cout << "\"" << trazi << "\" se pojavljuje " << broj << " puta\n";
}
```

IZLAZ PROGRAMA

```
"an" se pojavljuje 5 puta
```

REZULTAT FIND SPREMAJ U SIZE_T

`find` vraća `size_t`, a kad ne nađe ništa, vraća posebnu vrijednost `string::npos` (najveći mogući `size_t`). Uvjet `if (s.find(x) != string::npos)` je ispravan način provjere „sadrži li string x“. Nemoj uspoređivati s `-1` preko `int`-a — radi slučajno, ne zato što je ispravno. (C++23 donosi i jednostavno `s.contains(x)`.)

8.4 Pretvorbe između brojeva i stringova

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int n = stoi("42");                  // string -> int
    double d = stod("3.75");             // string -> double
    string s = to_string(n * 2);         // broj -> string
    cout << n + 1 << " " << d * 2 << " " << s + "!" << "\n";

    string znamenke = "2026";
    int zbroj = 0;
    for (char c : znamenke) zbroj += c - '0';   // znak-znamenka -> broj
    cout << zbroj << "\n";
}
```

IZLAZ PROGRAMA

```
43 7.5 84!
10
```

`stoi` i `stod` bacaju iznimku ako string ne počinje brojem (npr. `stoi("abc")`) — kako se to hvata, vidjet ćemo u poglavlju 21.

8.5 `stringstream` — string kao tok

`stringstream` iz zaglavlja `<sstream>` omogućuje da se stringom služiš *kao da je* `cin`. Idealno je za rastavljanje teksta na riječi ili brojeve:

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;
```

```cpp
int main() {
    string recenica = "C++ je  brz i mocan";
    stringstream ss(recenica);
    string rijec;
    vector<string> rijeci;
    while (ss >> rijec) rijeci.push_back(rijec);   // >> preskače višestruke razmake
    cout << "Broj rijeci: " << rijeci.size() << "\n";

    stringstream brojevi("10 20 30");
    int x, zbroj = 0;
    while (brojevi >> x) zbroj += x;
    cout << "Zbroj: " << zbroj << "\n";

    // rastavljanje po zarezu (npr. CSV redak) pomoću getline s graničnikom
    stringstream csv("Ana,21,Zagreb");
    string polje;
    while (getline(csv, polje, ',')) cout << "[" << polje << "] ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
Broj rijeci: 5
Zbroj: 60
[Ana] [21] [Zagreb]
```

Zadnji primjer — `getline` s trećim argumentom — koristit ćemo pri čitanju CSV datoteka u poglavlju 22.

8.6 C-stringovi (ukratko)

Prije `std::string`-a tekst se u C-u čuvao kao **polje znakova koje završava nul-znakom** `'\0'`. Svaki literal u navodnicima, npr. `"Bok"`, zapravo je takvo polje: `{'B', 'o', 'k', '\0'}` — četiri znaka za tri slova.

```cpp
char ime[20] = "Ana";        // polje od 20 znakova, iskorištena 4 (uključujući '\0')
strlen(ime);                 // 3 — broji do '\0'  (iz <cstring>)
strcpy(ime, "Marija");       // kopiranje — opasno ako odredište nije dovoljno veliko!
```

C-stringovi ne znaju svoju duljinu, ne rastu i lako se prepune (to je bio izvor bezbrojnih sigurnosnih propusta u povijesti softvera). **U modernom C++-u koristi `std::string`.** C-stringove trebaš prepoznati jer ih srećeš u starijem kodu, u argumentima `main(int argc, char* argv[])` i kod nekih biblioteka. Kad nekoj C funkciji trebaš dati `std::string`, koristi `s.c_str()`.

UKRATKO

- `std::string` je niz znakova koji sam upravlja memorijom: `+`, `+=`, `size()`, `[]`, usporedbe s `==` i `<`.
- Ključne metode: `substr`, `find` (provjera s `string::npos`), `replace`, `insert`, `erase`.
- `<cctype>`: `isdigit`, `isalpha`, `toupper`, `tolower` … Znak-znamenka u broj: `c - '0'`.
- `stoi`, `stod`, `to_string` za pretvorbe. `stringstream` za rastavljanje teksta.
- C-stringovi završavaju s `'\0'` — prepoznaj ih, ali koristi `std::string`.

Zadaci

Zadatak 8.1 ★☆☆

Korisnik unosi rečenicu (cijeli red). Ispiši broj riječi, broj velikih slova i rečenicu u kojoj je svaka riječ napisana velikim početnim slovom.

Zadatak 8.2 ★★☆

Napiši funkciju `bool jePalindrom(const string& s)` koja zanemaruje razmake i razliku velikih i malih slova. „Ana voli Milovana“ je palindrom.

Zadatak 8.3 ★★☆

Cezarova šifra: napiši funkcije `sifriraj(string s, int k)` i `desifriraj(string s, int k)` koje svako slovo pomiču za `k` mjesta u abecedi (u krug, `z` + 1 = `a`), a ostale znakove ostavljaju kakvi jesu. Pazi na velika i mala slova.

Zadatak 8.4 ★★☆

Napiši funkciju `string zamijeniSve(string s, const string& staro, const string& novo)` koja zamjenjuje **sva** pojavljivanja podniza `staro` s `novo`. Pazi da radi i kad `novo` sadrži `staro` (npr. zamjena `"a"` s `"aa"`).

Zadatak 8.5 ★★★

Kompresija: napiši funkciju koja niz `"aaabccddddd"` pretvara u `"a3b1c2d5"`, i funkciju koja radi obrnuto. Ako komprimirani niz nije kraći od originala, prva funkcija vraća original.

Zadatak 8.6 ★★★

Napiši funkciju koja provjerava jesu li dvije riječi **anagrami** (sadrže ista slova u različitom redoslijedu, npr. „listen“ i „silent“). Riješi na dva načina: (a) sortiranjem, (b) brojanjem pojavljivanja slova u polju od 26 elemenata. Koji je brži i zašto?

DIO II

Memorija

Pokazivači i reference, stog i gomila, dinamička memorija i strukture. Dio na kojem većina ljudi zapne — i dio koji C++ čini onim što jest.

9. Pokazivači i reference

Ako postoji jedna tema zbog koje C++ ima reputaciju teškog jezika, to su pokazivači. Dobra vijest: pokazivači nisu teški — teško je kad ih netko objasni prebrzo. Ideja je zapravo jednostavna: **pokazivač je varijabla u kojoj piše adresa druge varijable**. Sve ostalo proizlazi iz toga. Idemo polako i s puno slika.

9.1 Adrese i operator &

Iz poglavlja 2 znamo da svaka varijabla živi na nekoj adresi u memoriji. Tu adresu dobivamo operatorom `&` (*adresa od*):

```
#include <iostream>
using namespace std;

int main() {
    int x = 42;
    cout << "vrijednost: " << x << "\n";
    cout << "adresa:     " << &x << "\n";   // ispisuje se heksadekadski
}
```

IZLAZ PROGRAMA

```
vrijednost: 42
adresa:     0x7ffd5e8c3a4c
```

Adresa će kod tebe biti drugačija, i mijenjat će se od pokretanja do pokretanja. To je u redu — važno je samo da *postoji* i da je možemo spremiti.

9.2 Pokazivač: varijabla koja čuva adresu

```
#include <iostream>
using namespace std;

int main() {
    int x = 42;
    int* p = &x;         // p je "pokazivač na int"; u njemu je adresa od x

    cout << *p << "\n";  // * = dereferenciranje: "idi na adresu i uzmi vrijednost"
    *p = 100;            // promijeni vrijednost NA TOJ ADRESI
    cout << x << "\n";   // x je promijenjen, iako ga nismo spomenuli!
}
```

IZLAZ PROGRAMA

```
42
100
```



Slika 9.1 — Pokazivač p sadrži adresu varijable x. Izraz *p znači „ono što je na toj adresi“, dakle x.

Zbunjujuće je što se znakovi `*` i `&` koriste u dva značenja, ovisno o tome gdje stoje:

Gdje stoji	`*` znači	`&` znači
u **deklaraciji** (uz tip)	`int* p` — p je pokazivač	`int& r = x` — r je referenca
u **izrazu**	`*p` — dereferenciranje (vrijednost na adresi)	`&x` — adresa od x

ANALOGIJA: ADRESA NA PAPIRIĆU

Varijabla je kuća, a njena vrijednost je ono što je unutra. Pokazivač je **papirić na kojem piše adresa kuće**. Papirić možeš kopirati, dati nekome ili prepisati drugom adresom — kuća se zbog toga ne mijenja. Ali ako odeš na adresu s papirića (dereferenciraš), možeš ući u kuću i promijeniti namještaj. To je `*p = 100`.

ZAMKA U DEKLARACIJI VIŠE POKAZIVAČA

`int* a, b;` deklarira **jedan** pokazivač (`a`) i **jedan običan** `int` (`b`)! Zvjezdica pripada imenu, ne tipu. Ispravno: `int *a, *b;` — ili, bolje, svaka deklaracija u svom redu.

9.3 `nullptr` i opasni pokazivači

Pokazivač koji trenutno ne pokazuje ni na što postavlja se na `nullptr` (C++11; u starom kodu vidjet ćeš `NULL` ili `0`). Dereferenciranje `nullptr`-a je nedefinirano ponašanje — u praksi gotovo uvijek rušenje programa. Zato prije dereferenciranja provjeravamo:

```
int* p = nullptr;
if (p != nullptr) {      // ili kraće: if (p)
    cout << *p;
}
```

Postoje tri vrste opasnih pokazivača koje moraš znati imenovati:

- **neinicijaliziran pokazivač** (`int* p;`) — sadrži slučajnu adresu, pokazuje „bilo kamo“;
- **nul-pokazivač** — `nullptr`; siguran za provjeru, ali ne i za dereferenciranje;
- **viseći pokazivač** (*dangling pointer*) — pokazuje na memoriju koja više ne postoji, npr. na lokalnu varijablu funkcije koja je završila, ili na obrisanu dinamičku memoriju (poglavlje 10).

✗ NEISPRAVNO

```
int* napravi() {
    int lokalna = 5;
    return &lokalna;     // lokalna nestaje kad funkcija završi — vraćamo viseći pokazivač!
}
```

Pravilo: pokazivač uvijek inicijaliziraj — na valjanu adresu ili na `nullptr`.

9.4 Pokazivači i reference — usporedba

Reference smo upoznali kod funkcija: referenca je drugo ime za postojeću varijablu. „Ispod haube“ prevoditelj reference obično izvodi pomoću adresa, ali za programera se ponašaju drugačije:

	Pokazivač `int* p`	Referenca `int& r`
mora se inicijalizirati	ne (ali trebao bi)	**da**
može biti „prazan“	da (`nullptr`)	**ne**
može se preusmjeriti na drugi objekt	da (`p = &y;`)	**ne** — zauvijek vezana
pristup vrijednosti	`*p`	samo `r`
aritmetika (`p + 1`)	da	ne

Kad što koristiti? Za parametre funkcija u pravilu koristi **reference** — sigurnije su i čitljivije. Pokazivače koristi kad objekt *možda ne postoji* (`nullptr` kao „nema ga“), kad se cilj mora mijenjati, pri radu s dinamičkom memorijom i C bibliotekama.

```cpp
#include <iostream>
using namespace std;

void zamijeniP(int* a, int* b) { int t = *a; *a = *b; *b = t; }  // preko pokazivača
void zamijeniR(int& a, int& b) { int t = a;  a = b;  b = t; }    // preko referenci

int main() {
    int x = 1, y = 2;
    zamijeniP(&x, &y);           // šaljemo ADRESE
    cout << x << " " << y << "\n";
    zamijeniR(x, y);             // šaljemo same varijable
    cout << x << " " << y << "\n";
}
```

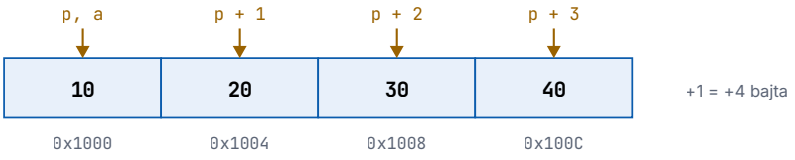
IZLAZ PROGRAMA

```
2 1
1 2
```

Obje funkcije rade isto. Verzija s referencama je čišća — nema `*` i `&` na svakom koraku.

9.5 Pokazivači i polja

Ovo je veza koja objašnjava mnogo toga. **Ime polja se u većini izraza pretvara u pokazivač na prvi element.** Nadalje, **aritmetika s pokazivačima** radi u jedinicama *elemenata*, ne bajtova: ako `p` pokazuje na `int`, onda `p + 1` pokazuje na *sljedeći* `int` (4 bajta dalje).



Slika 9.2 — Aritmetika pokazivača: p + i pokazuje na i-ti element, bez obzira na veličinu tipa

```cpp
#include <iostream>
using namespace std;

int main() {
    int a[4] = {10, 20, 30, 40};
    int* p = a;                  // isto što i &a[0]

    cout << *p << " " << *(p + 2) << " " << p[3] << "\n";
    cout << (a[2] == *(a + 2)) << "\n";   // a[i] je SAMO skraćenica za *(a + i)

    for (int* q = a; q != a + 4; q++)     // prolazak pokazivačem
        cout << *q << " ";
    cout << "\n";
    cout << "Razmak: " << (&a[3] - &a[0]) << " elementa\n";
}
```

IZLAZ PROGRAMA

```
10 30 40
1
10 20 30 40
Razmak: 3 elementa
```

ZATO POLJE U FUNKCIJI NE ZNA SVOJU VELIČINU

Sad je jasno što se događa u poglavlju 7: `void f(int a[], int n)` zapravo je `void f(int* a, int n)`. Funkcija dobiva **samo adresu prvog elementa** — pokazivač ne nosi informaciju o tome koliko elemenata slijedi. I zato funkcija može mijenjati originalno polje: radi na istoj memoriji. Obrazac „pokazivač na početak i pokazivač iza kraja“ iz zadnje petlje temelj je **iteratora** u STL-u (poglavlje 19).

9.6 `const` i pokazivači

Kod pokazivača postoje dvije stvari koje mogu biti konstantne: **podatak** na koji pokazuje i **sam pokazivač**. Pravilo čitanja: **čitaj zdesna nalijevo**.

```cpp
int x = 1, y = 2;

const int* p1 = &x;    // "p1 je pokazivač na const int"
*p1 = 5;               // GREŠKA: podatak se ne smije mijenjati preko p1
p1 = &y;               // OK: sam pokazivač se smije preusmjeriti

int* const p2 = &x;    // "p2 je const pokazivač na int"
*p2 = 5;               // OK: podatak se smije mijenjati
p2 = &y;               // GREŠKA: pokazivač je zaključan na x

const int* const p3 = &x;   // ništa se ne smije mijenjati
```

U praksi je najčešći prvi oblik: `const int*` (ili `const T&`) u parametrima funkcija znači „samo ću čitati“.

UKRATKO

- Pokazivač = varijabla koja sadrži adresu. `&x` daje adresu, `*p` daje vrijednost na adresi.
- `nullptr` = ne pokazuje ni na što. Nikad ne dereferenciraj `nullptr`, neinicijaliziran ili viseći pokazivač.
- Referenca: drugo ime, mora se inicijalizirati, ne može biti prazna ni preusmjerena. Za parametre preferiraj reference.
- Ime polja → pokazivač na prvi element; `a[i]` ≡ `*(a + i)`; `p + 1` = sljedeći element.
- `const int* p` — ne mijenjaš podatak; `int* const p` — ne mijenjaš pokazivač. Čitaj zdesna nalijevo.

NA ISPITU

Zadaci s pokazivačima gotovo su uvijek tipa „što ispisuje program“. Rješavaj ih **crtanjem kućica i strelica** kao na slikama 9.1 i 9.2: za svaku varijablu nacrtaj kućicu, za svaki pokazivač strelicu, i prati promjene redak po redak. Nitko to ne radi u glavi — ni iskusni programeri.

Zadaci

Zadatak 9.1 ★☆☆

Što ispisuje program? Nacrtaj kućice i strelice.

```
int a = 5, b = 10;
int* p = &a;
int* q = &b;
*p = *q + 1;
p = q;
*p = 0;
cout << a << " " << b << " " << *p << " " << *q << "\n";
```

Zadatak 9.2 ★★☆

Napiši funkciju `void minMax(const int* a, int n, int* mn, int* mx)` koja preko pokazivača vraća najmanji i najveći element polja. Zatim napiši istu funkciju s referencama. Koja je verzija čitljivija?

Zadatak 9.3 ★★☆

Napiši funkciju `void obrni(int* a, int n)` koja obrće polje **koristeći dva pokazivača** — jedan kreće s početka, drugi s kraja, a zamjenjuju vrijednosti dok se ne sretnu. Ne koristi indekse (`[]`).

Zadatak 9.4 ★★☆

Napiši svoju verziju funkcije `int duljina(const char* s)` koja računa duljinu C-stringa (broji znakove do `'\0'`) koristeći samo pokazivače.

Zadatak 9.5 ★★★

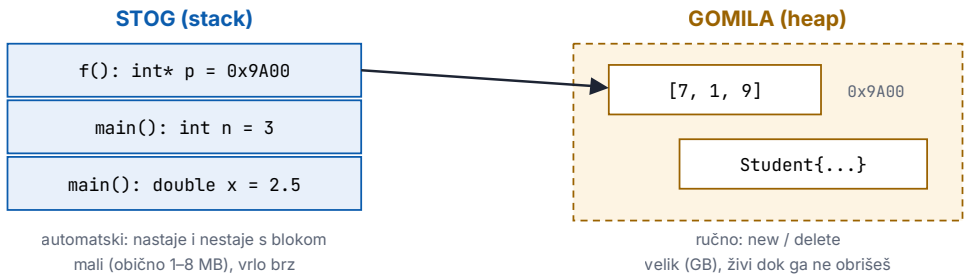
Što ispisuje program? Objasni svaki redak.

```
int a[] = {2, 4, 6, 8, 10};
int* p = a + 1;
cout << *p++ << " ";
cout << *p << " ";
cout << ++*p << " ";
cout << *(p + 2) - *(p - 2) << " ";
cout << p - a << "\n";
```

10. Stog, gomila i dinamička memorija

Sve varijable koje smo dosad koristili imale su veličinu poznatu pri prevođenju i nestajale su same od sebe na kraju bloka. Ali što ako veličinu doznamo tek dok program radi (korisnik unese koliko ima studenata)? Ili ako objekt mora preživjeti funkciju koja ga je stvorila? Za to služi **dinamička memorija**. Ovo poglavlje objašnjava gdje program drži podatke, kako ručno tražiti i vraćati memoriju — i zašto je moderni C++ smislio načine da to gotovo nikad ne radiš ručno.

10.1 Dva područja memorije: stog i gomila



Slika 10.1 — Lokalne varijable (uključujući pokazivače) žive na stogu; dinamički objekti na gomili

	Stog (stack)	Gomila (heap)
Što tu živi	lokalne varijable, parametri, okviri poziva	objekti stvoreni s `new`
Tko upravlja	prevoditelj, **automatski**	**programer** (`new` / `delete`)
Životni vijek	do kraja bloka	dok ga eksplicitno ne obrišeš
Veličina	mala (tipično nekoliko MB)	velika (koliko ima RAM-a)
Veličina objekta	mora biti poznata pri prevođenju	može se odrediti tijekom izvođenja

Zato `int polje[10000000];` kao lokalna varijabla često sruši program (*stack overflow*), a isto polje na gomili radi bez problema.

10.2 `new` i `delete`

Operator `new` traži memoriju na gomili, stvara u njoj objekt i vraća **pokazivač** na njega. Operator `delete` tu memoriju vraća sustavu.

```
#include <iostream>
using namespace std;

int main() {
    int* p = new int(42);      // jedan int na gomili, inicijaliziran na 42
    cout << *p << "\n";
    *p += 8;
    cout << *p << "\n";
    delete p;                  // vraćamo memoriju
    p = nullptr;               // dobra navika: p više ne "visi"
}
```

IZLAZ PROGRAMA

```
42
50
```

Primijeti: sam pokazivač `p` je obična lokalna varijabla na **stogu**. Objekt na koji pokazuje je na **gomili**. Kad `main` završi, `p` nestaje automatski — ali objekt na gomili ne bi nestao da ga nismo obrisali.

10.3 Dinamička polja: `new[]` i `delete[]`

Najčešća upotreba: polje čija je veličina poznata tek tijekom izvođenja.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;                          // veličinu određuje korisnik
    double* ocjene = new double[n];    // polje od n double-ova na gomili
    double zbroj = 0;
    for (int i = 0; i < n; i++) {
        cin >> ocjene[i];              // koristi se kao obično polje
        zbroj += ocjene[i];
    }
    cout << "Prosjek: " << zbroj / n << "\n";
    delete[] ocjene;                   // polje se briše s delete[] !
}
```

IZLAZ PROGRAMA

```
Prosjek: 4.25
```

NEW IDE S DELETE, NEW[] IDE S DELETE[]

Ako memoriju zatražiš s `new[]`, moraš je vratiti s `delete[]`. Miješanje (`new[]` + `delete`) je nedefinirano ponašanje. Pravilo je mehaničko: koliko `new`, toliko `delete`; koliko `new[]`, toliko `delete[]` — i to u parovima istog oblika.

10.4 Četiri klasične greške

Ručno upravljanje memorijom otvara vrata četirima greškama koje su godinama bile glavni uzrok rušenja programa i sigurnosnih propusta:

1. Curenje memorije (memory leak) — memorija se zatraži, ali nikad ne vrati. Program je sve gladniji dok ne potroši svu memoriju.

✗ NEISPRAVNO

```cpp
void obradi() {
    int* podaci = new int[1000000];
    // ... rad s podacima ...
    if (greska) return;          // IZLAZ bez delete[] — 4 MB su izgubljeni zauvijek
    delete[] podaci;
}
```

2. Viseći pokazivač (use after free) — koristi se memorija koja je već vraćena.

✗ NEISPRAVNO

```cpp
int* p = new int(5);
delete p;
cout << *p;        // memorija više nije naša — nedefinirano ponašanje
```

3. Dvostruko brisanje (double free) — ista memorija obriše se dvaput, npr. kad dva pokazivača pokazuju na isti objekt pa ga svaki „vlasnik“ briše. Obično ruši program.

4. Krivi oblik brisanja — `delete` umjesto `delete[]` ili obrnuto.

> **ZAŠTO SU OVE GREŠKE TAKO OPASNE?**
>
> Nijednu od njih prevoditelj ne može otkriti, a često se ne manifestiraju odmah. Viseći pokazivač može mjesecima „raditi“ jer je stara vrijednost slučajno još u memoriji — sve dok neki drugi `new` ne dobije tu istu memoriju. Tada se greška pojavi na potpuno drugom mjestu u programu. Alati poput *AddressSanitizer* (`g++ -fsanitize=address`) i *Valgrind* pomažu ih pronaći.

10.5 Dinamička matrica

Matrica čije dimenzije doznajemo tek tijekom izvođenja gradi se kao polje pokazivača, od kojih svaki pokazuje na jedan redak:

```cpp
#include <iostream>
using namespace std;

int main() {
    int r = 3, s = 4;
    int** m = new int*[r];            // polje od r pokazivača
    for (int i = 0; i < r; i++)
        m[i] = new int[s];            // svaki redak zasebno

    for (int i = 0; i < r; i++)
        for (int j = 0; j < s; j++)
            m[i][j] = i * 10 + j;
    cout << m[2][3] << "\n";

    for (int i = 0; i < r; i++)       // brisanje obrnutim redom:
        delete[] m[i];                // prvo reci...
    delete[] m;                       // ...pa polje pokazivača
}
```

IZLAZ PROGRAMA

```
23
```

Usporedi s `vector<vector<int>> m(r, vector<int>(s));` — jedna linija, bez ijednog `delete`, bez mogućnosti curenja. To je glavna poruka ovog poglavlja.

10.6 Kako raste `vector`? (pogled ispod haube)

Pitanje za razmišljanje: polje na gomili ima fiksnu veličinu. Kako onda `vector` može rasti s `push_back`? Odgovor: **ne raste — mijenja se.** Kad ponestane mjesta, vektor zatraži novo, veće polje (obično dvostruko), prekopira stare elemente, obriše staro polje i nastavi raditi s novim. Pojednostavljeno:

```cpp
// imamo: int* podaci (polje), int velicina, int kapacitet
void dodaj(int x) {
    if (velicina == kapacitet) {
        int noviKap = (kapacitet == 0) ? 1 : 2 * kapacitet;
        int* novo = new int[noviKap];
        for (int i = 0; i < velicina; i++) novo[i] = podaci[i];   // kopiraj
        delete[] podaci;                                          // obriši staro
        podaci = novo;
        kapacitet = noviKap;
    }
    podaci[velicina++] = x;
}
```

Budući da se kapacitet udvostručuje, preseljenja su sve rjeđa, pa je `push_back` u prosjeku vrlo brz. Ovo je i razlog zašto pokazivači na elemente vektora mogu postati viseći nakon `push_back` — elementi su se možda preselili!

10.7 Poruka za moderni C++

U modernom C++-u **gotovo nikad ne pišeš `new` i `delete` izravno**. Umjesto toga:

- za nizove koristiš `std::vector` (i `std::string` za tekst);
- za pojedinačne dinamičke objekte koristiš **pametne pokazivače** `std::unique_ptr` i `std::shared_ptr` (poglavlje 20);
- vlastite klase koje upravljaju resursima pišeš po načelu **RAII** (poglavlje 13) — destruktor sam oslobađa resurs.

Zašto onda učimo `new` i `delete`? Zato što ih nalaziš u postojećem kodu i na ispitima, i zato što **bez razumijevanja onoga što se događa ispod haube ne možeš razumjeti ni zašto su moderni alati napravljeni tako kako jesu**.

UKRATKO

- Stog: automatske lokalne varijable, brz i mali. Gomila: dinamički objekti, velika, ručno upravljana.
- `new T` → pokazivač na novi objekt; `delete p` ga briše. `new T[n]` → polje; `delete[] p`.
- Klasične greške: curenje, viseći pokazivač, dvostruko brisanje, krivi oblik brisanja.
- Nakon `delete` postavi pokazivač na `nullptr`.
- U praksi: `vector`, `string`, pametni pokazivači, RAII — a ne ručni `new`/`delete`.

Zadaci

Zadatak 10.1 ★☆☆

Korisnik unosi `n`, a zatim `n` cijelih brojeva u dinamičko polje. Ispiši brojeve koji su veći od prosjeka i oslobodi memoriju.

Zadatak 10.2 ★★☆

Napiši funkciju `int* spoji(const int* a, int na, const int* b, int nb)` koja stvara novo dinamičko polje u kojem su elementi prvog pa drugog polja. Tko je odgovoran za brisanje vraćenog polja? Kako bi se ta odgovornost mogla „zaboraviti“?

Zadatak 10.3 ★★☆

Pronađi i objasni sve greške s memorijom:

✗ NEISPRAVNO

```
int* f() {
    int* a = new int[5];
    int b[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) a[i] = b[i];
    return b;
}
int main() {
    int* p = f();
    int* q = new int(7);
    q = p;
    delete q;
    delete[] p;
}
```

Zadatak 10.4 ★★★

Implementiraj jednostavan „vektor“: strukturu (poglavlje 11) s članovima `int* podaci`, `int velicina`, `int kapacitet` te funkcije `dodaj`, `dohvati(indeks)` i `oslobodi`. Kapacitet udvostručuj kad ponestane mjesta. Dodaj 20 elemenata i ispiši kapacitet nakon svakog dodavanja.

11. Strukture i enumeracije

Student nije samo ime. Student je ime, prezime, JMBAG, godina studija i popis ocjena — **skup podataka koji zajedno opisuju jednu stvar**. Držati to u pet odvojenih vektora (`imena`, `prezimena`, `ocjene` ...) i paziti da indeksi uvijek odgovaraju je noćna mora. **Struktura** omogućuje da ih povežemo u jedan novi tip. Ovo je i most prema klasama: klasa je, u biti, struktura s ponašanjem.

11.1 Definicija i korištenje strukture

```
#include <iostream>
#include <string>
using namespace std;

struct Tocka {                  // definiramo NOVI TIP imena Tocka
    double x = 0;               // članovi (s početnim vrijednostima)
    double y = 0;
};                              // ; na kraju je obavezna!

int main() {
    Tocka a;                    // a.x = 0, a.y = 0
    Tocka b{3, 4};              // inicijalizacija redom članova
    a.x = 1;                    // pristup članu: operator .
    cout << "a = (" << a.x << ", " << a.y << ")\n";
    cout << "b = (" << b.x << ", " << b.y << ")\n";

    Tocka c = b;                // strukture se kopiraju član po član
    c.x = 100;
    cout << b.x << " " << c.x << "\n";   // b nije promijenjen
}
```

IZLAZ PROGRAMA

```
a = (1, 0)
b = (3, 4)
3 100
```

Nakon definicije, `Tocka` je tip kao i `int`: možeš napraviti varijable tog tipa, vektor točaka, funkciju koja prima i vraća točku. Strukture se ponašaju kao **vrijednosti** — pridruživanje radi kopiju, baš kao kod `int`-a.

TOČKA-ZAREZ IZA STRUKTURE

Zaboravljeni `;` iza zatvorene vitičaste zagrade strukture (ili klase) daje jednu od najčudnijih poruka o greškama, često u potpuno drugoj liniji. Ako ti prevoditelj javlja besmislenu grešku odmah ispod definicije strukture — provjeri točku-zarez.

11.2 Strukture i funkcije

Strukture se funkcijama predaju **po `const` referenci** kad ih funkcija samo čita (izbjegavamo kopiranje), a po referenci kad ih mijenja. Funkcija može i **vratiti** strukturu — to je najčišći način da se vrati više vrijednosti odjednom.

```
#include <iostream>
#include <cmath>
using namespace std;

struct Tocka { double x, y; };
```

```cpp
double udaljenost(const Tocka& a, const Tocka& b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

Tocka poloviste(const Tocka& a, const Tocka& b) {
    return { (a.x + b.x) / 2, (a.y + b.y) / 2 };   // vraćamo novu točku
}

void pomakni(Tocka& t, double dx, double dy) {      // mijenja original
    t.x += dx;
    t.y += dy;
}

int main() {
    Tocka a{0, 0}, b{6, 8};
    cout << udaljenost(a, b) << "\n";
    Tocka p = poloviste(a, b);
    cout << p.x << " " << p.y << "\n";
    pomakni(a, 1, 1);
    cout << a.x << " " << a.y << "\n";
}
```

IZLAZ PROGRAMA

```
10
3 4
1 1
```

11.3 Vektor struktura

Ovo je obrazac koji ćeš koristiti stalno: `vector` u kojem je svaki element struktura. Svaki zapis drži sve svoje podatke na okupu.

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Student {
    string ime;
    int bodovi;
};

int main() {
    vector<Student> grupa = { {"Ana", 87}, {"Ivan", 64}, {"Petra", 93}, {"Luka", 71} };
    grupa.push_back({"Maja", 58});

    double zbroj = 0;
    const Student* najbolji = &grupa[0];
    for (const Student& s : grupa) {           // const& — bez kopiranja
        zbroj += s.bodovi;
        if (s.bodovi > najbolji->bodovi) najbolji = &s;
    }
    cout << "Prosjek: " << zbroj / grupa.size() << "\n";
    cout << "Najbolji: " << najbolji->ime << " (" << najbolji->bodovi << ")\n";
    for (const auto& s : grupa)
        if (s.bodovi >= 70) cout << s.ime << " ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
Prosjek: 74.6
Najbolji: Petra (93)
Ana Petra Luka
```

11.4 Pokazivači na strukture i operator ->

U prošlom primjeru pojavio se novi operator. Kad imaš **pokazivač** na strukturu, članu pristupaš operatorom `->`:

```
Student s{"Ana", 87};
Student* p = &s;
cout << (*p).ime;    // dereferenciraj pa uzmi član — zagrade su obavezne
cout << p->ime;      // isto, samo čitljivije: "član objekta na koji p pokazuje"
```

Pravilo: **točka za objekt, strelica za pokazivač**. `p->ime` je samo skraćenica za `(*p).ime`. Zagrade su tu jer `.` ima veći prioritet od `*`, pa bi `*p.ime` značilo `*(p.ime)` — a to nema smisla.

11.5 Ugniježđene strukture

Član strukture može biti i druga struktura, vektor ili string:

```
struct Datum { int dan, mjesec, godina; };

struct Ispit {
    string predmet;
    Datum datum;                // struktura unutar strukture
    vector<int> bodoviPoZadatku;
};

Ispit i{"Programiranje", {15, 6, 2026}, {10, 8, 15}};
cout << i.datum.godina;         // 2026 — pristup "u dubinu"
```

11.6 Enumeracije: `enum` i `enum class`

Kad varijabla smije poprimiti samo nekoliko imenovanih vrijednosti (dani u tjednu, stanje narudžbe, boja semafora), koristi se **enumeracija**. Moderni oblik je `enum class`:

```
#include <iostream>
using namespace std;

enum class Semafor { Crveno, Zuto, Zeleno };

Semafor sljedece(Semafor s) {
    switch (s) {
        case Semafor::Crveno: return Semafor::Zeleno;
        case Semafor::Zeleno: return Semafor::Zuto;
        case Semafor::Zuto:   return Semafor::Crveno;
    }
    return Semafor::Crveno;
}

int main() {
    Semafor s = Semafor::Crveno;
    for (int i = 0; i < 4; i++) {
        cout << static_cast<int>(s) << " ";   // enum class -> int samo eksplicitno
        s = sljedece(s);
    }
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
0 2 1 0
```

Stari oblik `enum Boja { Crvena, Plava };` također postoji, ali ima dva problema: imena vrijednosti „cure“ u okolni doseg (ne smiješ imati `Crvena` ni u jednom drugom enumu), i vrijednosti se tiho pretvaraju u `int`, pa `Crvena + 5` ili usporedba boje s danom u tjednu prolazi bez greške. `enum class` rješava oboje — vrijednosti pišeš s prefiksom (`Semafor::Crveno`) i nema tihih pretvorbi.

SAVJET: ENUM UMJESTO „MAGIČNIH“ BROJEVA I STRINGOVA

`int stanje = 2;` — što je 2? `string stanje = "poslano";` — a što ako negdje napišeš `"poslan"`? `StanjeNarudzbe::Poslano` je samodokumentirajuće, a tipfeler prevoditelj odmah otkrije.

UKRATKO

- `struct` grupira povezane podatke u novi tip. Pristup članu: `objekt.clan`; preko pokazivača: `p->clan`.
- Strukture se kopiraju član po član. Funkcijama ih predaj po `const&` (čitanje) ili `&` (mijenjanje); vraćanje strukture = vraćanje više vrijednosti.
- `vector<Struktura>` je jedan od najčešćih obrazaca u praksi.
- `enum class` za skup imenovanih vrijednosti — sigurniji od starog `enum`.

Zadaci

Zadatak 11.1 ★☆☆

Definiraj strukturu `Pravokutnik` (širina, visina) i funkcije `povrsina`, `opseg` i `jeKvadrat`. Učitaj tri pravokutnika i ispiši onaj s najvećom površinom.

Zadatak 11.2 ★★☆

Definiraj strukturu `Razlomak` (brojnik, nazivnik) i funkcije `zbroji`, `pomnozi` i `skrati` (koristi NZD). Funkcije vraćaju novi, skraćeni razlomak. Izračunaj 1/2 + 1/3 + 1/6.

Zadatak 11.3 ★★☆

Napravi `vector<Student>` gdje student ima ime i `vector<int>` ocjena. Napiši funkciju koja računa prosjek studenta, a zatim sortiraj studente po prosjeku silazno (vlastitim bubble sortom) i ispiši ih.

Zadatak 11.4 ★★☆

Definiraj `enum class Dan { Pon, Uto, Sri, Cet, Pet, Sub, Ned }`, funkciju `Dan sutra(Dan d)`, funkciju `bool jeVikend(Dan d)` i funkciju `string naziv(Dan d)`. Ispiši koji je dan za 100 dana ako je danas ponedjeljak.

Zadatak 11.5 ★★★

Mala **evidencija skladišta**: struktura `Artikl` (šifra, naziv, količina, cijena). Napiši izbornik (`do-while` + `switch`) s opcijama: dodaj artikl, ispiši sve, pronađi po šifri, promijeni količinu, ispiši ukupnu vrijednost skladišta, izlaz.

DIO III

Objektno orijentirano programiranje

Klase i objekti, konstruktori i destruktori, kopiranje i RAII, preopterećenje operatora, nasljeđivanje i polimorfizam.

12. Klase i objekti

Objektno orijentirano programiranje (OOP) nije skup ključnih riječi, nego **način razmišljanja**: program se sastoji od objekata koji imaju svoje stanje (podatke) i ponašanje (funkcije), i koji sami brinu o tome da im stanje uvijek bude ispravno. U ovom poglavlju naučit ćemo pisati klase i — još važnije — razumjeti *zašto* su napravljene tako kako jesu.

12.1 Od strukture do klase: problem koji rješavamo

Zamislimo bankovni račun kao strukturu:

✗ NEISPRAVNO

```
struct Racun {
    string vlasnik;
    double stanje;
};

Racun r{"Ana", 100};
r.stanje = -5000;        // nitko nas ne sprječava!
r.stanje -= 200;         // isplata bez provjere pokrića
```

Struktura je samo „vreća podataka“. Svatko može u nju upisati bilo što, i pravilo „stanje ne smije biti negativno“ postoji samo u našim glavama. U programu od sto tisuća redova netko će ga prekršiti — pitanje je samo kada.

Rješenje je **enkapsulacija** (učahurivanje): podatke sakrijemo (`private`), a izvana dopustimo samo **kontrolirane operacije** (`public`) koje provjeravaju pravila. Tako objekt sam jamči svoju ispravnost.

> **ANALOGIJA: AUTOMOBIL**
>
> Vozač ima **sučelje**: volan, papučice, mjenjač. Ne može izravno dirati ubrizgavanje goriva ni paljenje svjećica — to je **unutarnja implementacija**, skrivena ispod haube. Zahvaljujući tome vozač ne mora znati kako motor radi, proizvođač može promijeniti motor a da vozač to ni ne primijeti, i nitko ne može slučajno ubrizgati gorivo u krivom trenutku. Klasa radi isto: `public` dio je volan, `private` dio je motor.

12.2 Prva klasa

```
#include <iostream>
#include <string>
using namespace std;

class Racun {
private:                                  // vidljivo samo unutar klase
    string vlasnik;
    double stanje;

public:                                   // sučelje prema van
    Racun(const string& v, double pocetno) {   // KONSTRUKTOR
        vlasnik = v;
        stanje = (pocetno >= 0) ? pocetno : 0;
    }

    void uplati(double iznos) {
        if (iznos > 0) stanje += iznos;
    }

    bool isplati(double iznos) {
```

```
        if (iznos <= 0 || iznos > stanje) return false;   // pravilo se provjerava OVDJE
        stanje -= iznos;
        return true;
    }

    double getStanje() const { return stanje; }       // "getter"
    string getVlasnik() const { return vlasnik; }
};

int main() {
    Racun r("Ana", 100);                  // stvaranje OBJEKTA klase Racun
    r.uplati(50);
    if (!r.isplati(500)) cout << "Nema pokrica!\n";
    r.isplati(30);
    cout << r.getVlasnik() << ": " << r.getStanje() << "\n";
    // r.stanje = -5000;                  // GREŠKA pri prevođenju: stanje je private
}
```

IZLAZ PROGRAMA

```
Nema pokrica!
Ana: 120
```

Osnovni pojmovi:

- **Klasa** je *nacrt* (tip); **objekt** je konkretan primjerak napravljen po nacrtu. `Racun` je klasa, `r` je objekt. Od jedne klase možeš napraviti tisuću objekata, svaki sa svojim stanjem.
- **Atributi** (podatkovni članovi) — `vlasnik`, `stanje`.
- **Metode** (funkcijski članovi) — `uplati`, `isplati` … Metode imaju izravan pristup atributima *objekta nad kojim su pozvane*.
- **Specifikatori pristupa**: `private` — dostupno samo metodama klase; `public` — dostupno svima; `protected` — o njemu kod nasljeđivanja.

SAVJET: ATRIBUTI PRIVATE, I TO UVIJEK

Dobro pravilo za početak: **svi atributi su `private`**, a javne su samo metode koje čine smisleno sučelje. Nemoj automatski pisati getter i setter za svaki atribut — `setStanje(double)` bi opet otvorio vrata za `-5000`. Pitaj se: *koje operacije ima smisla raditi s ovim objektom?* Za račun to su `uplati` i `isplati`, a ne „postavi stanje“.

12.3 Konstruktori

Konstruktor je posebna metoda koja se poziva **automatski pri stvaranju objekta**. Ima isto ime kao klasa i nema povratni tip. Njegov posao je dovesti objekt u ispravno početno stanje — tako da ne postoji trenutak u kojem objekt „postoji, a nije ispravan“.

Postoji bolji način pisanja konstruktora od onog u prošlom primjeru — **inicijalizacijska lista**:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    string ime;
    int godina;
    const string jmbag;              // const član: MORA se inicijalizirati u listi

public:
    Student(const string& i, int g, const string& j)
        : ime(i), godina(g), jmbag(j)            // inicijalizacijska lista
    {
        // tijelo može ostati prazno
```

```
    }

    Student(const string& i) : Student(i, 1, "0000000000") {}  // delegirajući konstruktor

    void ispisi() const {
        cout << ime << ", " << godina << ". godina, JMBAG " << jmbag << "\n";
    }
};

int main() {
    Student a("Ana", 2, "0016123456");
    Student b("Ivan");
    a.ispisi();
    b.ispisi();
}
```

IZLAZ PROGRAMA

```
Ana, 2. godina, JMBAG 0016123456
Ivan, 1. godina, JMBAG 0000000000
```

ZAŠTO INICIJALIZACIJSKA LISTA, A NE PRIDRUŽIVANJE U TIJELU?

Kad tijelo konstruktora počne, svi atributi **već su stvoreni**. Pridruživanje u tijelu (`ime = i;`) znači: prvo napravi prazan string, pa ga onda prepiši. Lista stvara atribut **odmah s pravom vrijednošću**. Za `int` razlika je nebitna, ali za tri slučaja lista je **obavezna**: `const` atributi, reference kao atributi i atributi čija klasa nema zadani konstruktor. Zato — uvijek koristi listu.

Pravila koja treba znati:

- **Zadani konstruktor** (*default constructor*) je konstruktor bez parametara. Ako ne napišeš *nijedan* konstruktor, prevoditelj ga generira sam. **Čim napišeš bilo koji konstruktor, automatski zadani nestaje** — pa `Student s;` više ne prolazi. Vratiti ga možeš s `Student() = default;`.
- Atributima možeš dati početne vrijednosti izravno u klasi: `int godina = 1;` — koriste se ako ih konstruktor ne postavi.
- Članovi se inicijaliziraju **redoslijedom kojim su deklarirani u klasi**, a ne redoslijedom u listi. Drži oba redoslijeda jednakima da izbjegneš iznenađenja.
- Konstruktor s jednim parametrom omogućuje implicitnu pretvorbu: ako postoji `Tocka(int)`, onda prolazi i `Tocka t = 5;`. Ako to ne želiš, označi konstruktor s `explicit`.

12.4 Destruktor

Destruktor se poziva **automatski kad objekt prestaje postojati** — kad lokalni objekt izađe iz dosega, ili kad se dinamički objekt obriše s `delete`. Ime mu je `~` + ime klase, nema parametara ni povratnog tipa. Služi za „pospremanje“: oslobađanje memorije, zatvaranje datoteka, otključavanje resursa.

```
#include <iostream>
#include <string>
using namespace std;

class Glasnik {
    string ime;
public:
    Glasnik(const string& i) : ime(i) { cout << "  + " << ime << "\n"; }
    ~Glasnik()                        { cout << "  - " << ime << "\n"; }
};

int main() {
    cout << "pocetak main\n";
    Glasnik a("A");
    {
```

```
        cout << "ulaz u blok\n";
        Glasnik b("B");
        Glasnik c("C");
        cout << "izlaz iz bloka\n";
    }                                   // ovdje umiru c, pa b (obrnutim redom!)
    Glasnik* d = new Glasnik("D");
    delete d;                           // destruktor se zove pri delete
    cout << "kraj main\n";
}                                       // ovdje umire a
```

IZLAZ PROGRAMA

```
pocetak main
  + A
ulaz u blok
  + B
  + C
izlaz iz bloka
  - C
  - B
  + D
  - D
kraj main
  - A
```

Objekti se uništavaju **obrnutim redoslijedom od stvaranja** — kao tanjuri naslagani jedan na drugi. To je važno: objekt stvoren kasnije može ovisiti o ranijem, pa ga treba ukloniti prvi.

12.5 Pokazivač `this`, `const` metode i `static` članovi

`this` je skriveni parametar svake metode — pokazivač na objekt nad kojim je metoda pozvana. Kad pozoveš `r.uplati(50)`, unutar metode je `this == &r`. Obično ga ne pišemo (`stanje` je isto što i `this->stanje`), ali treba nam kad se ime parametra poklapa s imenom atributa, ili kad metoda vraća sam objekt (`return *this;` — vidjet ćemo kod operatora).

`const` metode. Riječ `const` iza zagrada metode (`double getStanje() const`) obećava da metoda **ne mijenja objekt**. To nije ukras: na **konstantnom objektu** (ili kroz `const&` referencu) smiješ zvati **samo `const` metode**. Budući da objekte stalno predajemo kao `const Racun&`, metoda za čitanje bez `const`-a postaje neupotrebljiva baš tamo gdje treba. **Pravilo: svaka metoda koja ne mijenja objekt označava se s `const`.**

`static` članovi pripadaju **klasi**, a ne pojedinom objektu — postoji jedna kopija koju dijele svi objekti:

```
#include <iostream>
using namespace std;

class Proizvod {
    int id;
    static int brojac;                  // jedan brojač za cijelu klasu
public:
    Proizvod() : id(++brojac) {}
    int getId() const { return id; }
    static int ukupno() { return brojac; }   // static metoda: nema this
};

int Proizvod::brojac = 0;               // definicija static atributa (izvan klase)

int main() {
    Proizvod a, b, c;
    cout << a.getId() << " " << c.getId() << " " << Proizvod::ukupno() << "\n";
}
```

IZLAZ PROGRAMA

```
1 3 3
```

12.6 `struct` ili `class`?

U C++-u su `struct` i `class` **gotovo isto**. Jedina razlika: u `struct`-u su članovi po zadanom `public`, a u `class`-u `private`. Po dogovoru, `struct` koristimo za jednostavne skupove podataka bez pravila (točka, par vrijednosti), a `class` za tipove koji imaju pravila i skrivenu implementaciju.

12.7 Klasa u više datoteka

U stvarnim projektima svaka klasa ima dvije datoteke: **zaglavlje** (`.h`) s deklaracijom klase — „što klasa nudi“ — i **implementaciju** (`.cpp`) s definicijama metoda — „kako to radi“.

```
// ===== Racun.h =====
#pragma once                    // zaštita od višestrukog uključivanja
#include <string>

class Racun {
    std::string vlasnik;        // u zaglavljima NE pišemo using namespace std
    double stanje;
public:
    Racun(const std::string& v, double pocetno);
    bool isplati(double iznos);
    double getStanje() const;
};

// ===== Racun.cpp =====
#include "Racun.h"              // navodnici = naše datoteke, <> = sistemske

Racun::Racun(const std::string& v, double pocetno)    // Racun:: = "metoda klase Racun"
    : vlasnik(v), stanje(pocetno >= 0 ? pocetno : 0) {}

bool Racun::isplati(double iznos) {
    if (iznos <= 0 || iznos > stanje) return false;
    stanje -= iznos;
    return true;
}

double Racun::getStanje() const { return stanje; }

// ===== main.cpp =====
#include "Racun.h"
int main() { Racun r("Ana", 100); r.isplati(30); }
```

Prevodi se s `g++ main.cpp Racun.cpp -o program` — svaka `.cpp` datoteka prevodi se zasebno, a povezivač ih spaja (sjeti se slike 1.1).

ZAŠTO #PRAGMA ONCE (ILI INCLUDE GUARD)?

`#include` doslovno kopira datoteku. Ako `main.cpp` uključi `Racun.h` i `Banka.h`, a `Banka.h` također uključuje `Racun.h`, klasa `Racun` bila bi definirana dvaput — greška. `#pragma once` kaže preprocesoru: „ovu datoteku uključi najviše jednom“. Stariji, standardni oblik istog je *include guard*: `#ifndef RACUN_H` / `#define RACUN_H` / … / `#endif`.

UKRATKO

- Klasa = nacrt (tip); objekt = primjerak. Enkapsulacija: atributi `private`, sučelje `public`.
- Konstruktor dovodi objekt u ispravno stanje; koristi **inicijalizacijsku listu**. Napisani konstruktor briše automatski zadani.
- Destruktor (`~Klasa`) se zove automatski kad objekt nestaje; objekti nestaju obrnutim redom.
- `this` pokazuje na trenutni objekt. Metode koje ne mijenjaju objekt označi s `const`.
- `static` član dijele svi objekti klase. `struct` i `class` razlikuju se samo po zadanom pristupu.
- Zaglavlje `.h` (deklaracija, `#pragma once`) + `.cpp` (definicije s `Klasa::`).

Zadaci

Zadatak 12.1 ★☆☆

Napiši klasu `Pravokutnik` s privatnim atributima `sirina` i `visina`, konstruktorom koji ne dopušta negativne dimenzije (postavlja ih na 0), te `const` metodama `povrsina()`, `opseg()` i `jeKvadrat()`.

Zadatak 12.2 ★★☆

Napiši klasu `Vrijeme` (sati, minute, sekunde) s konstruktorom koji normalizira vrijednosti (npr. 0:75:80 postaje 1:16:20), metodom `dodajSekunde(int s)` i metodom `ispisi()` u formatu `hh:mm:ss`. Kako je najlakše interno čuvati vrijeme?

Zadatak 12.3 ★★☆

Napiši klasu `Stog` (stack) za cijele brojeve koja interno koristi `vector<int>`, s metodama `push`, `pop`, `top`, `empty` i `size`. Što `pop` i `top` trebaju raditi na praznom stogu? Pomoću nje provjeri jesu li zagrade u izrazu `"(a+[b*c]-{d})"` ispravno uparene.

Zadatak 12.4 ★★☆

Dodaj klasi `Racun` iz odjeljka 12.2 statički atribut koji broji koliko je računa ukupno otvoreno, jedinstveni broj računa (dodjeljuje se u konstruktoru) i metodu `prijenos(Racun& drugi, double iznos)`. Podijeli klasu u `Racun.h` i `Racun.cpp`.

Zadatak 12.5 ★★★

Napiši klasu `Polinom` koja čuva koeficijente u `vector<double>`, s metodama `vrijednost(double x)` (koristi Hornerov algoritam), `derivacija()` koja vraća novi `Polinom`, `stupanj()` i `ispisi()` (npr. `3x^2 - 2x + 1`).

13. Kopiranje, RAII i pravilo tri/pet/nule

Ovo je poglavlje u kojem C++ postaje „pravi“ C++. Naučit ćemo što se točno događa kad kopiraš objekt, zašto klasa koja upravlja memorijom treba posebne funkcije, i upoznati **RAII** — ideju koja je vjerojatno najvažniji doprinos C++-a programiranju općenito. Na ispitima iz OOP-a u C++-u ove teme gotovo nikad ne izostaju.

13.1 RAII: resurs je vezan uz život objekta

Resurs je sve što se mora uzeti i obavezno vratiti: dinamička memorija, otvorena datoteka, mrežna veza, zaključani *mutex*. U poglavlju 10 vidjeli smo koliko je lako zaboraviti `delete`. RAII (*Resource Acquisition Is Initialization* — „stjecanje resursa je inicijalizacija“) rješava to jednostavnom idejom:

1. resurs se **uzima u konstruktoru**;
2. resurs se **vraća u destruktoru**.

Budući da C++ **jamči** da će se destruktor lokalnog objekta pozvati kad objekt izađe iz dosega — bez obzira izlazi li se normalno, preko `return` usred funkcije ili zbog iznimke — resurs se *ne može* zaboraviti osloboditi.

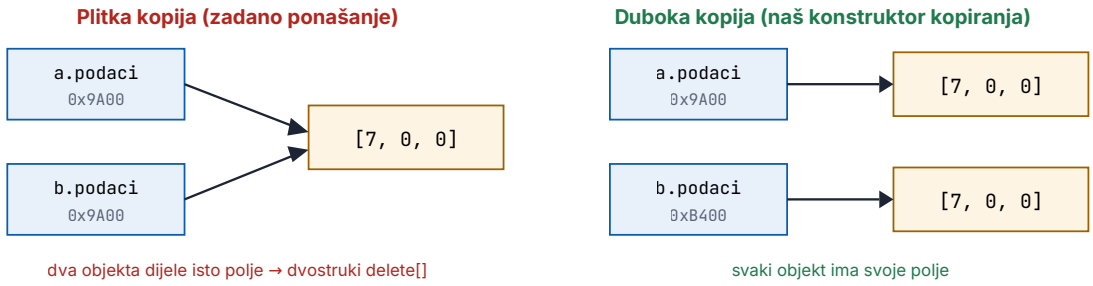
```
class DinamickoPolje {
    int* podaci;
    int n;
public:
    explicit DinamickoPolje(int vel) : podaci(new int[vel]{}), n(vel) {}  // uzmi
    ~DinamickoPolje() { delete[] podaci; }                                 // vrati
};

void f() {
    DinamickoPolje p(1000);
    if (nesto) return;       // destruktor se svejedno pozove — nema curenja
}                            // i ovdje se pozove
```

`std::vector`, `std::string`, `std::ifstream` i pametni pokazivači — svi rade po RAII načelu. Zato ih koristimo umjesto ručnog `new`/`delete`.

13.2 Problem: plitka kopija

Ako ne napišeš drugačije, prevoditelj za svaku klasu generira kopiranje **član po član**. Za `int` i `string` to je savršeno. Ali za pokazivač to znači da se kopira **adresa**, a ne podaci na koje pokazuje:



Slika 13.1 — Plitka i duboka kopija objekta koji sadrži pokazivač

S plitkom kopijom dogode se dvije katastrofe: promjena kroz `b` vidi se i u `a` (dijele podatke), a na kraju oba destruktora zovu `delete[]` na **istu adresu** — dvostruko brisanje, rušenje programa. Rješenje je napisati vlastito kopiranje koje radi **duboku kopiju**.

13.3 Četiri posebne funkcije za upravljanje resursom

Klasa koja *sama* upravlja resursom treba, osim konstruktora i destruktora, još četiri funkcije. Pogledajmo ih sve u jednom primjeru — s ispisima, da se vidi kad se koja poziva:

```
#include <iostream>
#include <utility>
using namespace std;

class Polje {
    int* podaci;
    int n;
public:
    explicit Polje(int vel) : podaci(new int[vel]{}), n(vel) {
        cout << "konstruktor (" << n << ")\n";
    }
    ~Polje() {
        cout << "destruktor (" << n << ")\n";
        delete[] podaci;                        // delete[] nullptr je bezopasan
    }
    // (1) KONSTRUKTOR KOPIRANJA: novi objekt kao kopija postojećeg
    Polje(const Polje& o) : podaci(new int[o.n]), n(o.n) {
        for (int i = 0; i < n; i++) podaci[i] = o.podaci[i];
        cout << "kopiranje (" << n << ")\n";
    }
    // (2) PRIDRUŽIVANJE KOPIRANJEM: postojeći objekt dobiva sadržaj drugoga
    Polje& operator=(const Polje& o) {
        cout << "pridruzivanje kopiranjem (" << o.n << ")\n";
        if (this == &o) return *this;           // zaštita od a = a
        int* novo = new int[o.n];
        for (int i = 0; i < o.n; i++) novo[i] = o.podaci[i];
        delete[] podaci;                        // oslobodi STARI sadržaj
        podaci = novo;
        n = o.n;
        return *this;                           // omogućuje a = b = c
    }
    // (3) KONSTRUKTOR PREMJEŠTANJA: "ukradi" resurs privremenog objekta
    Polje(Polje&& o) noexcept : podaci(o.podaci), n(o.n) {
        o.podaci = nullptr;                     // izvor ostaje prazan, ali ispravan
        o.n = 0;
        cout << "premjestanje (" << n << ")\n";
    }
    // (4) PRIDRUŽIVANJE PREMJEŠTANJEM
    Polje& operator=(Polje&& o) noexcept {
        cout << "pridruzivanje premjestanjem (" << o.n << ")\n";
        if (this != &o) {
            delete[] podaci;
            podaci = o.podaci;  n = o.n;
            o.podaci = nullptr; o.n = 0;
        }
        return *this;
    }
    void postavi(int i, int v) { podaci[i] = v; }
    int dohvati(int i) const { return podaci[i]; }
    int velicina() const { return n; }
};

int main() {
    Polje a(3);
    a.postavi(0, 7);
    Polje b = a;                    // novi objekt iz postojećeg -> kopiranje
    b.postavi(0, 99);
    cout << a.dohvati(0) << " " << b.dohvati(0) << "\n";
    Polje c(1);
    c = a;                          // postojeći objekt -> pridruživanje
    Polje d = std::move(a);         // "a mi više ne treba" -> premještanje
    cout << "a: " << a.velicina() << ", d: " << d.velicina() << "\n";
```

```
    c = Polje(5);                   // privremeni objekt -> pridruživanje premještanjem
    cout << "kraj\n";
}
```

IZLAZ PROGRAMA

```
konstruktor (3)
kopiranje (3)
7 99
konstruktor (1)
pridruzivanje kopiranjem (3)
premjestanje (3)
a: 0, d: 3
konstruktor (5)
pridruzivanje premjestanjem (5)
destruktor (0)
kraj
destruktor (3)
destruktor (5)
destruktor (3)
destruktor (0)
```

Prođi kroz ispis redak po redak — to je najbolja vježba za ovo poglavlje. Nekoliko važnih zapažanja:

- `Polje b = a;` **nije** pridruživanje, iako ima znak `=`. Objekt `b` se tek *stvara*, pa se zove **konstruktor** kopiranja. Pridruživanje se zove samo kad objekt s lijeve strane **već postoji** (`c = a;`).
- Pridruživanje mora **osloboditi stari sadržaj** (konstruktor to ne mora — novi objekt nema starog sadržaja) i paziti na **samopridruživanje** `a = a` — bez provjere bismo obrisali podatke koje upravo kopiramo.
- Operatori pridruživanja vraćaju `*this` po referenci, da bi radilo ulančavanje `a = b = c`.
- `destruktor (0)` nakon `c = Polje(5)` je privremeni objekt kojem smo „ukrali“ polje — umire prazan.

13.4 Semantika premještanja (move)

Zašto uopće premještanje? Zamisli funkciju koja vraća vektor s milijun elemenata, ili vektor koji se preseljava u veće polje (odjeljak 10.6). Kopiranje milijun elemenata, pa brisanje originala koji ionako nikome ne treba, čisto je rasipanje. Premještanje umjesto toga **preuzme pokazivač** — tri naredbe, bez obzira na veličinu.

Kako prevoditelj zna kada smije premjestiti? C++ razlikuje:

- **lvalue** — objekt s imenom, koji će se još koristiti (`a`, `v[3]`). Njega smijemo samo kopirati.
- **rvalue** — privremena vrijednost koja će odmah nestati (`Polje(5)`, rezultat funkcije, `x + y`). Iz nje smijemo „krasti“.

Oznaka `Polje&&` je **rvalue referenca** — veže se samo na privremene objekte. Funkcija `std::move(a)` ne premješta ništa sama po sebi; ona samo kaže prevoditelju „tretiraj `a` kao privremen, meni više ne treba“. Nakon toga objekt `a` je u „ispravnom, ali nespecificiranom“ stanju — smiješ ga uništiti ili mu pridružiti novu vrijednost, ali ne računaj na njegov sadržaj.

> **SAVJET: NOEXCEPT NA PREMJEŠTANJU**
>
> Označi konstruktor i pridruživanje premještanjem s `noexcept`. `std::vector` pri preseljenju u veće polje koristi premještanje **samo ako je označeno kao** `noexcept` — inače, radi sigurnosti, kopira. Bez te riječi tvoja klasa u vektoru radi sporije.

13.5 Pravilo tri, pravilo pet i pravilo nule

- **Pravilo tri** (klasični C++): ako klasa treba vlastiti **destruktor**, **konstruktor kopiranja** ili **operator pridruživanja kopiranjem**, onda gotovo sigurno treba **sva tri**. Razlog: sve tri postoje samo zato što klasa ručno upravlja resursom.
- **Pravilo pet** (C++11): isto, uz dodatak **konstruktora premještanja** i **pridruživanja premještanjem**.
- **Pravilo nule** (moderni C++): najbolje je da klasa **ne mora pisati nijednu** od tih pet funkcija. To postižeš tako da resursima ne upravljaš ručno, nego koristiš članove koji to već rade ispravno — `vector`, `string`, `unique_ptr`. Prevoditelj tada sam generira ispravno kopiranje, premještanje i uništavanje.

```
#include <string>
#include <vector>

class Student {                        // pravilo nule: ni jedne posebne funkcije
    std::string ime;
    std::vector<int> ocjene;           // vector sam zna kopirati, premjestiti i obrisati
public:
    Student(std::string i) : ime(std::move(i)) {}
    void dodajOcjenu(int o) { ocjene.push_back(o); }
};
```

Ako klasa ne smije biti kopirana (npr. predstavlja jedinstvenu vezu s bazom), zabrani kopiranje eksplicitno: `Veza(const Veza&) = delete;` i `Veza& operator=(const Veza&) = delete;`.

UKRATKO

- **RAII**: resurs se uzima u konstruktoru, vraća u destruktoru. Destruktor se poziva uvijek, pa nema curenja.
- Zadano kopiranje je **plitko** (kopira pokazivače). Za klasu s vlastitim resursom treba **duboka** kopija.
- `T b = a;` → konstruktor kopiranja; `b = a;` (b postoji) → pridruživanje. Pridruživanje oslobađa stari sadržaj, provjerava `this == &o` i vraća `*this`.
- Premještanje „krade“ resurs rvalue objekta; `std::move` označava objekt kao privremen. Označi ga s `noexcept`.
- Pravilo tri/pet: sve ili ništa. Pravilo nule: koristi `vector` / `string` / `unique_ptr` i ne piši ništa.

Zadaci

Zadatak 13.1 ★☆☆

Za svaki redak odredi koja se funkcija klase `Polje` iz 13.3 poziva (konstruktor, kopiranje, pridruživanje kopiranjem, premještanje, pridruživanje premještanjem): `Polje x(4); Polje y(x); Polje z = x; y = z; Polje w = Polje(2); z = std::move(y);` (Napomena: za `Polje w = Polje(2);` moderni prevoditelji ne zovu ni kopiranje ni premještanje — potraži pojam *copy elision*.)

Zadatak 13.2 ★★☆

Klasa `Tekst` čuva C-string: `char* znakovi` alociran s `new[]`. Napiši konstruktor iz `const char*`, destruktor, konstruktor kopiranja, operator pridruživanja i metodu `duljina()`. Testiraj da kopije budu neovisne.

Zadatak 13.3 ★★☆

Objasni što ne valja u ovom operatoru pridruživanja i popravi ga:

✗ NEISPRAVNO

```
Polje& operator=(const Polje& o) {
    delete[] podaci;
    n = o.n;
    podaci = new int[n];
    for (int i = 0; i < n; i++) podaci[i] = o.podaci[i];
    return *this;
}
```

Zadatak 13.4 ★★★

Napiši klasu `Matrica` (r × s, podaci u jednom dinamičkom polju veličine `r*s`) s pravilom pet i metodom `Matrica pomnozi(const Matrica& b) const`. Zatim napiši drugu verziju iste klase po **pravilu nule** koristeći `vector<double>`. Usporedi duljinu koda.

14. Preopterećenje operatora

Zašto `string` možemo zbrajati s `+`, uspoređivati s `==` i ispisivati s `<<`? Zato što je netko za klasu `string` **preopteretio operatore** — napisao funkcije koje se pozivaju kad se operator primijeni na objekte te klase. Isto možemo učiniti i za svoje klase, pa da `a + b` za razlomke, vektore ili novčane iznose izgleda jednako prirodno kao za brojeve.

14.1 Ideja: operator je samo funkcija s posebnim imenom

Kad prevoditelj vidi `a + b`, a `a` je objekt tvoje klase, on traži funkciju imena `operator+`. Postoje dva mjesta gdje je može naći:

- kao **metodu** lijevog operanda: `a.operator+(b)`;
- kao **slobodnu funkciju** s dva parametra: `operator+(a, b)`.

Pravila:

- ne možeš izmisliti nove operatore (npr. `**`), ni promijeniti prioritet ili broj operanada postojećih;
- barem jedan operand mora biti tvoj tip (ne možeš promijeniti značenje `int + int`);
- neki se operatori ne mogu preopteretiti: `.`, `::`, `?:`, `sizeof`;
- **operator treba raditi ono što čitatelj očekuje**. `+` koji oduzima je tehnički moguć, ali je zločin.

14.2 Potpun primjer: klasa `Razlomak`

```cpp
#include <iostream>
#include <numeric>          // std::gcd
using namespace std;

class Razlomak {
    long long b, n;         // brojnik, nazivnik (nazivnik uvijek > 0)

    void skrati() {
        if (n < 0) { b = -b; n = -n; }
        long long d = gcd(b, n);
        if (d > 1) { b /= d; n /= d; }
    }

public:
    Razlomak(long long brojnik = 0, long long nazivnik = 1) : b(brojnik), n(nazivnik) {
        if (n == 0) { n = 1; }          // pojednostavljeno; bolje: baciti iznimku (pogl. 21)
        skrati();
    }

    // --- složeni operatori kao METODE: mijenjaju *this ---
    Razlomak& operator+=(const Razlomak& o) {
        b = b * o.n + o.b * n;
        n = n * o.n;
        skrati();
        return *this;
    }
    Razlomak& operator*=(const Razlomak& o) {
        b *= o.b;  n *= o.n;
        skrati();
        return *this;
    }

    // --- unarni minus ---
    Razlomak operator-() const { return Razlomak(-b, n); }
```

```
    // --- prefiks i postfiks ++ (dodaje 1) ---
    Razlomak& operator++()    { b += n; return *this; }            // ++r
    Razlomak  operator++(int) { Razlomak stari = *this; b += n; return stari; } // r++

    double uDouble() const { return static_cast<double>(b) / n; }

    // --- usporedbe i ispis kao prijateljske (friend) funkcije ---
    friend bool operator==(const Razlomak& x, const Razlomak& y) {
        return x.b == y.b && x.n == y.n;       // radi jer su uvijek skraćeni
    }
    friend bool operator<(const Razlomak& x, const Razlomak& y) {
        return x.b * y.n < y.b * x.n;          // a/b < c/d  <=>  ad < cb  (b, d > 0)
    }
    friend ostream& operator<<(ostream& os, const Razlomak& r) {
        os << r.b;
        if (r.n != 1) os << "/" << r.n;
        return os;                             // vraćamo tok -> ulančavanje
    }
    friend istream& operator>>(istream& is, Razlomak& r) {
        char crta;
        is >> r.b >> crta >> r.n;              // format: 3/4
        r.skrati();
        return is;
    }
};

// binarni operatori kao SLOBODNE funkcije, izgrađeni od složenih
Razlomak operator+(Razlomak x, const Razlomak& y) { return x += y; }
Razlomak operator*(Razlomak x, const Razlomak& y) { return x *= y; }

int main() {
    Razlomak a(1, 2), b(1, 3), c(6, 8);
    cout << a << " + " << b << " = " << a + b << "\n";
    cout << c << " (skraceno), -c = " << -c << "\n";
    cout << a * b + 1 << "\n";                 // 1 se pretvara u Razlomak(1)
    cout << (a < b) << " " << (a == Razlomak(2, 4)) << "\n";
    Razlomak d = a++;                          // d = staro a, a se poveća
    cout << d << " " << a << " ";
    cout << ++a << "\n";                       // a se poveća, pa se ispiše
    cout << Razlomak(22, 7).uDouble() << "\n";
}
```

IZLAZ PROGRAMA

```
1/2 + 1/3 = 5/6
3/4 (skraceno), -c = -3/4
7/6
0 1
1/2 3/2 5/2
3.14286
```

Ovaj primjer sadrži gotovo sve što treba znati. Idemo po dijelovima.

14.3 Metoda ili slobodna funkcija?

Složeni operatori (`+=`, `-=`, `*=`...) mijenjaju lijevi operand, pa su prirodno **metode**. Vraćaju `*this` po referenci (kao i `=`), da bi radilo `(a += b) += c`.

Binarni operatori (`+`, `-`, `*`, `==`, `<`...) pišu se kao **slobodne funkcije**. Razlog je simetrija: da je `operator+` metoda, `a + 1` bi radilo (1 se pretvori u `Razlomak` preko konstruktora), ali `1 + a` ne bi — jer `1` nije objekt i nema metodu. Slobodna funkcija dopušta pretvorbu na **obje** strane. Uobičajeni trik je implementirati `+` pomoću `+=`: prvi parametar primamo **po vrijednosti** (to je već kopija), na nju dodamo drugi i vratimo je.

Operatori `<<` i `>>` za ispis i unos *moraju* biti slobodne funkcije, jer je lijevi operand tok (`cout`), a ne naš objekt. Uvijek primaju i vraćaju tok **po referenci** — tako radi ulančavanje `cout << a << b`.

`friend`: slobodna funkcija nema pristup privatnim članovima. Ključna riječ `friend` unutar klase daje joj taj pristup. Koristi je umjereno — samo za funkcije koje su logično dio sučelja klase (kao `<<` i usporedbe).

Operator	Uobičajeni oblik	Vraća
`+=`, `-=`, `*=`, `=`	metoda	`T&` (`*this`)
`+`, `-`, `*`, `/`	slobodna funkcija	`T` (novi objekt, po vrijednosti)
`==`, `!=`, `<`, `>`…	slobodna (često `friend`)	`bool`
`<<`, `>>`	slobodna (`friend`)	`ostream&` / `istream&`
unarni `-`, `!`	metoda (`const`)	`T`
`++`, `--` prefiks	metoda	`T&`
`++`, `--` postfiks	metoda s lažnim `int` parametrom	`T` (stara vrijednost)
`[]`	metoda (dvije verzije: `const` i ne-`const`)	`T&` / `const T&`
`()`	metoda	bilo što (funkcijski objekti)

14.4 Prefiks i postfiks ++

Kako prevoditelj razlikuje `++a` od `a++`? Po dogovoru, postfiks verzija ima **lažni parametar** `int` koji se ne koristi. Prefiks povećava i vraća *sebe* (referencu). Postfiks mora vratiti **staru vrijednost**, pa napravi kopiju, poveća sebe i vrati kopiju — zato je postfiks nešto sporiji, i zato iskusni programeri u petljama s iteratorima pišu `++it`.

14.5 Operator indeksiranja []

Za klase koje se ponašaju kao nizovi, `[]` se piše u dvije verzije — jedna za mijenjanje, druga za čitanje s konstantnih objekata:

```
class Vektor3 {
    double k[3] = {0, 0, 0};
public:
    double&       operator[](int i)       { return k[i]; }   // v[0] = 5;
    const double& operator[](int i) const { return k[i]; }   // cout << cv[0];
};
```

Vraćanje **reference** omogućuje da `v[0] = 5` mijenja element unutar objekta. Da smo vratili `double` po vrijednosti, pridruživanje bi mijenjalo privremenu kopiju.

> **C++20: OPERATOR <=> (SVEMIRSKI BROD)**
>
> U C++20 možeš napisati samo jedan operator, `auto operator<=>(const T&) const = default;`, i prevoditelj iz njega generira `<`, `<=`, `>`, `>=` (a s `= default` i `==`). Za jednostavne klase koje se uspoređuju član po član to je velika ušteda. Na ispitima se zasad uglavnom traže klasični operatori.

UKRATKO

- `a @ b` postaje `a.operator@(b)` ili `operator@(a, b)`. Operator mora raditi ono što se očekuje.
- Složeni (`+=`) i `=` su metode i vraćaju `*this` po referenci. Binarni (`+`, `==`) su slobodne funkcije radi simetrije; `+` se gradi od `+=`.
- `<<` i `>>` su slobodne `friend` funkcije koje primaju i vraćaju tok po referenci.
- Postfiks `++` ima lažni `int` parametar i vraća staru vrijednost.
- `[]` vraća referencu i dolazi u `const` i ne-`const` verziji.

Zadaci

Zadatak 14.1 ★☆☆

Klasi `Razlomak` dodaj operatore `-=`, `/=`, binarne `-` i `/`, te `!=`, `>`, `<=` i `>=` (posljednja četiri izrazi pomoću `==` i `<`).

Zadatak 14.2 ★★☆

Napiši klasu `Vektor2D` (x, y) s operatorima `+`, `-`, množenjem skalarom s **obje** strane (`v * 2` i `2 * v`), skalarnim produktom (`operator*` za dva vektora), `==` i `<<`. Dodaj metodu `duljina()`.

Zadatak 14.3 ★★☆

Napiši klasu `Novac` koja interno čuva iznos u **centima** (`long long`) i ispisuje ga kao `12.05 EUR`. Preopteretite `+`, `-`, `+=`, `<`, `==` i `<<`. Zašto centi, a ne `double`?

Zadatak 14.4 ★★★

Napiši klasu `VelikiBroj` koja čuva nenegativan cijeli broj proizvoljne duljine kao `string` ili `vector<int>` znamenki. Implementiraj konstruktor iz stringa, `+`, `*` (množenje „na papiru“), `==`, `<` i `<<`. Izračunaj 50! (faktorijel od 50).

15. Nasljeđivanje

Student je osoba. Profesor je osoba. Obojica imaju ime, prezime i OIB, i obojica se znaju predstaviti — ali student ima i JMBAG i prosjek, a profesor katedru i plaću. **Nasljeđivanje** omogućuje da ono zajedničko napišemo **jednom**, u baznoj klasi `Osoba`, a izvedene klase `Student` i `Profesor` to preuzmu i dodaju svoje.

15.1 Osnovna ideja: odnos „je“ (is-a)

Nasljeđivanje izražava odnos **„X je vrsta Y“**: student *je* osoba, krug *je* geometrijski oblik, tekući račun *je* bankovni račun. Terminologija:

- **bazna klasa** (roditelj, nadklasa) — općenitija: `Osoba`;
- **izvedena klasa** (dijete, podklasa) — specifičnija: `Student`.

Izvedena klasa **nasljeđuje sve članove** bazne (atribute i metode), može **dodati nove** i može **promijeniti ponašanje** naslijeđenih metoda.

15.2 Sintaksa i `protected`

```
#include <iostream>
#include <string>
using namespace std;

class Osoba {
protected:                              // vidljivo ovoj klasi I izvedenim klasama
    string ime;
    int godine;
public:
    Osoba(const string& i, int g) : ime(i), godine(g) {}
    void predstaviSe() const {
        cout << "Ja sam " << ime << ", imam " << godine << " godina.";
    }
    string getIme() const { return ime; }
};

class Student : public Osoba {          // Student JE Osoba
    string jmbag;
    double prosjek;
public:
    Student(const string& i, int g, const string& j, double p)
        : Osoba(i, g),                  // PRVO se konstruira bazni dio
          jmbag(j), prosjek(p) {}

    void predstaviSe() const {          // skriva (redefinira) baznu verziju
        Osoba::predstaviSe();           // poziv bazne verzije
        cout << " Student sam, prosjek " << prosjek << ".";
    }
    bool imaStipendiju() const { return prosjek >= 4.5; }   // nova metoda
};

int main() {
    Osoba o("Marko", 45);
    Student s("Ana", 20, "0016123456", 4.7);
    o.predstaviSe(); cout << "\n";
    s.predstaviSe(); cout << "\n";
    cout << s.getIme() << " stipendija: " << s.imaStipendiju() << "\n";  // getIme je naslijeđen
}
```

IZLAZ PROGRAMA

```
Ja sam Marko, imam 45 godina.
Ja sam Ana, imam 20 godina. Student sam, prosjek 4.7.
Ana stipendija: 1
```

Ključne stvari iz primjera:

- `class Student : public Osoba` — Student nasljeđuje Osobu **javno** (gotovo uvijek želiš `public`).
- Konstruktor izvedene klase **mora pozvati konstruktor bazne** u inicijalizacijskoj listi (`: Osoba(i, g)`). Ako to ne učiniš, poziva se zadani konstruktor bazne klase — a ako ga nema, greška.
- `Osoba::predstaviSe()` poziva baznu verziju metode iz izvedene.
- **`protected`** je između `private` i `public`: izvana nedostupno, ali izvedene klase mu smiju pristupiti. Da je `ime` bilo `private`, `Student` ga ne bi mogao izravno čitati (morao bi preko `getIme()`).

> **SAVJET: PROTECTED ATRIBUTI — S MJEROM**
>
> `protected` atributi su praktični, ali oslabljuju enkapsulaciju: svaka izvedena klasa (i oni koje će netko napisati za pet godina) može ih pokvariti. Mnogi iskusni programeri drže atribute `private` i čak i izvedenim klasama nude samo `protected` *metode*. Za početak je `protected` atribut u redu, ali budi svjestan kompromisa.

15.3 Redoslijed konstrukcije i destrukcije

Izvedeni objekt u sebi sadrži cijeli bazni objekt, „kao jezgru“. Zato se gradi **iznutra prema van**, a ruši **izvana prema unutra**:

```cpp
#include <iostream>
using namespace std;

struct A { A() { cout << "A() "; }  ~A() { cout << "~A() "; } };
struct B : A { B() { cout << "B() "; }  ~B() { cout << "~B() "; } };
struct C : B { C() { cout << "C() "; }  ~C() { cout << "~C() "; } };

int main() {
    { C c; cout << "| "; }
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
A() B() C() | ~C() ~B() ~A()
```

Logika je jednostavna: dok se izvodi konstruktor klase `C`, mora već postojati ispravan `B` dio na koji se `C` smije osloniti. Pri uništavanju je obrnuto: `C` se mora počistiti dok njegov `B` dio još postoji.

15.4 Vrste nasljeđivanja

Specifikator ispred bazne klase određuje kako se naslijeđeni članovi vide *izvana*:

Član u baznoj klasi	`public` nasljeđivanje	`protected` nasljeđivanje	`private` nasljeđivanje
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	nedostupan	nedostupan	nedostupan

Privatni članovi bazne klase **postoje** u izvedenom objektu (zauzimaju memoriju), ali im izvedena klasa ne može izravno pristupiti. U praksi se gotovo uvijek koristi `public` nasljeđivanje; ostala dva su rijetka i izražavaju „implementirano pomoću“, a ne „je“.

15.5 Izvedeni objekt kao bazni

Budući da student *jest* osoba, svugdje gdje se očekuje osoba smije se dati student:

```
void pozdravi(const Osoba& o) { cout << "Bok, " << o.getIme() << "!\n"; }

Student s("Ana", 20, "0016123456", 4.7);
pozdravi(s);              // OK: Student& se pretvara u Osoba&
Osoba* p = &s;            // OK: pokazivač na bazni tip može pokazivati na izvedeni objekt
p->predstaviSe();         // PAZI: zove se Osoba::predstaviSe, ne Student::predstaviSe!
```

Zadnji redak je iznenađenje: iako `p` pokazuje na studenta, poziva se *bazna* verzija metode. Razlog je što prevoditelj bira metodu prema **tipu pokazivača** (`Osoba*`), a ne prema stvarnom objektu. Rješenje tog problema — **virtualne funkcije** — tema je sljedećeg poglavlja i srž objektno orijentiranog programiranja.

15.6 Kompozicija ili nasljeđivanje?

Nasljeđivanje nije jedini način ponovne upotrebe koda. **Kompozicija** znači da klasa *sadrži* objekt druge klase kao atribut — odnos **„ima“** (*has-a*): automobil *ima* motor, student *ima* popis ocjena, knjižnica *ima* vektor knjiga.

```
class Motor { /* ... */ };
class Automobil {
    Motor motor;              // kompozicija: Automobil IMA Motor
    vector<Kotac> kotaci;
};
// class Automobil : public Motor   <- pogrešno: automobil NIJE motor
```

PRAVILO IZ PRAKSE: PREDNOST KOMPOZICIJI

Kad dvojiš, izaberi kompoziciju. Nasljeđivanje stvara vrlo čvrstu vezu — promjena bazne klase utječe na sve izvedene. Test: *je li tvrdnja „X je Y“ istinita u svakom kontekstu?* Klasičan protuprimjer je `Kvadrat : public Pravokutnik`. Matematički kvadrat jest pravokutnik, ali ako `Pravokutnik` ima metodu `postaviSirinu(w)` koja mijenja samo širinu, kvadrat bi je naslijedio i prestao biti kvadrat. Izvedena klasa mora se moći koristiti *svugdje* gdje i bazna, bez iznenađenja (Liskovljevo načelo zamjene).

Višestruko nasljeđivanje. C++ dopušta nasljeđivanje od više klasa (`class C : public A, public B`). To je moćno, ali unosi komplikacije (npr. „dijamantni problem“ kad `A` i `B` imaju zajedničku baznu klasu, koji se rješava virtualnim nasljeđivanjem). U praksi se višestruko nasljeđivanje uglavnom koristi za kombiniranje *sučelja* — apstraktnih klasa iz sljedećeg poglavlja.

UKRATKO

- Nasljeđivanje = odnos „je“. `class Izvedena : public Bazna`.
- `protected` članovi dostupni su izvedenim klasama, ali ne i izvana.
- Konstruktor izvedene klase poziva konstruktor bazne u inicijalizacijskoj listi. Gradnja: bazna → izvedena; rušenje: obrnuto.
- `Bazna::metoda()` poziva baznu verziju. Izvedeni objekt smije se koristiti kao bazni (`Bazna&`, `Bazna*`).
- Bez `virtual`, metoda se bira prema tipu pokazivača/reference, ne prema stvarnom objektu.
- Kompozicija („ima“) često je bolji izbor od nasljeđivanja („je“).

Zadaci

Zadatak 15.1 ★☆☆

Iz klase `Osoba` iz 15.2 izvedi klasu `Profesor` s atributima `katedra` i `placa`, vlastitom verzijom `predstaviSe()` (koja poziva baznu) i metodom `povisica(double postotak)`.

Zadatak 15.2 ★★☆

Napravi hijerarhiju `Vozilo` (marka, godina, metoda `opis()`) → `Automobil` (broj vrata) i `Motocikl` (ima li prikolicu). U konstruktore i destruktore dodaj ispise i provjeri redoslijed poziva kad se stvori i uništi po jedan objekt svake klase.

Zadatak 15.3 ★★☆

Napiši klasu `Racun` (broj, stanje, `uplati`, `isplati`) i iz nje izvedi `StedniRacun` (kamatna stopa, metoda `pripisiKamate()`) i `TekuciRacun` (dopušteni minus — `isplati` smije ići do `-dopusteniMinus`). Kako `TekuciRacun` mijenja pravilo isplate ako je `stanje` `private` u baznoj klasi?

Zadatak 15.4 ★★★

Za svaki par odluči je li bolje nasljeđivanje ili kompozicija i obrazloži: (a) `Stog` i `vector`, (b) `Pas` i `Zivotinja`, (c) `Knjiznica` i `Knjiga`, (d) `Menadzer` i `Zaposlenik`, (e) `Kvadrat` i `Pravokutnik`, (f) `Ekran` i `Piksel`.

16. Polimorfizam i virtualne funkcije

Polimorfizam (grč. „mnogo oblika“) je mogućnost da **isti poziv** radi **različite stvari**, ovisno o stvarnom tipu objekta. Imaš vektor geometrijskih oblika — krugova, pravokutnika, trokuta — i želiš izračunati ukupnu površinu jednom petljom, bez hrpe `if`-ova koji provjeravaju „koji je ovo oblik“. Svaki oblik zna sam izračunati svoju površinu; ti samo pitaš. To je srce objektno orijentiranog dizajna.

16.1 Statičko i dinamičko vezanje

Na kraju prošlog poglavlja vidjeli smo problem: preko `Osoba*` koji pokazuje na studenta pozvala se metoda iz `Osoba`. Pogledajmo razliku koju čini jedna riječ — `virtual`:

```
#include <iostream>
using namespace std;

struct Zivotinja {
    void zvukObicni() const { cout << "..."; }
    virtual void zvuk() const { cout << "..."; }       // VIRTUALNA metoda
};

struct Pas : Zivotinja {
    void zvukObicni() const { cout << "Vau"; }
    void zvuk() const override { cout << "Vau"; }       // nadjačava (override)
};

void poReferenci(const Zivotinja& z) { z.zvuk(); }
void poVrijednosti(Zivotinja z)      { z.zvuk(); }      // kopija -> "rezanje"!

int main() {
    Pas p;
    Zivotinja& z = p;                // referenca na baznu klasu, objekt je Pas
    z.zvukObicni(); cout << " | "; z.zvuk(); cout << "\n";
    poVrijednosti(p); cout << " | "; poReferenci(p); cout << "\n";
}
```

IZLAZ PROGRAMA

```
... | Vau
... | Vau
```

- Za **nevirtualnu** metodu prevoditelj bira funkciju **pri prevođenju**, prema tipu reference/pokazivača (`Zivotinja&`) — to je **statičko vezanje**.
- Za **virtualnu** metodu odluka se donosi **tijekom izvođenja**, prema **stvarnom tipu objekta** (`Pas`) — to je **dinamičko vezanje**.

Polimorfizam radi **samo preko pokazivača i referenci**. Kad objekt izvedene klase kopiraš u varijablu bazne klase (`poVrijednosti`), kopira se *samo bazni dio* — izvedeni dio se „odreže“. To se zove **rezanje objekta** (*object slicing*) i zato polimorfne objekte uvijek predaješ po referenci ili pokazivaču.

UVIJEK PIŠI OVERRIDE

Riječ `override` (C++11) kaže prevoditelju: „ova metoda *treba* nadjačati virtualnu metodu bazne klase“. Ako se potpis ne poklapa — npr. zaboravio si `const`, pa si zapravo napisao *novu* metodu — prevoditelj javlja grešku. Bez `override` takva greška prolazi tiho i polimorfizam jednostavno ne radi. Riječ `virtual` u izvedenoj klasi nije potrebna; metoda ostaje virtualna.

16.2 Apstraktne klase i čiste virtualne funkcije

Kolika je površina „oblika“ općenito? Pitanje nema smisla — površinu ima tek konkretan krug ili pravokutnik. Takvu metodu označavamo kao **čistu virtualnu** (`= 0`): ona nema implementaciju u baznoj klasi i *mora* se nadjačati u izvedenoj.

Klasa s barem jednom čistom virtualnom funkcijom je **apstraktna**: od nje se **ne može napraviti objekt** (`Oblik o;` je greška), ali se mogu imati pokazivači i reference na nju. Apstraktna klasa definira **sučelje** — ugovor koji sve izvedene klase moraju ispuniti.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Oblik {                                   // apstraktna klasa (sučelje)
public:
    virtual ~Oblik() = default;                 // virtualni destruktor — obavezno!
    virtual double povrsina() const = 0;        // čista virtualna funkcija
    virtual string naziv() const = 0;
    void ispisi() const {                       // obična metoda koja koristi virtualne
        cout << naziv() << ": " << povrsina() << "\n";
    }
};

class Krug : public Oblik {
    double r;
public:
    explicit Krug(double r) : r(r) {}
    double povrsina() const override { return 3.14159265358979 * r * r; }
    string naziv() const override { return "krug"; }
};

class Pravokutnik : public Oblik {
    double a, b;
public:
    Pravokutnik(double a, double b) : a(a), b(b) {}
    double povrsina() const override { return a * b; }
    string naziv() const override { return "pravokutnik"; }
};

class Kvadrat : public Pravokutnik {
public:
    explicit Kvadrat(double a) : Pravokutnik(a, a) {}
    string naziv() const override { return "kvadrat"; }   // povrsina() nasljeđuje
};

int main() {
    vector<unique_ptr<Oblik>> oblici;           // pametni pokazivači — pogl. 20
    oblici.push_back(make_unique<Krug>(1.0));
    oblici.push_back(make_unique<Pravokutnik>(2, 3));
    oblici.push_back(make_unique<Kvadrat>(4));

    double ukupno = 0;
    for (const auto& o : oblici) {              // JEDNA petlja za sve vrste oblika
        o->ispisi();
        ukupno += o->povrsina();
    }
    cout << "Ukupno: " << ukupno << "\n";
}                                               // unique_ptr sam briše sve oblike
```

IZLAZ PROGRAMA

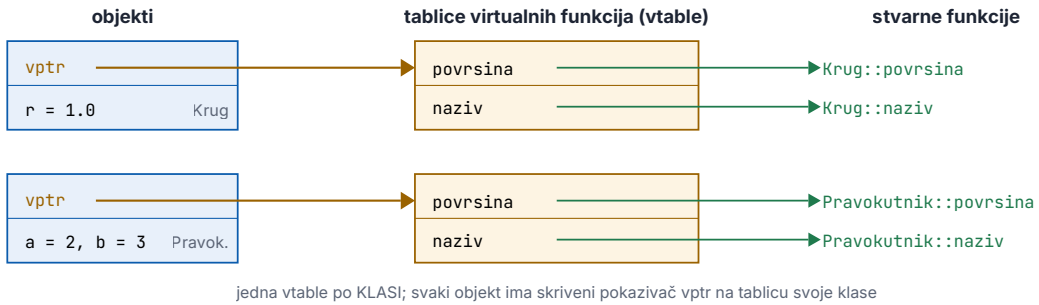
```
krug: 3.14159
pravokutnik: 6
kvadrat: 16
Ukupno: 25.1416
```

Pogledaj snagu ovog dizajna: ako sutra dodaš klasu `Trokut`, petlja u `main`-u i metoda `ispisi()` **ne mijenjaju se ni za jedan znak**. Kod koji koristi oblike ovisi samo o sučelju `Oblik`, a ne o konkretnim klasama. To je načelo „otvoreno za proširenje, zatvoreno za promjene“.

Primijeti i da `Oblik::ispisi()` — obična, nevirtualna metoda bazne klase — poziva `naziv()` i `povrsina()`, a one se razrješavaju dinamički. Bazna klasa definira *tijek* algoritma, a izvedene popunjavaju *korake*.

16.3 Kako to radi: tablica virtualnih funkcija

Kako program tijekom izvođenja zna koju funkciju pozvati? Većina prevoditelja koristi isti mehanizam:



Slika 16.1 — Dinamičko vezanje: poziv o->povrsina() ide preko vptr objekta u tablicu njegove klase

Svaka klasa s virtualnim funkcijama ima **tablicu virtualnih funkcija** (*vtable*) — polje pokazivača na funkcije. Svaki objekt takve klase ima skriveni pokazivač **vptr** na tablicu svoje klase (postavlja ga konstruktor). Poziv `o->povrsina()` prevodi se u „uzmi vptr iz objekta, u tablici pročitaj adresu funkcije `povrsina`, pozovi je“. Cijena je jedan dodatni pokazivač po objektu i jedno indirektno skakanje po pozivu — zanemarivo u gotovo svim programima.

16.4 Virtualni destruktor

Ovo je pravilo koje se **mora** znati: **klasa koja se koristi polimorfno mora imati virtualni destruktor.**

```cpp
#include <iostream>
using namespace std;

struct Baza {
    virtual ~Baza() { cout << "~Baza\n"; }      // probaj maknuti 'virtual'
};

struct Izvedena : Baza {
    int* podaci = new int[1000];
    ~Izvedena() override { delete[] podaci; cout << "~Izvedena\n"; }
};

int main() {
    Baza* b = new Izvedena;
    delete b;              // brišemo PREKO POKAZIVAČA NA BAZNU KLASU
}
```

IZLAZ PROGRAMA

```
~Izvedena
~Baza
```

ŠTO BI SE DOGODILO BEZ VIRTUAL?

`delete b` bi, kao i svaki nevirtualni poziv, izabrao destruktor prema tipu pokazivača — samo `~Baza`. Destruktor `~Izvedena` nikad se ne bi pozvao, polje od 1000 elemenata nikad se ne bi oslobodilo. Standard to formalno proglašava nedefiniranim ponašanjem. Pravilo: ako klasa ima ijednu virtualnu funkciju, neka joj i destruktor bude virtualan (`virtual ~Klasa() = default;`).

16.5 Još nekoliko alata

- **`final`** — `class Kvadrat final : public Pravokutnik` zabranjuje daljnje nasljeđivanje; `void f() override final` zabranjuje daljnje nadjačavanje metode.
- **`dynamic_cast`** — sigurna pretvorba baznog pokazivača u izvedeni, s provjerom tijekom izvođenja. Vraća `nullptr` ako objekt nije traženog tipa:

```
for (const auto& o : oblici) {
    if (auto k = dynamic_cast<Krug*>(o.get())) {    // je li ovo krug?
        cout << "nasao sam krug\n";
    }
}
```

Ali oprez: ako ti kod vrvi `dynamic_cast`-ovima („ako je krug, radi ovo; ako je pravokutnik, radi ono“), to je obično znak da si **zaobišao** polimorfizam. To ponašanje vjerojatno pripada u virtualnu metodu.

- **Poziv virtualne funkcije iz konstruktora** ne ponaša se polimorfno: dok se izvodi konstruktor bazne klase, izvedeni dio još ne postoji, pa se zove bazna verzija. Izbjegavaj takve pozive.

UKRATKO

- `virtual` → funkcija se bira prema **stvarnom tipu objekta**, tijekom izvođenja (dinamičko vezanje). Bez njega — prema tipu pokazivača/reference.
- Polimorfizam radi samo preko **pokazivača i referenci**. Kopiranje u bazni tip „reže“ objekt.
- Uvijek piši `override`. Čista virtualna funkcija (`= 0`) čini klasu apstraktnom (nema objekata, samo sučelje).
- Polimorfna bazna klasa **mora** imati virtualni destruktor.
- Mehanizam: vtable po klasi, vptr u svakom objektu.

NA ISPITU

Standardni zadatak: zadana je hijerarhija klasa, neke metode jesu, a neke nisu virtualne; poziva se preko pokazivača na baznu klasu — što se ispisuje? Za svaki poziv pitaj: *je li metoda virtualna u tipu pokazivača?* Ako da — gledaj stvarni objekt. Ako ne — gledaj tip pokazivača. I pazi na prijenos po vrijednosti (rezanje).

Zadaci

Zadatak 16.1 ★☆☆

Hijerarhiji oblika iz 16.2 dodaj klasu `Trokut` (tri stranice, površina Heronovom formulom) i metodu `opseg()` u sve klase. Promijeni li se išta u petlji u `main`-u?

Zadatak 16.2 ★★☆

Što ispisuje program? Objasni svaki redak.

```
struct A {
    virtual void f() const { cout << "A::f "; }
    void g() const { cout << "A::g "; f(); }
};
struct B : A {
    void f() const override { cout << "B::f "; }
    void g() const { cout << "B::g "; }
};
int main() {
    B b; A a = b; A& r = b; A* p = &b;
    a.f(); r.f(); r.g(); p->g(); b.g(); cout << "\n";
}
```

Zadatak 16.3 ★★☆

Napravi apstraktnu klasu `Zaposlenik` (ime, čista virtualna `double placa() const`) i izvedene klase `StalniZaposlenik` (fiksna mjesečna plaća), `SatnicaZaposlenik` (satnica × sati) i `Prodavac` (osnovica + 5 % prometa). Spremi ih u `vector<unique_ptr<Zaposlenik>>`, ispiši plaće i ukupni trošak plaća.

Zadatak 16.4 ★★★

Napravi mini „zoološki vrt“: apstraktna klasa `Zivotinja` s metodama `zvuk()`, `hrana()` i `dnevnaKolicinaHrane()`. Barem četiri vrste životinja. Program ispisuje sve životinje i ukupnu količinu svake vrste hrane (npr. meso: 12 kg, sijeno: 30 kg) — bez ijednog `dynamic_cast`-a i bez provjere tipa.

DIO IV

Moderni C++ i STL

Predlošci, standardna biblioteka (kontejneri, iteratori, algoritmi, lambde), pametni pokazivači, iznimke, rad s datotekama i navike modernog C++-a.

17. Predlošci (templates)

Napisali smo funkciju `najveci(int, int)`. Onda nam treba za `double`. Pa za `string`. Tri funkcije s *identičnim* tijelom, koje se razlikuju samo po tipu. **Predlošci** rješavaju taj problem: kod pišemo jednom, s tipom kao **parametrom**, a prevoditelj sam generira verziju za svaki tip koji upotrijebimo. Na predlošcima je izgrađena cijela standardna biblioteka — `vector<int>`, `map<string, int>` — pa ih moramo razumjeti.

17.1 Predlošci funkcija

```cpp
#include <iostream>
#include <string>
using namespace std;

template <typename T>              // T je "tip koji će se odrediti kasnije"
T najveci(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << najveci(3, 7) << "\n";                  // T = int
    cout << najveci(2.5, 1.5) << "\n";              // T = double
    cout << najveci(string("ana"), string("iva")) << "\n";   // T = string
    cout << najveci<double>(3, 4.5) << "\n";        // T zadan eksplicitno
}
```

IZLAZ PROGRAMA

```
7
2.5
iva
4.5
```

Prevoditelj iz argumenata **zaključuje** (*deducira*) što je `T`. U zadnjem pozivu argumenti su različitih tipova (`int` i `double`), pa zaključivanje ne uspijeva — tada `T` zadajemo sami u šiljastim zagradama.

> **ŠTO SE ZAPRAVO DOGAĐA?**
>
> Predložak **nije funkcija** — to je *recept* za izradu funkcija. Kad prevoditelj vidi poziv `najveci(3, 7)`, uzme recept, zamijeni `T` s `int` i prevede tako dobivenu funkciju. To se zove **instanciranje** i događa se **pri prevođenju**. Zato predlošci nemaju nikakvu cijenu tijekom izvođenja — generirani kod jednako je brz kao ručno napisan. Za svaki korišteni tip nastaje zasebna funkcija.

Predložak postavlja **implicitne zahtjeve** na tip: `najveci` zahtijeva da se `T` može uspoređivati s `>` i kopirati. Ako ga pozoveš s tipom koji to ne podržava (npr. struktura bez operatora `>`), dobit ćeš grešku pri prevođenju — nažalost često dugačku i nečitljivu. Savjet: u takvoj poruci traži prvu liniju koja spominje *tvoju* datoteku i redak.

Predložak može imati više parametara i raditi s kontejnerima:

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

template <typename T>
void ispisi(const vector<T>& v) {
    cout << "[ ";
    for (const T& x : v) cout << x << " ";
```

```
    cout << "]\n";
}

template <typename T>
T zbroj(const vector<T>& v) {
    T s{};                          // T{} = "nula" za taj tip (0, 0.0, "")
    for (const T& x : v) s += x;
    return s;
}

int main() {
    vector<int> a = {1, 2, 3};
    vector<string> b = {"C", "+", "+"};
    ispisi(a);
    ispisi(b);
    cout << zbroj(a) << " " << zbroj(b) << "\n";
}
```

IZLAZ PROGRAMA

```
[ 1 2 3 ]
[ C + + ]
6 C++
```

Primijeti `T s{};` — vitičasta inicijalizacija bez vrijednosti daje „nultu“ vrijednost bilo kojeg tipa. Zato funkcija radi i za brojeve i za stringove.

17.2 Predlošci klasa

Isti princip vrijedi za klase. Ovako otprilike izgledaju kontejneri iz standardne biblioteke:

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

template <typename T>
class Stog {
    vector<T> elementi;
public:
    void push(const T& x) { elementi.push_back(x); }
    void pop() {
        if (elementi.empty()) throw runtime_error("pop na praznom stogu");
        elementi.pop_back();
    }
    const T& top() const { return elementi.back(); }
    bool empty() const { return elementi.empty(); }
    size_t size() const { return elementi.size(); }
};

template <typename K, typename V>        // dva parametra tipa
struct Par {
    K kljuc;
    V vrijednost;
};

int main() {
    Stog<int> s;                         // kod klasa tip se (obično) navodi eksplicitno
    for (int i = 1; i <= 3; i++) s.push(i * 10);
    while (!s.empty()) { cout << s.top() << " "; s.pop(); }
    cout << "\n";

    Stog<string> rijeci;
    rijeci.push("prvi");
    rijeci.push("drugi");
    cout << rijeci.top() << " (" << rijeci.size() << ")\n";
```

```
    Par<string, int> p{"godine", 21};
    cout << p.kljuc << " = " << p.vrijednost << "\n";
}
```

IZLAZ PROGRAMA

```
30 20 10
drugi (2)
godine = 21
```

`Stog` nije tip — `Stog<int>` i `Stog<string>` jesu, i to dva potpuno različita tipa. (Od C++17 prevoditelj u nekim slučajevima može zaključiti parametre i za klase, npr. `vector v = {1, 2, 3};` — ali eksplicitno navođenje je jasnije.)

Parametri koji nisu tipovi. Parametar predloška može biti i konstantna vrijednost, npr. veličina: `template <typename T, int N> class Polje { T podaci[N]; };`. Upravo tako radi `std::array<int, 5>` — veličina je dio tipa i poznata je pri prevođenju.

17.3 Predlošci idu u zaglavlja

DEFINICIJA PREDLOŠKA MORA BITI VIDLJIVA

Kod običnih klasa deklaracija ide u `.h`, a definicije metoda u `.cpp`. **Kod predložaka to ne radi.** Da bi instancirao `Stog<double>`, prevoditelj mora vidjeti *cijeli* recept u trenutku kad prevodi datoteku koja ga koristi. Zato se predlošci pišu **u cijelosti u zaglavlju** (`.h` ili `.hpp`). Ako ih razdvojiš, dobit ćeš greške povezivanja (`undefined reference`).

17.4 Ograničenja predložaka: koncepti (C++20)

C++20 donosi **koncepte** — način da se zahtjevi na tip napišu eksplicitno, pa poruke o greškama postanu jasne:

```
#include <concepts>
#include <iostream>

template <std::integral T>              // T mora biti cjelobrojni tip
T nzd(T a, T b) {
    while (b != 0) { T r = a % b; a = b; b = r; }
    return a;
}

// skraćeni oblik: auto u parametrima također stvara predložak
auto kvadrat(auto x) { return x * x; }

int main() {
    std::cout << nzd(48, 18) << " " << kvadrat(1.5) << "\n";
    // nzd(4.5, 1.5);   // greška: "double does not satisfy integral"
}
```

IZLAZ PROGRAMA

```
6 2.25
```

UKRATKO

- `template <typename T>` — kod s tipom kao parametrom. Prevoditelj **instancira** zasebnu verziju za svaki korišteni tip, pri prevođenju, bez cijene u izvođenju.
- Tip se kod funkcija obično zaključuje iz argumenata; može se zadati eksplicitno: `f<double>(...)`.
- `Stog<int>` i `Stog<string>` su različiti tipovi. `T{}` daje nultu vrijednost tipa.
- Predlošci se pišu u cijelosti u zaglavljima.
- C++20 koncepti (`std::integral T`) izražavaju zahtjeve na tip i daju jasnije greške.

Zadaci

Zadatak 17.1 ★☆☆

Napiši predložak funkcije `void zamijeni(T& a, T& b)` i isprobaj ga na `int`, `double` i `string`. (Standardna biblioteka ima `std::swap` koji radi isto.)

Zadatak 17.2 ★★☆

Napiši predloške funkcija `int pronadji(const vector<T>& v, const T& x)` (vraća indeks ili −1) i `T najveciElement(const vector<T>& v)`. Što bi `najveciElement` trebao raditi s praznim vektorom?

Zadatak 17.3 ★★☆

Napiši predložak klase `Kutija<T>` koja čuva najviše jedan element: metode `stavi(T)`, `uzmi()` (vraća element i isprazni kutiju), `jePrazna()`. Isprobaj s `Kutija<int>` i `Kutija<vector<string>>`.

Zadatak 17.4 ★★★

Napiši predložak klase `Red<T, N>` (red čekanja, *queue*) fiksnog kapaciteta `N` implementiran kao **kružni međuspremnik** nad poljem `T podaci[N]`: metode `dodaj`, `ukloni`, `prvi`, `jePrazan`, `jePun`. Indeksi se „omataju“ pomoću `% N`.

18. STL kontejneri

Standardna biblioteka predložaka (*Standard Template Library*, STL) je skup gotovih, provjerenih i vrlo brzih struktura podataka i algoritama. Dobar C++ programer ne piše svoju listu, stog ili rječnik — zna koji kontejner iz STL-a odabrati i zašto. U ovom poglavlju upoznajemo kontejnere; u sljedećem, algoritme koji na njima rade.

18.1 Pregled

Vrsta	Kontejner	Ukratko
Slijedni	`vector`	dinamičko polje — **zadani izbor**
	`array`	polje fiksne veličine (veličina u tipu)
	`deque`	brzo dodavanje na oba kraja
	`list`	dvostruko povezana lista
Adapteri	`stack`	stog (LIFO — zadnji unutra, prvi van)
	`queue`	red (FIFO — prvi unutra, prvi van)
	`priority_queue`	uvijek daje najveći (ili najmanji) element
Asocijativni (sortirani)	`set`, `multiset`	skup jedinstvenih (ili ponovljenih) vrijednosti, sortiran
	`map`, `multimap`	parovi ključ → vrijednost, sortirani po ključu
Neuređeni (hash)	`unordered_set`	skup, bez redoslijeda, prosječno O(1)
	`unordered_map`	ključ → vrijednost, bez redoslijeda, prosječno O(1)

18.2 `vector` u dubinu

Osnove `vector`-a znamo iz poglavlja 7. Sada ono što razlikuje korisnika od stručnjaka.

Veličina i kapacitet. `size()` je broj elemenata, a `capacity()` broj elemenata za koje je memorija *već rezervirana*. Kad se kapacitet popuni, vektor seli podatke u veće polje (odjeljak 10.6):

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    for (int i = 0; i < 9; i++) {
        v.push_back(i);
        cout << v.size() << "/" << v.capacity() << " ";
    }
    cout << "\n";
    vector<int> w;
    w.reserve(1000);          // unaprijed rezerviraj — nema preseljenja do 1000
    cout << w.size() << "/" << w.capacity() << "\n";
}
```

IZLAZ PROGRAMA

```
1/1 2/2 3/4 4/4 5/8 6/8 7/8 8/8 9/16
0/1000
```

(Točan način rasta ovisi o implementaciji; ovo je GCC.) Ako unaprijed znaš koliko ćeš elemenata dodati, `reserve` izbjegava suvišna preseljenja.

Umetanje i brisanje u sredini rade preko iteratora (poglavlje 19) i koštaju O(n), jer se svi elementi iza moraju pomaknuti:

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40, 50};
    v.insert(v.begin() + 1, 15);        // umetni 15 na poziciju 1
    v.erase(v.begin() + 3);             // obriši element na poziciji 3 (30)
    v.erase(v.end() - 1);               // obriši zadnji
    for (int x : v) cout << x << " ";
    cout << "\n";

    vector<pair<string, int>> ljudi;
    ljudi.emplace_back("Ana", 21);      // gradi element izravno u vektoru
    ljudi.push_back({"Ivo", 23});       // gradi privremeni pa premješta
    cout << ljudi[1].first << " " << ljudi[1].second << "\n";
}
```

IZLAZ PROGRAMA

```
10 15 20 40
Ivo 23
```

PONIŠTAVANJE (INVALIDACIJA) ITERATORA I POKAZIVAČA

Svaka operacija koja može preseliti elemente (`push_back`, `insert`, `reserve`, `resize` ...) može **poništiti** sve postojeće pokazivače, reference i iteratore na elemente vektora. Nakon `push_back` stari pokazivač na `v[0]` može biti viseći. Klasična greška: dodavanje elemenata u vektor dok ga prolaziš range-for petljom.

18.3 `deque` i `list`

- **`deque`** (*double-ended queue*) — kao `vector`, ali uz `push_back`/`pop_back` ima i brze `push_front`/`pop_front`. Elementi nisu u jednom neprekinutom bloku memorije.
- **`list`** — dvostruko povezana lista: svaki element je zaseban čvor s pokazivačima na prethodni i sljedeći. Umetanje i brisanje *na poznatoj poziciji* je O(1), ali nema pristupa indeksom (`l[5]` ne postoji), a zbog raspršenosti po memoriji prolazak je znatno sporiji nego kod vektora.

SAVJET IZ PRAKSE: VECTOR GOTOVO UVIJEK POBJEĐUJE

Teorija kaže da je lista bolja za česta umetanja u sredinu. U praksi je `vector` često brži čak i tada, jer su njegovi elementi jedan do drugoga, a procesor iznimno brzo čita susjednu memoriju (priručna memorija, *cache*). Počni s `vector`-om; mijenjaj tek kad mjerenje pokaže da treba.

18.4 Adapteri: `stack`, `queue`, `priority_queue`

Adapteri su kontejneri s **ograničenim sučeljem** — namjerno dopuštaju samo neke operacije:

```cpp
#include <iostream>
#include <queue>
```

```cpp
#include <stack>
#include <vector>
using namespace std;

int main() {
    stack<int> s;                          // LIFO: kao hrpa tanjura
    for (int i = 1; i <= 3; i++) s.push(i);
    while (!s.empty()) { cout << s.top() << " "; s.pop(); }
    cout << "| ";

    queue<int> q;                          // FIFO: kao red na blagajni
    for (int i = 1; i <= 3; i++) q.push(i);
    while (!q.empty()) { cout << q.front() << " "; q.pop(); }
    cout << "| ";

    priority_queue<int> pq;                // uvijek daje NAJVEĆI
    for (int x : {5, 1, 8, 3}) pq.push(x);
    while (!pq.empty()) { cout << pq.top() << " "; pq.pop(); }
    cout << "| ";

    priority_queue<int, vector<int>, greater<int>> minpq;   // najmanji prvi
    for (int x : {5, 1, 8, 3}) minpq.push(x);
    cout << minpq.top() << "\n";
}
```

IZLAZ PROGRAMA

```
3 2 1 | 1 2 3 | 8 5 3 1 | 1
```

Primijeti da `pop()` **ne vraća** element — samo ga uklanja. Element pročitaš s `top()` ili `front()` *prije* `pop()`.

18.5 `set` — skup jedinstvenih, sortiranih vrijednosti

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {5, 1, 5, 3, 1, 9};      // duplikati se odbacuju
    s.insert(4);
    s.insert(3);                           // već postoji — ništa se ne događa
    s.erase(9);
    for (int x : s) cout << x << " ";      // uvijek sortirano!
    cout << "\n";
    cout << s.size() << " " << s.count(4) << " " << s.count(7) << "\n";
    if (s.find(5) != s.end()) cout << "5 je u skupu\n";
}
```

IZLAZ PROGRAMA

```
1 3 4 5
4 1 0
5 je u skupu
```

`set` je interno uravnoteženo binarno stablo, pa su umetanje, brisanje i traženje O(log n). Koristi ga kad trebaš jedinstvene vrijednosti i brzo pitanje „postoji li x?“. (C++20: `s.contains(x)`.)

18.6 `map` — rječnik ključ → vrijednost

`map` je vjerojatno drugi najkorišteniji kontejner, odmah nakon vektora. Čuva parove `(ključ, vrijednost)`, sortirane po ključu, s jedinstvenim ključevima:

```cpp
#include <iostream>
#include <map>
#include <sstream>
```

```cpp
#include <string>
using namespace std;

int main() {
    map<string, int> telefon;
    telefon["Ana"] = 12345;                 // dodaj ili promijeni
    telefon["Ivo"] = 67890;
    telefon.insert({"Eva", 55555});
    cout << telefon["Ana"] << " " << telefon.size() << "\n";

    // brojanje riječi — klasičan primjer
    string tekst = "a b a c b a";
    stringstream ss(tekst);
    map<string, int> brojac;
    string r;
    while (ss >> r) brojac[r]++;            // nepostojeći ključ -> kreira se s 0
    for (const auto& [rijec, n] : brojac)   // strukturirano vezanje (C++17)
        cout << rijec << ":" << n << " ";
    cout << "\n";

    auto it = telefon.find("Marko");        // traženje BEZ stvaranja elementa
    if (it == telefon.end()) cout << "Marko nije u imeniku\n";
    else cout << it->second << "\n";
}
```

IZLAZ PROGRAMA

```
12345 3
a:3 b:2 c:1
Marko nije u imeniku
```

ZAMKA: OPERATOR [] STVARA ELEMENT

`m[k]` za ključ koji ne postoji **dodaje** ga u mapu sa zadanom vrijednošću (0, prazan string...). To je praktično za brojanje, ali opasno kad samo želiš *provjeriti* postoji li ključ: `if (m["Marko"] == 0)` tiho doda Marka u mapu. Za provjeru koristi `find`, `count` ili (C++20) `contains`. Iz istog razloga `[]` ne radi na `const map` — koristi `at()`, koji baca iznimku ako ključ ne postoji.

Elementi mape su tipa `pair<const K, V>`: `.first` je ključ (ne smije se mijenjati — o njemu ovisi sortiranje), `.second` vrijednost. Oblik `auto& [kljuc, vrijednost]` (strukturirano vezanje) daje im ljepša imena.

18.7 Neuređeni kontejneri: `unordered_map` i `unordered_set`

Sučelje im je gotovo jednako kao kod `map` i `set`, ali su interno **tablice raspršivanja** (*hash tables*): iz ključa se izračuna broj (*hash*) koji odredi gdje element stoji. Traženje, umetanje i brisanje su **prosječno O(1)** — za velike količine podataka znatno brže od O(log n). Cijena: elementi **nisu sortirani** i redoslijed ispisa je nepredvidiv.

Pravilo: ako trebaš sortiran redoslijed (ili „prvi ključ veći od x“), koristi `map`/`set`. Ako ti treba samo brzo traženje po ključu, koristi `unordered_map`/`unordered_set`.

18.8 Kako odabrati kontejner

Treba mi...	Kontejner
niz elemenata, pristup po indeksu, dodavanje na kraj	`vector`
dodavanje/uklanjanje na oba kraja	`deque`
obrada „zadnji unutra, prvi van“ (povratak, zagrade, DFS)	`stack`
obrada redom dolaska (red čekanja, BFS)	`queue`
uvijek sljedeći najvažniji/najmanji element	`priority_queue`
jedinstvene vrijednosti, pitanje „postoji li?“	`set` / `unordered_set`
pridruživanje vrijednosti ključu (rječnik, brojanje)	`map` / `unordered_map`

Operacija	`vector`	`list`	`set` / `map`	`unordered_*`
pristup po indeksu	O(1)	—	—	—
dodavanje na kraj	O(1)*	O(1)	—	—
umetanje u sredinu	O(n)	O(1)**	O(log n)	O(1)*
traženje po vrijednosti/ključu	O(n)	O(n)	O(log n)	O(1)*

* amortizirano / prosječno ** kad već imaš iterator na poziciju

UKRATKO

- `vector` je zadani izbor. `size` ≠ `capacity`; `reserve` sprječava preseljenja; preseljenje poništava pokazivače i iteratore.
- `stack` (LIFO), `queue` (FIFO), `priority_queue` (najveći prvi) — `pop()` ne vraća element.
- `set` — jedinstveno i sortirano; `map` — ključ → vrijednost, sortirano po ključu. `m[k]` stvara element; za provjeru koristi `find` / `count`.
- `unordered_*` — hash, prosječno O(1), bez redoslijeda.

Zadaci

Zadatak 18.1 ★☆☆

Učitaj riječi do kraja unosa i ispiši: (a) koliko je **različitih** riječi (koristi `set`), (b) svaku riječ i broj pojavljivanja, abecednim redom (koristi `map`).

Zadatak 18.2 ★★☆

Pomoću `stack<char>` napiši funkciju koja provjerava jesu li zagrade `()[]{}` u izrazu ispravno uparene i ugniježđene. Zatim pomoću `stack` izračunaj vrijednost izraza u postfiksnoj notaciji, npr. `"3 4 + 2 *"` = 14.

Zadatak 18.3 ★★☆

Imenik: `map<string, vector<string>>` gdje je ključ ime, a vrijednost popis brojeva telefona. Napravi izbornik: dodaj broj, obriši osobu, ispiši brojeve osobe, ispiši sve abecedno.

Zadatak 18.4 ★★☆

Simuliraj red na blagajni pomoću `queue`: kupci dolaze s imenom i brojem artikala; blagajna obrađuje jedan artikl po „minuti“. Ispiši kojim redom i u kojoj minuti kupci odlaze.

Zadatak 18.5 ★★★

Zadan je vektor od `n` brojeva i broj `k`. Pomoću `priority_queue` pronađi `k` najvećih elemenata u vremenu O(n log k) (savjet: drži **min**-red veličine najviše `k`). Zašto je to bolje od sortiranja cijelog vektora kad je `n` velik, a `k` mali?

Zadatak 18.6 ★★★

Pomoću `unordered_map` riješi problem „dva zbroja“: u vektoru pronađi dva različita elementa čiji je zbroj jednak zadanom broju, u **jednom** prolazu (O(n)). Usporedi s rješenjem s dvije ugniježđene petlje.

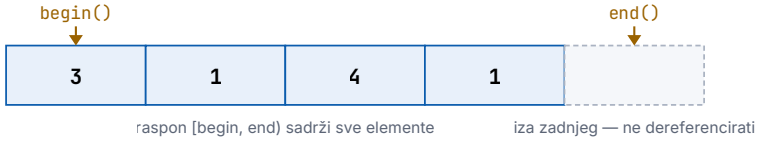
19. Iteratori, algoritmi i lambda-izrazi

STL ima genijalnu arhitekturu: **kontejneri** čuvaju podatke, **algoritmi** ih obrađuju, a **iteratori** su ljepilo između njih. Zahvaljujući tome, jedna funkcija `sort` radi na vektoru, polju i dequeu, a `find` na svakom kontejneru. Kad tome dodaš **lambda-izraze** — male funkcije napisane na mjestu upotrebe — dobiješ moderan, kratak i izražajan stil programiranja.

19.1 Iteratori

Iterator je objekt koji pokazuje na element kontejnera i zna prijeći na sljedeći. Ponaša se poput pokazivača iz poglavlja 9: `*it` daje element, `++it` prelazi na sljedeći, `it1 == it2` uspoređuje pozicije.

Svaki kontejner ima `begin()` — iterator na **prvi** element — i `end()` — iterator **iza zadnjeg** elementa. `end()` ne pokazuje na element i ne smije se dereferencirati; služi kao oznaka kraja. Takav raspon zovemo **poluotvoreni** `[begin, end)`.



Slika 19.1 — Iteratori begin() i end(): poluotvoreni raspon

```cpp
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {3, 1, 4, 1};
    for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
        *it *= 10;                                  // mijenjamo element
    for (auto it = v.cbegin(); it != v.cend(); ++it) // auto + const iterator (samo čitanje)
        cout << *it << " ";
    cout << "\n";

    map<string, int> m = {{"b", 2}, {"a", 1}};
    for (auto it = m.begin(); it != m.end(); ++it)   // isti obrazac za mapu
        cout << it->first << "=" << it->second << " ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
30 10 40 10
a=1 b=2
```

Primijeti da je petlja **ista** za vektor i mapu — to je smisao iteratora. Range-for (`for (auto& x : v)`) zapravo je samo skraćenica za ovakvu petlju s iteratorima. Zašto onda uopće pisati iteratore ručno? Zato što ih **algoritmi** traže kao argumente, i zato što neke operacije (`insert`, `erase`) rade s pozicijama.

19.2 Lambda-izrazi

Mnogi algoritmi primaju **funkciju** kao argument — npr. kriterij sortiranja ili uvjet pretraživanja. Pisati zasebnu imenovanu funkciju za svaki takav uvjet je nezgrapno. **Lambda** je bezimena funkcija napisana izravno na mjestu upotrebe:

```
[hvatanje](parametri) -> povratni_tip { tijelo }     // -> tip se obično izostavlja
```

```
#include <iostream>
using namespace std;

int main() {
    auto kvadrat = [](int x) { return x * x; };       // lambda spremljena u varijablu
    cout << kvadrat(7) << "\n";

    int granica = 10;
    auto veciOdGranice = [granica](int x) { return x > granica; };  // hvata KOPIJU
    cout << veciOdGranice(15) << veciOdGranice(5) << "\n";

    int brojac = 0;
    auto broji = [&brojac]() { brojac++; };           // hvata PO REFERENCI
    broji(); broji(); broji();
    cout << brojac << "\n";
}
```

IZLAZ PROGRAMA

```
49
10
3
```

Uglate zagrade određuju **koje vanjske varijable lambda smije koristiti** i kako:

Hvatanje	Značenje
`[]`	ništa izvana
`[x]`	kopija od `x` (u trenutku stvaranja lambde)
`[&x]`	referenca na `x` — lambda vidi i mijenja original
`[=]`	kopije svih korištenih varijabli
`[&]`	reference na sve korištene varijable
`[this]`	pristup članovima objekta (unutar metode)

HVATANJE REFERENCOM I ŽIVOTNI VIJEK

Lambda koja hvata po referenci (`[&]`) ne smije nadživjeti varijable koje je uhvatila. Ako je vratiš iz funkcije ili spremiš za kasnije, reference će visjeti. Za lambde koje se odmah koriste (kao argument algoritmu) `[&]` je sasvim u redu.

19.3 Najvažniji algoritmi

Algoritmi su u zaglavljima `<algorithm>` i `<numeric>`. Gotovo svi primaju raspon iteratora `(begin, end)`, a mnogi i funkciju — najčešće lambdu.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <string>
#include <vector>
using namespace std;

struct Student { string ime; double prosjek; };

void ispisi(const vector<int>& v) { for (int x : v) cout << x << " "; cout << "\n"; }
```

```cpp
int main() {
    vector<int> v = {5, 3, 8, 1, 9, 2, 8};

    sort(v.begin(), v.end());                                  // uzlazno
    ispisi(v);
    sort(v.begin(), v.end(), [](int a, int b) { return a > b; });   // silazno
    ispisi(v);

    cout << "zbroj: " << accumulate(v.begin(), v.end(), 0) << "\n";
    cout << "max: " << *max_element(v.begin(), v.end())
         << ", min: " << *min_element(v.begin(), v.end()) << "\n";
    auto paran = [](int x) { return x % 2 == 0; };
    cout << "broj osmica: " << count(v.begin(), v.end(), 8)
         << ", parnih: " << count_if(v.begin(), v.end(), paran) << "\n";

    auto it = find(v.begin(), v.end(), 3);                     // traži vrijednost
    if (it != v.end()) cout << "3 je na indeksu " << (it - v.begin()) << "\n";
    auto it2 = find_if(v.begin(), v.end(), [](int x) { return x < 3; });  // traži po uvjetu
    cout << "prvi manji od 3: " << *it2 << "\n";

    cout << boolalpha << all_of(v.begin(), v.end(), [](int x) { return x > 0; }) << "\n";

    reverse(v.begin(), v.end());
    ispisi(v);

    transform(v.begin(), v.end(), v.begin(), [](int x) { return x * 10; });
    ispisi(v);

    // "erase-remove": brisanje svih elemenata koji zadovoljavaju uvjet
    v.erase(remove_if(v.begin(), v.end(), [](int x) { return x > 50; }), v.end());
    ispisi(v);                                                 // C++20: erase_if(v, uvjet);

    // sortiranje struktura po kriteriju
    vector<Student> s = {{"Ana", 4.2}, {"Ivo", 3.8}, {"Eva", 4.9}};
    sort(s.begin(), s.end(), [](const Student& a, const Student& b) {
        return a.prosjek > b.prosjek;
    });
    for (const auto& st : s) cout << st.ime << " ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
1 2 3 5 8 8 9
9 8 8 5 3 2 1
zbroj: 36
max: 9, min: 1
broj osmica: 2, parnih: 3
3 je na indeksu 4
prvi manji od 3: 2
true
1 2 3 5 8 8 9
10 20 30 50 80 80 90
10 20 30 50
Eva Ana Ivo
```

Algoritam	Što radi
`sort(b, e)` / `sort(b, e, cmp)`	sortira; `cmp(a, b)` vraća `true` ako `a` treba ići prije `b`
`stable_sort`	sortira i čuva redoslijed jednakih elemenata
`find`, `find_if`	iterator na prvi pronađeni ili `end`
`count`, `count_if`	broj elemenata jednakih vrijednosti / koji zadovoljavaju uvjet
`accumulate(b, e, pocetak)`	zbroj (ili druga operacija) — iz `<numeric>`
`min_element`, `max_element`	**iterator** na najmanji/najveći
`all_of`, `any_of`, `none_of`	vrijedi li uvjet za sve / neki / nijedan element
`reverse`, `rotate`, `fill`, `iota`	obrni; rotiraj; popuni vrijednošću; popuni s 0, 1, 2…
`transform`	primijeni funkciju na svaki element
`remove_if` + `erase`	ukloni elemente po uvjetu
`unique` + `erase`	ukloni **susjedne** duplikate (prvo sortiraj!)
`binary_search`, `lower_bound`	traženje u **sortiranom** rasponu, O(log n)

ZAŠTO REMOVE_IF NE BRIŠE?

Algoritmi rade s iteratorima i **ne znaju ništa o kontejneru** — ne mogu mu promijeniti veličinu. `remove_if` zato samo *premjesti* elemente koje treba zadržati na početak i vrati iterator na „novi kraj“. Iza njega ostaje smeće. Tek `erase(noviKraj, end())` — metoda samog vektora — stvarno skraćuje vektor. Otud idiom *erase-remove*. U C++20 postoji jednostavnije `std::erase_if(v, uvjet)`.

KOMPARATOR MORA BITI „STROGI“

Funkcija za `sort` mora vraćati `true` samo kad je `a` **strogo** prije `b`. Komparator s `<=` umjesto `<` je nedefinirano ponašanje — `sort` može pasti ili pisati izvan vektora.

19.4 Rasponi (ranges, C++20)

C++20 donosi verzije algoritama koje primaju cijeli kontejner, bez `begin`/`end`, i **poglede** (*views*) koji se mogu nizati operatorom `|`:

```cpp
#include <algorithm>
#include <iostream>
#include <ranges>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {5, 3, 8, 1, 9, 2};
    ranges::sort(v);                                   // bez begin/end
    auto rezultat = v | views::filter([](int x) { return x % 2 == 1; })
                      | views::transform([](int x) { return x * x; });
    for (int x : rezultat) cout << x << " ";           // kvadrati neparnih
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
1 9 25 81
```

Pogledi su „lijeni“: ništa se ne računa dok se ne prolazi kroz rezultat, i ne stvaraju se privremeni vektori.

SAVJET PROFESORA: PRVO POTRAŽI ALGORITAM

Kad napišeš petlju, zapitaj se: *postoji li algoritam koji ovo radi?* Petlja „nađi najveći“ je `max_element`, „izbroji one koji...“ je `count_if`, „je li ijedan...“ je `any_of`. Algoritam ima ime koje opisuje namjeru, provjeren je i ne može imati grešku za jedan. Na ispitima se ponekad traži da petlju napišeš ručno — ali u praksi, algoritam.

UKRATKO

- Iterator se ponaša kao pokazivač: `*it`, `++it`, `it != end`. Raspon `[begin, end)`; `end` se ne dereferencira.
- Lambda: `[hvatanje](parametri) { tijelo }`. `[x]` kopija, `[&x]` referenca, `[=]`/`[&]` sve.
- Ključni algoritmi: `sort` (s komparatorom), `find`/`find_if`, `count`/`count_if`, `accumulate`, `min_element`/`max_element`, `transform`, `all_of`/`any_of`, erase-remove.
- Komparator mora biti strog (`<`, ne `<=`).
- C++20: `ranges::sort(v)`, pogledi `views::filter` / `views::transform`.

Zadaci

Zadatak 19.1 ★☆☆

Za vektor cijelih brojeva, **bez ijedne ručno napisane petlje**, ispiši: zbroj, prosjek, najveći element, broj negativnih elemenata i je li ijedan element djeljiv sa 7.

Zadatak 19.2 ★★☆

Imaš `vector<Student>` (ime, prezime, godina, prosjek). Sortiraj studente: prvo po godini uzlazno, a unutar iste godine po prosjeku silazno, a kod istog prosjeka po prezimenu abecedno. Koristi jedan `sort` s lambdom.

Zadatak 19.3 ★★☆

Iz vektora stringova ukloni sve prazne stringove i sve stringove kraće od 3 znaka (erase-remove), zatim svaki preostali pretvori u velika slova (`transform`) i izbaci duplikate (`sort` + `unique`).

Zadatak 19.4 ★★☆

Napiši funkciju `vector<int> filtriraj(const vector<int>& v, F uvjet)` kao predložak koji prima bilo kakvu lambdu. Pozovi je s tri različite lambde, od kojih jedna hvata vanjsku varijablu.

Zadatak 19.5 ★★★

Učitaj tekst i ispiši **10 najčešćih riječi** s brojem pojavljivanja. Koristi `map` (ili `unordered_map`) za brojanje, prebaci parove u `vector<pair<string,int>>` i sortiraj lambdom. Kod izjednačenja, riječi poredaj abecedno.

20. Pametni pokazivači

U poglavlju 10 vidjeli smo četiri klasične greške s ručnim `new`/`delete`, a u poglavlju 13 rješenje: RAII — resurs se oslobađa u destruktoru. **Pametni pokazivači** su upravo to: klase iz zaglavlja `<memory>` koje se ponašaju kao pokazivači, ali **same brišu objekt** kad više nije potreban. U modernom C++-u ručni `delete` u običnom kodu praktički ne postoji.

20.1 Pojam vlasništva

Ključno pitanje kod svakog dinamičkog objekta je: **tko je njegov vlasnik** — tko je odgovoran da ga obriše? Sirovi pokazivač `Student*` na to ne odgovara. Je li funkcija koja ga je vratila očekuje da ga ja obrišem? Smijem li ga spremiti za kasnije? Pametni pokazivači vlasništvo **zapisuju u tip**:

Tip	Vlasništvo	Kad koristiti
`unique_ptr<T>`	**jedan** vlasnik	zadani izbor za dinamičke objekte
`shared_ptr<T>`	**više** vlasnika, objekt živi dok postoji ijedan	kad vlasništvo stvarno dijeli više dijelova programa
`weak_ptr<T>`	**nije** vlasnik, samo promatra `shared_ptr` objekt	prekidanje ciklusa, „ako još postoji“
`T*` ili `T&`	**nije** vlasnik	samo za pristup objektu koji posjeduje netko drugi

20.2 `unique_ptr` — jedini vlasnik

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <utility>
using namespace std;

struct Resurs {
    string ime;
    Resurs(string i) : ime(std::move(i)) { cout << "stvoren " << ime << "\n"; }
    ~Resurs() { cout << "unisten " << ime << "\n"; }
};

unique_ptr<Resurs> napravi(const string& ime) {
    return make_unique<Resurs>(ime);        // vlasništvo se predaje pozivatelju
}

int main() {
    auto a = make_unique<Resurs>("A");      // umjesto: new Resurs("A")
    cout << a->ime << "\n";                 // koristi se kao pokazivač: -> i *

    // auto b = a;                          // GREŠKA: unique_ptr se ne može kopirati
    auto b = std::move(a);                  // ali može se PREMJESTITI
    cout << (a == nullptr ? "a je prazan" : "a nije prazan") << "\n";

    {
        auto c = napravi("C");
    }                                       // c izlazi iz dosega -> objekt se briše
    b.reset();                              // eksplicitno brisanje prije kraja
    cout << "kraj main\n";
}
```

IZLAZ PROGRAMA

```
stvoren A
A
a je prazan
stvoren C
unisten C
unisten A
kraj main
```

- `make_unique<T>(argumenti)` stvara objekt i vraća `unique_ptr`. Nema `new`, pa nema ni mogućnosti da se `delete` zaboravi.
- `unique_ptr` se **ne može kopirati** — dvije kopije značile bi dva vlasnika i dvostruko brisanje. Prevoditelj to jednostavno ne dopušta. Može se **premjestiti** (`std::move`) — vlasništvo prelazi na novi pokazivač, a stari postaje prazan.
- Kad `unique_ptr` nestane (kraj bloka, `reset()`, uništenje objekta koji ga sadrži), briše i objekt. Veličina i brzina jednaki su sirovom pokazivaču — **nema nikakve cijene**.
- `p.get()` vraća sirovi pokazivač (za funkcije koje samo koriste objekt), `unique_ptr<T[]>` postoji i za polja (iako je `vector` gotovo uvijek bolji).

Polimorfizam s `vector<unique_ptr<Oblik>>` iz poglavlja 16 najčešća je upotreba: vektor posjeduje oblike, i kad nestane vektor, nestanu i svi oblici — pravilnim, virtualnim destruktorima.

20.3 `shared_ptr` — zajedničko vlasništvo

Ponekad objekt stvarno nema jednog vlasnika: npr. tekstura u igri koju koristi više modela, pa ne znaš koji će zadnji prestati. `shared_ptr` broji koliko pokazivača dijeli objekt (**brojanje referenci**, *reference counting*) i briše objekt kad broj padne na nulu.

```cpp
#include <iostream>
#include <memory>
using namespace std;

struct Tekstura {
    ~Tekstura() { cout << "tekstura obrisana\n"; }
};

int main() {
    shared_ptr<Tekstura> t1 = make_shared<Tekstura>();
    cout << "vlasnika: " << t1.use_count() << "\n";
    {
        shared_ptr<Tekstura> t2 = t1;              // kopija -> +1
        auto t3 = t1;                              // +1
        cout << "vlasnika: " << t1.use_count() << "\n";
    }                                              // t2 i t3 nestaju -> -2
    cout << "vlasnika: " << t1.use_count() << "\n";
    t1.reset();                                    // zadnji vlasnik -> brisanje
    cout << "kraj\n";
}
```

IZLAZ PROGRAMA

```
vlasnika: 1
vlasnika: 3
vlasnika: 1
tekstura obrisana
kraj
```

`shared_ptr` ima cijenu: dodatni blok memorije za brojač i (sigurno za dretve) povećavanje/smanjivanje brojača pri svakoj kopiji. Zato **nije zadani izbor** — koristi `unique_ptr` dok ti stvarno ne treba dijeljeno vlasništvo.

20.4 `weak_ptr` i problem ciklusa

Ako dva objekta imaju `shared_ptr` jedan na drugoga, brojači im nikad ne padnu na nulu — ni kad ih nitko drugi ne koristi. To je curenje memorije unatoč pametnim pokazivačima:

```
#include <iostream>
#include <memory>
using namespace std;

struct Osoba {
    string ime;
    shared_ptr<Osoba> partner;        // shared -> shared: CIKLUS!
    weak_ptr<Osoba> prijatelj;        // weak ne povećava brojač
    Osoba(string i) : ime(i) {}
    ~Osoba() { cout << "~" << ime << "\n"; }
};

int main() {
    {
        auto a = make_shared<Osoba>("Ana");
        auto b = make_shared<Osoba>("Ivo");
        a->partner = b;
        b->partner = a;               // ciklus: nitko od njih se nikad neće obrisati
    }
    cout << "(Ana i Ivo nisu obrisani!)\n";
    {
        auto c = make_shared<Osoba>("Eva");
        auto d = make_shared<Osoba>("Luka");
        c->prijatelj = d;
        d->prijatelj = c;             // weak_ptr: nema ciklusa vlasništva
        if (auto p = c->prijatelj.lock())      // lock(): shared_ptr ako objekt još živi
            cout << c->ime << " -> " << p->ime << "\n";
    }
}
```

IZLAZ PROGRAMA

```
(Ana i Ivo nisu obrisani!)
Eva -> Luka
~Luka
~Eva
```

`weak_ptr` pokazuje na objekt, ali ga ne drži na životu. Prije upotrebe pozoveš `lock()`, koji vraća `shared_ptr` ako objekt još postoji ili prazan pokazivač ako je već obrisan. Tipičan obrazac: roditelj posjeduje djecu (`shared_ptr` ili `unique_ptr`), a djeca na roditelja pokazuju s `weak_ptr` (ili sirovim pokazivačem).

PRAVILA ZA SVAKODNEVNI RAD

1. Objekt stvaraj na stogu ako možeš (`Student s;`). Dinamički samo kad moraš (polimorfizam, velik objekt, nepoznat životni vijek).
2. Za dinamički objekt: `make_unique`. Za dijeljeno vlasništvo: `make_shared`.
3. Funkcija koja samo **koristi** objekt prima `T&`, `const T&` ili `T*` — ne pametni pokazivač. Pametni pokazivač u parametru znači da funkcija sudjeluje u **vlasništvu**.
4. Nikad ne piši `delete`. Ako ga napišeš, zapitaj se zašto.

UKRATKO

- Pametni pokazivači zapisuju vlasništvo u tip i sami brišu objekt (RAII).
- `unique_ptr`: jedan vlasnik, bez kopiranja, može se premjestiti, bez cijene. Stvara se s `make_unique`. **Zadani izbor.**
- `shared_ptr`: brojanje referenci, briše kad zadnji vlasnik nestane. Stvara se s `make_shared`.
- `weak_ptr`: promatra bez vlasništva; `lock()` prije upotrebe. Prekida cikluse.
- Sirovi pokazivači i reference — samo za pristup bez vlasništva.

Zadaci

Zadatak 20.1 ★☆☆

Prepiši primjer dinamičkog objekta iz 10.2 i dinamičkog polja iz 10.3 tako da ne sadrži ni `new` ni `delete` (koristi `unique_ptr` odnosno `vector`).

Zadatak 20.2 ★★☆

Napravi jednostruko povezanu listu cijelih brojeva u kojoj svaki čvor ima `int vrijednost` i `unique_ptr<Cvor> sljedeci`. Implementiraj `dodajNaPocetak`, `ispisi` i `zbroj`. Zašto ne treba pisati destruktor koji briše čvorove?

Zadatak 20.3 ★★☆

Objasni što ispisuje program i koliko je vlasnika u svakom trenutku:

```
auto p = make_shared<int>(5);
auto q = p;
weak_ptr<int> w = p;
cout << p.use_count() << " ";
p.reset();
cout << q.use_count() << " " << w.expired() << " ";
q.reset();
cout << w.expired() << "\n";
```

Zadatak 20.4 ★★★

Modeliraj stablo mapa (direktorija): `Mapa` ima ime, `vector<unique_ptr<Mapa>> podmape` i sirovi pokazivač `Mapa* roditelj`. Napiši metodu `dodajPodmapu(ime)` koja vraća pokazivač na novu mapu, i metodu `punaPutanja()` koja vraća npr. `/home/ana/projekti`. Tko posjeduje koga?

21. Iznimke (exceptions)

Što treba učiniti funkcija `stoi("abc")`? Ne može vratiti broj, jer broja nema. Vratiti 0 ili −1? Ali to su valjani brojevi — pozivatelj ne bi znao je li došlo do greške. **Iznimke** su mehanizam kojim funkcija javlja da **ne može obaviti svoj posao**, a greška se obrađuje na mjestu koje zna što s njom učiniti — koje može biti nekoliko razina poziva iznad.

21.1 `throw`, `try` i `catch`

```cpp
#include <iostream>
#include <stdexcept>
using namespace std;

double dijeli(double a, double b) {
    if (b == 0) throw invalid_argument("dijeljenje s nulom");   // BACI iznimku
    return a / b;
}

int main() {
    try {                                       // "pokušaj"
        cout << dijeli(10, 4) << "\n";
        cout << dijeli(1, 0) << "\n";           // ovdje se baca iznimka...
        cout << "ovo se nikad ne ispise\n";     // ...pa se ostatak bloka preskače
    }
    catch (const invalid_argument& e) {         // "uhvati" iznimku tog tipa
        cout << "Greska: " << e.what() << "\n";
    }
    cout << "program nastavlja\n";
}
```

IZLAZ PROGRAMA

```
2.5
Greska: dijeljenje s nulom
program nastavlja
```

Kad se izvrši `throw`, normalno izvršavanje **odmah prestaje**. Program „izlazi“ iz funkcija jednu po jednu (bez vraćanja vrijednosti), sve dok ne naiđe na `try` blok s odgovarajućim `catch`-om. Tamo se izvrši kod za obradu i program nastavlja **iza** cijelog `try-catch`. Ako nitko ne uhvati iznimku, program se prekida (`std::terminate`).

21.2 Standardne iznimke

Standardna biblioteka ima hijerarhiju klasa iznimaka u zaglavlju `<stdexcept>`, a sve nasljeđuju `std::exception`, koja ima virtualnu metodu `what()` s opisom greške:

Klasa	Kad se koristi / tko je baca
`exception`	bazna klasa svih standardnih iznimaka
`logic_error`	greška u logici programa, mogla se izbjeći provjerom
`invalid_argument`	neispravan argument — npr. `stoi("abc")`
`out_of_range`	indeks izvan granica — npr. `v.at(100)`, `map::at`
`domain_error`, `length_error`	vrijednost izvan domene; prevelika duljina
`runtime_error`	greška koja se može otkriti tek tijekom izvođenja
`overflow_error`, `range_error`	preljev, rezultat izvan raspona
`bad_alloc`	`new` nije uspio dobiti memoriju

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};
    string ulazi[] = {"1", "42", "abc", "99999999999"};
    for (const string& s : ulazi) {
        try {
            int n = stoi(s);          // može baciti invalid_argument ili out_of_range
            int element = v.at(n);    // može baciti out_of_range
            cout << "broj: " << n << ", element: " << element << "\n";
        }
        catch (const invalid_argument& e) { cout << "'" << s << "' nije broj\n"; }
        catch (const out_of_range& e)     { cout << "'" << s << "' je izvan raspona\n"; }
        catch (const exception& e)        { cout << "nesto drugo: " << e.what() << "\n"; }
    }
}
```

IZLAZ PROGRAMA

```
broj: 1, element: 2
'42' je izvan raspona
'abc' nije broj
'99999999999' je izvan raspona
```

Za `"42"` pretvorba uspije, ali `v.at(42)` baca `out_of_range`. Za `"99999999999"` baca ga već `stoi`, jer broj ne stane u `int`.

Pravila za `catch`:

- **Hvataj po `const` referenci** (`const exception& e`). Po vrijednosti bi se iznimka kopirala i „odrezala“ na bazni tip (poglavlje 16), pa bi `what()` izgubio poruku izvedene klase.
- `catch` blokovi se provjeravaju **redom**, izvršava se prvi koji odgovara. Zato **specifičniji tipovi idu prvi**, a `exception` na kraj — inače bi on uhvatio sve.
- `catch (...)` hvata baš sve (i ono što ne nasljeđuje `exception`). Koristi ga kao zadnju liniju obrane.
- Unutar `catch` bloka, `throw;` (bez argumenta) ponovno baca istu iznimku dalje — korisno kad je samo želiš zapisati u dnevnik.

21.3 Vlastite iznimke

Vlastitu iznimku napraviš nasljeđivanjem neke standardne — najčešće `runtime_error` ili `logic_error`:

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class NedovoljnoSredstava : public runtime_error {
    double manjak;
public:
    NedovoljnoSredstava(double m)
        : runtime_error("nedovoljno sredstava, nedostaje " + to_string(m)), manjak(m) {}
    double getManjak() const { return manjak; }
};

class Racun {
    double stanje = 100;
public:
    void isplati(double iznos) {
        if (iznos <= 0) throw invalid_argument("iznos mora biti pozitivan");
        if (iznos > stanje) throw NedovoljnoSredstava(iznos - stanje);
        stanje -= iznos;
    }
};

int main() {
    Racun r;
    try {
        r.isplati(30);
        r.isplati(150);
    }
    catch (const NedovoljnoSredstava& e) {
        cout << e.what() << " (manjak: " << e.getManjak() << ")\n";
    }
}
```

IZLAZ PROGRAMA

```
nedovoljno sredstava, nedostaje 80.000000 (manjak: 80)
```

21.4 Odmatanje stoga i RAII

Što se događa s lokalnim objektima funkcija iz kojih iznimka „izleti“? C++ jamči da se za sve njih **pozovu destruktori** — to se zove **odmatanje stoga** (*stack unwinding*). Upravo zato RAII i iznimke idu ruku pod ruku:

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

struct Straza {
    string ime;
    Straza(string i) : ime(i) { cout << "+" << ime << " "; }
    ~Straza() { cout << "-" << ime << " "; }
};

void duboko() {
    Straza c("C");
    throw runtime_error("problem");
}
void srednje() {
    Straza b("B");
    duboko();
    cout << "(nikad) ";
}

int main() {
```

```
    try {
        Straza a("A");
        srednje();
    } catch (const exception& e) {
        cout << "| uhvaceno: " << e.what() << "\n";
    }
}
```

IZLAZ PROGRAMA

```
+A +B +C -C -B -A | uhvaceno: problem
```

Sve tri „straže“ uredno su uništene prije nego što se ušlo u `catch`. Da je `Straza` bila `unique_ptr`, `vector` ili otvorena datoteka — sve bi bilo oslobođeno i zatvoreno. S ručnim `new`/`delete` taj `delete` bi bio preskočen, i memorija bi iscurila.

DESTRUKTOR NE SMIJE BACATI IZNIMKU

Ako destruktor baci iznimku *dok se stog već odmata zbog druge iznimke*, program se odmah prekida. Zato su destruktori implicitno `noexcept` i ne smiju propustiti iznimku van. Isto vrijedi i za operacije premještanja (poglavlje 13).

21.5 Kada koristiti iznimke

- **Za iznimne situacije**, ne za normalan tok programa. Korisnik koji upiše krivi broj u izborniku nije iznimka — to je očekivano i obrađuje se `if`-om. Datoteka s konfiguracijom koja ne postoji, pokvareni podaci, nedostatak memorije — to jesu.
- **Kad konstruktor ne može napraviti ispravan objekt.** Konstruktor ne može vratiti kod greške, a objekt „napola napravljen“ ne smije postojati. Bacanje iznimke iz konstruktora je standardni način: objekt tada uopće ne nastaje.
- **Kad mjesto otkrivanja greške ne zna kako je obraditi.** Funkcija `ucitajKonfiguraciju` ne zna treba li program prikazati poruku, koristiti zadane vrijednosti ili završiti — to zna tek kod koji ju je pozvao.
- `noexcept` na funkciji obećava da neće baciti iznimku. Ako ipak baci, program se prekida. Koristi ga za destruktore, premještanja i jednostavne funkcije za koje si siguran.

UKRATKO

- `throw` baca iznimku; izvršavanje skače do najbližeg odgovarajućeg `catch`-a, preskačući ostatak koda.
- Hvataj po `const&`; specifične iznimke prije općenitih; `catch (...)` za sve ostalo.
- Standardna hijerarhija: `exception` → `logic_error` (`invalid_argument`, `out_of_range`), `runtime_error` …
- Vlastite iznimke nasljeđuju `runtime_error`/`logic_error`.
- Odmatanje stoga zove destruktore svih lokalnih objekata — RAII zato jamči oslobađanje resursa i kod iznimaka.
- Iznimke su za iznimne situacije; destruktori ne smiju bacati.

Zadaci

Zadatak 21.1 ★☆☆

Napiši funkciju `int ucitajBroj(const string& s, int min, int max)` koja pretvara string u broj pomoću `stoi` i baca `out_of_range` ako broj nije u zadanom rasponu. U `main`-u je pozovi za nekoliko ulaza i uhvati sve moguće iznimke s različitim porukama.

Zadatak 21.2 ★★☆

Klasi `Razlomak` iz poglavlja 14 dodaj bacanje iznimke `invalid_argument` kad je nazivnik 0 (u konstruktoru i pri dijeljenju s nulom). Pokaži da neispravan razlomak nikad ne može nastati.

Zadatak 21.3 ★★☆

Napravi hijerarhiju iznimaka za bankovni sustav: `BankaGreska : runtime_error`, a iz nje `NedovoljnoSredstava`, `RacunBlokiran` i `NepostojeciRacun`. Napiši funkciju `prijenos(izvor, odrediste, iznos)` koja može baciti svaku od njih, i `main` koji ih obrađuje — jednu posebno, a ostale zajedno preko bazne klase.

Zadatak 21.4 ★★★

Što ispisuje program? Objasni redoslijed.

```
struct X { int id; X(int i) : id(i) { cout << "X" << id << " "; }
                   ~X() { cout << "~X" << id << " "; } };
void f(int n) {
    X x(n);
    if (n == 3) throw n;
    f(n + 1);
}
int main() {
    try { f(1); }
    catch (int e) { cout << "uhvacen " << e; }
    cout << "\n";
}
```

22. Rad s datotekama

Podaci koje program drži u memoriji nestaju kad program završi. Da bi preživjeli, moraju u datoteku. Dobra vijest: s datotekama u C++-u radiš **gotovo potpuno isto kao s `cin` i `cout`** — to su samo drugi tokovi. Sve što znaš o `>>`, `<<` i `getline` vrijedi i ovdje.

22.1 Tokovi za datoteke

Zaglavlje `<fstream>` donosi tri klase:

Klasa	Namjena	Analogno
`ofstream`	pisanje u datoteku (*output file stream*)	`cout`
`ifstream`	čitanje iz datoteke (*input file stream*)	`cin`
`fstream`	i čitanje i pisanje	—

```cpp
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
    // --- pisanje ---
    ofstream izlaz("ocjene.txt");           // otvara (i stvara/prazni) datoteku
    if (!izlaz) {                           // provjera: je li otvaranje uspjelo?
        cerr << "Ne mogu otvoriti datoteku za pisanje\n";
        return 1;
    }
    izlaz << "Ana 5\n" << "Ivo 3\n" << "Eva 4\n";
    izlaz.close();                          // (ne mora se — destruktor je zatvara)

    // --- čitanje riječ po riječ ---
    ifstream ulaz("ocjene.txt");
    if (!ulaz) { cerr << "Ne mogu otvoriti datoteku\n"; return 1; }
    string ime;
    int ocjena, zbroj = 0, n = 0;
    while (ulaz >> ime >> ocjena) {         // čitaj DOK čitanje uspijeva
        cout << ime << ": " << ocjena << "\n";
        zbroj += ocjena;
        n++;
    }
    cout << "Prosjek: " << static_cast<double>(zbroj) / n << "\n";
}                                           // ulaz se ovdje automatski zatvara (RAII)
```

IZLAZ PROGRAMA

```
Ana: 5
Ivo: 3
Eva: 4
Prosjek: 4
```

Nekoliko važnih detalja:

- **Uvijek provjeri je li datoteka otvorena.** Ako datoteka ne postoji ili nemaš prava pristupa, tok je u stanju greške i sva čitanja tiho ne uspijevaju. Provjera `if (!ulaz)` ili `if (!ulaz.is_open())`.
- `cerr` je izlazni tok za poruke o greškama. Ispisuje se na ekran kao `cout`, ali se može odvojeno preusmjeriti.

- Datoteka se **zatvara sama** kad objekt toka nestane (RAII). `close()` trebaš samo ako istu datoteku želiš ponovno otvoriti ili ako je druga strana treba vidjeti prije kraja funkcije.
- **Relativna putanja** (`"ocjene.txt"`) traži se u **radnom direktoriju** programa. U Visual Studiju to je obično mapa projekta, a ne mapa s `.exe` datotekom — čest razlog „datoteka ne postoji, a vidim je!“.

ANTIOBRAZAC: WHILE (!ULAZ.EOF())

Ovo se viđa u mnogim starim skriptama, ali je **pogrešno**:

✗ NEISPRAVNO

```
while (!ulaz.eof()) {
    ulaz >> x;
    zbroj += x;     // zadnji broj se zbroji DVAPUT
}
```

`eof()` postaje istinit tek **nakon** što čitanje pokuša pročitati iza kraja. Zadnji uspješan `>>` još ne postavlja `eof`, pa petlja ide još jednom; taj `>>` ne uspije, `x` ostaje star (ili postaje 0), i obradiš smeće. Ispravno je uvjet petlje vezati uz **samo čitanje**: `while (ulaz >> x)` ili `while (getline(ulaz, red))`.

22.2 Čitanje red po red i dodavanje u datoteku

`getline(ulaz, red)` čita cijeli red, baš kao s `cin`. Kad datoteku treba **dopuniti** umjesto prepisati, otvara se u načinu `ios::app` (*append*):

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
    { ofstream f("dnevnik.txt"); f << "prvi red\n"; }             // novi sadržaj
    { ofstream f("dnevnik.txt", ios::app); f << "drugi red\n"; }  // dodaj na kraj

    ifstream f("dnevnik.txt");
    string red;
    int broj = 0;
    while (getline(f, red)) cout << ++broj << ": " << red << "\n";
}
```

IZLAZ PROGRAMA

```
1: prvi red
2: drugi red
```

22.3 Primjer: CSV datoteka u vektor struktura

CSV (*comma-separated values*) je najčešći jednostavni format za tablične podatke — izvoz iz Excela, baza, sustava. Svaki red je jedan zapis, a polja su odvojena zarezom (ili točkom-zarezom). Kombiniramo sve što znamo: `getline` za redove, `stringstream` + `getline` s graničnikom za polja, `stoi`/`stod` za pretvorbu, i `try-catch` za pokvarene redove.

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

struct Artikl {
    string naziv;
```

```
    int kolicina;
    double cijena;
};

vector<Artikl> ucitajCSV(const string& putanja) {
    ifstream f(putanja);
    if (!f) throw runtime_error("ne mogu otvoriti " + putanja);
    vector<Artikl> artikli;
    string red;
    getline(f, red);                                    // preskoči zaglavlje
    int brojReda = 1;
    while (getline(f, red)) {
        brojReda++;
        if (red.empty()) continue;
        stringstream ss(red);
        string naziv, kol, cij;
        getline(ss, naziv, ',');
        getline(ss, kol, ',');
        getline(ss, cij, ',');
        try {
            artikli.push_back({naziv, stoi(kol), stod(cij)});
        } catch (const exception&) {
            cerr << "Preskacem neispravan red " << brojReda << "\n";
        }
    }
    return artikli;
}

int main() {
    {                                                   // pripremi primjer datoteke
        ofstream p("skladiste.csv");
        p << "naziv,kolicina,cijena\n" << "olovka,120,0.45\n" << "biljeznica,40,2.10\n"
          << "gumica,x,0.30\n" << "ravnalo,15,1.25\n";
    }
    try {
        vector<Artikl> a = ucitajCSV("skladiste.csv");
        double ukupno = 0;
        cout << fixed << setprecision(2);
        for (const auto& x : a) {
            cout << left << setw(12) << x.naziv << right << setw(5) << x.kolicina
                 << setw(8) << x.kolicina * x.cijena << "\n";
            ukupno += x.kolicina * x.cijena;
        }
        cout << "Ukupna vrijednost: " << ukupno << "\n";
    } catch (const exception& e) {
        cerr << "Greska: " << e.what() << "\n";
    }
}
```

IZLAZ PROGRAMA

```
Preskacem neispravan red 4
olovka        120   54.00
biljeznica     40   84.00
ravnalo        15   18.75
Ukupna vrijednost: 156.75
```

Ovaj primjer vrijedi proučiti detaljno: to je obrazac koji ćeš koristiti u gotovo svakom projektu i na mnogim ispitima (učitaj datoteku → obradi → ispiši ili spremi rezultat).

BINARNE DATOTEKE

Sve do sada bile su **tekstualne** datoteke — možeš ih otvoriti u Notepadu. Postoje i **binarne** datoteke, u koje se zapisuju sirovi bajtovi iz memorije (`ofstream f("x.bin", ios::binary); f.write(...)`). Brže su i manje, ali nisu čitljive ljudima i ovise o platformi. Za učenje i većinu zadataka — koristi tekst.

UKRATKO

- `ofstream` piše, `ifstream` čita — koriste se kao `cout` i `cin`. Zaglavlje `<fstream>`.
- Uvijek provjeri je li datoteka otvorena. Datoteka se zatvara sama (RAII).
- Petlje čitanja: `while (f >> x)` i `while (getline(f, red))` — nikad `while (!f.eof())`.
- `ios::app` dodaje na kraj datoteke umjesto da je prepiše.
- CSV: `getline` za redove, `stringstream` + `getline(ss, polje, ',')` za polja, `stoi`/`stod` + `try-catch`.

Zadaci

Zadatak 22.1 ★☆☆

Napiši program koji za zadanu tekstualnu datoteku ispisuje broj redova, broj riječi i broj znakova (kao naredba `wc` na Linuxu).

Zadatak 22.2 ★★☆

Datoteka `brojevi.txt` sadrži cijele brojeve razdvojene razmacima i novim redovima. Upiši sve parne brojeve u `parni.txt`, a neparne u `neparni.txt`, svaki u svoj red, sortirane uzlazno.

Zadatak 22.3 ★★☆

Proširi `Racun` iz poglavlja 12 metodama `spremi(ostream&)` i statičkom `ucitaj(istream&)`. Napiši program koji na početku učitava sve račune iz `racuni.txt`, omogućuje uplate i isplate kroz izbornik, i na kraju ih sprema natrag.

Zadatak 22.4 ★★★

Zadana je CSV datoteka s rezultatima ispita: `jmbag;ime;prezime;z1;z2;z3;z4` (bodovi po zadacima). Učitaj je, izračunaj ukupne bodove i ocjenu (skala iz poglavlja 4), i zapiši novu datoteku `rezultati.csv` sortiranu po ukupnim bodovima silazno, s dodanim stupcima `ukupno;ocjena`. Neispravne redove zapiši u `greske.txt` s brojem reda.

23. Moderni C++, dobre prakse i debugiranje

Za kraj teorijskog dijela skupit ćemo na jedno mjesto ono što razlikuje moderan C++ od „C-a s klasama“ iz starih udžbenika, proći najčešće greške i naučiti kako ih pronaći. Ovo poglavlje možeš koristiti i kao podsjetnik prije ispita ili prije prvog posla.

23.1 Stari stil → moderni stil

Umjesto (stari stil)	Piši (moderni C++)	Zašto
`NULL` ili `0` za pokazivače	`nullptr`	ima vlastiti tip, nema zabune s brojem 0
`#define MAX 100`	`constexpr int MAX = 100;`	ima tip i doseg, vidljiv u debuggeru
`(int)x`	`static_cast<int>(x)`	eksplicitno, lako se pronađe
`typedef vector<int> Vi;`	`using Vi = vector<int>;`	čitljivije, radi i s predlošcima
`enum Boja {...}`	`enum class Boja {...}`	bez sudara imena i tihih pretvorbi
`char ime[50]`, `char*`	`std::string`, `std::string_view`	nema prepunjenja, zna duljinu
`int a[n]`, `new int[n]`	`std::vector<int>`, `std::array`	zna veličinu, sam oslobađa memoriju
`new T` / `delete p`	`make_unique<T>()`	nema curenja ni dvostrukog brisanja
`for (int i = 0; i < v.size(); i++)`	`for (const auto& x : v)`	nema grešaka s indeksima
ručne petlje za traženje, brojanje…	`find_if`, `count_if`, `any_of` …	ime opisuje namjeru
izlazni parametri (`int& rez`)	vraćanje strukture, `pair`, `optional`	jasniji potpis funkcije
`Tip x = Tip(...)` s dugim imenom tipa	`auto x = ...`	manje ponavljanja

23.2 Nekoliko modernih alata na jednom mjestu

```cpp
#include <iostream>
#include <map>
#include <optional>
#include <string>
#include <string_view>
#include <vector>
using namespace std;

// optional: "možda postoji rezultat" — bez magičnih vrijednosti poput -1
optional<int> pronadjiIndeks(const vector<string>& v, string_view trazeno) {
    for (size_t i = 0; i < v.size(); i++)
        if (v[i] == trazeno) return i;
    return nullopt;                          // "nema rezultata"
}
```

```cpp
// string_view: pogled na tekst bez kopiranja (prima string, literal, dio stringa)
int brojSamoglasnika(string_view s) {
    int n = 0;
    for (char c : s) if (string_view("aeiouAEIOU").find(c) != string_view::npos) n++;
    return n;
}

// constexpr funkcija: može se izračunati već pri prevođenju
constexpr long long faktorijel(int n) { return n <= 1 ? 1 : n * faktorijel(n - 1); }

// [[nodiscard]]: prevoditelj upozorava ako se rezultat zanemari
[[nodiscard]] pair<int, int> minMax(const vector<int>& v) {
    int mn = v[0], mx = v[0];
    for (int x : v) { mn = min(mn, x); mx = max(mx, x); }
    return {mn, mx};
}

int main() {
    vector<string> imena = {"Ana", "Ivo", "Eva"};
    if (auto idx = pronadjiIndeks(imena, "Ivo"))          // optional u uvjetu
        cout << "Ivo na indeksu " << *idx << "\n";
    if (!pronadjiIndeks(imena, "Marko")) cout << "Marka nema\n";

    cout << brojSamoglasnika("Programiranje") << "\n";

    static_assert(faktorijel(5) == 120);                  // provjera pri prevođenju!

    auto [mn, mx] = minMax({4, -2, 9, 7});                // strukturirano vezanje
    cout << mn << " " << mx << "\n";

    map<string, int> bodovi = {{"Ana", 87}, {"Ivo", 64}};
    for (const auto& [ime, b] : bodovi) cout << ime << "=" << b << " ";
    cout << "\n";

    if (auto it = bodovi.find("Ana"); it != bodovi.end())  // if s inicijalizacijom (C++17)
        cout << "Ana ima " << it->second << " bodova\n";
}
```

IZLAZ PROGRAMA

```
Ivo na indeksu 1
Marka nema
5
-2 9
Ana=87 Ivo=64
Ana ima 87 bodova
```

- `std::optional<T>` — vrijednost koja možda postoji. Zamjenjuje „vrati −1 ako nema“ i slične dogovore koje je lako zaboraviti.
- `std::string_view` — lagani „pogled“ na tuđi tekst (pokazivač + duljina), bez kopiranja. Idealan za parametre funkcija koje tekst samo čitaju. Pazi: ne posjeduje tekst, pa ne smije nadživjeti string na koji gleda.
- `constexpr` funkcije mogu se izvršiti pri prevođenju; `static_assert` provjerava uvjet pri prevođenju.
- **Strukturirano vezanje** `auto [a, b] = ...` raspakira `pair`, `tuple`, strukturu ili element mape.

23.3 Česte greške — kontrolni popis

Prije predaje zadatka (ili kad program radi čudno) prođi ovaj popis:

1. **Neinicijalizirane varijable** — akumulatori, pokazivači, članovi klase bez vrijednosti.
2. **Cjelobrojno dijeljenje** — `zbroj / n` gdje su oba `int`.

3. **Izlazak izvan granica** — `<=` umjesto `<`, `v[v.size()]`, prazan vektor i `v[0]`, `v.size() - 1` na praznom vektoru.
4. **`=` umjesto `==`**, `;` iza `if`/`for`/`while`, izostavljene vitičaste zagrade.
5. **Preljev** — umnošci i faktorijeli u `int`-u; miješanje `int` i `size_t`.
6. **Usporedba `double` s `==`**.
7. **`cin >>` pa `getline`** — zaostali `'\n'`.
8. **Viseći pokazivači i reference** — na lokalne varijable, na elemente vektora nakon `push_back`, na obrisanu memoriju.
9. **Kopija umjesto reference** — `for (auto x : v)` kad želiš mijenjati elemente; parametar `vector<T>` umjesto `const vector<T>&`; `auto x = v[i]` pa mijenjanje `x`.
10. **Klase**: nedostaje `const` na metodama za čitanje; nedostaje virtualni destruktor u polimorfnoj baznoj klasi; ručni resurs bez pravila tri/pet; zaboravljen `;` iza klase.
11. **`map[k]` za provjeru postojanja** — tiho dodaje ključ.
12. **`while (!f.eof())`** pri čitanju datoteke.

23.4 Kako pronaći grešku

1. Uključi upozorenja i čitaj ih. `g++ -std=c++20 -Wall -Wextra -pedantic`. U Visual Studiju: *Warning Level 4* (`/W4`). Upozorenje prevoditelja je besplatan pregled koda — velik dio grešaka s popisa gore prevoditelj prepoznaje.

2. Sanitizatori. Prevedi s `-fsanitize=address,undefined -g` (GCC/Clang). Program će se zaustaviti *točno* na mjestu izlaska izvan granica, korištenja obrisane memorije ili preljeva, s porukom koja kaže što se dogodilo i gdje. To je najmoćniji alat za C++ greške s memorijom.

3. Debugger. Svaki IDE ima debugger (Visual Studio, VS Code, CLion; u terminalu `gdb`). Osnovne radnje:

Radnja	Što radi
breakpoint (točka prekida)	zaustavi program na toj liniji
step over (F10)	izvrši trenutnu liniju, ne ulazeći u funkcije
step into (F11)	uđi u funkciju koja se poziva na liniji
step out	izvrši ostatak funkcije i vrati se pozivatelju
watch / *locals*	prati vrijednosti varijabli
call stack	pogledaj lanac poziva koji je doveo do ove točke

4. Smanji problem. Ako program od 300 redova ne radi, napravi kopiju i briši dijelove dok greška ne nestane — zadnji obrisani dio je sumnjiv. Često se u tom procesu greška sama otkrije.

5. Objasni kod naglas. Metoda „gumene patkice“: objašnjavaj liniju po liniju što kod radi — nekome, ili predmetu na stolu. Iznenađujuće često usred objašnjavanja shvatiš da kod *ne radi* ono što tvrdiš.

> **SAVJET PROFESORA: PIŠI MALO, TESTIRAJ ČESTO**
>
> Najveća razlika između početnika i iskusnih programera nije u znanju sintakse, nego u **ritmu rada**. Početnik napiše 200 redova, prevede, dobije 40 grešaka i ne zna odakle početi. Iskusni napiše 10 redova, prevede, isproba, pa sljedećih 10. Kad nešto pukne, zna da je problem u zadnjih 10 redova.

23.5 Stil i navike koje se isplate

- **Imena** opisuju namjeru: `brojStudenata`, `jePunoljetan`, `izracunajProsjek`. Funkcije su glagoli, klase imenice.
- **Male funkcije** koje rade jednu stvar. Ako funkciji trebaš dati ime s „i“ (`ucitajIIspisi`), to su dvije funkcije.
- **Const-ispravnost**: `const` svugdje gdje se nešto ne mijenja — varijable, parametri po referenci, metode.
- **Najuži mogući doseg**: varijablu deklariraj što bliže mjestu prve upotrebe, i odmah je inicijaliziraj.
- **Bez globalnih varijabli** (osim konstanti).
- **Vrijednosti umjesto pokazivača**: objekt na stogu > `unique_ptr` > `shared_ptr` > sirovi `new`.
- **Standardna biblioteka prije vlastitog koda**: `vector`, `string`, `map`, algoritmi.
- **Dosljedno oblikovanje**: uvlačenje, razmaci, vitičaste zagrade. Alati poput `clang-format` to rade automatski.

23.6 Kamo dalje

- **cppreference.com** — najbolja referenca za jezik i standardnu biblioteku; svaka funkcija s primjerom.
- **learncpp.com** — besplatan, vrlo temeljit tečaj za samostalno učenje.
- **Bjarne Stroustrup: *A Tour of C++*** — kratak pregled modernog C++-a od autora jezika.
- **Scott Meyers: *Effective Modern C++*** — 42 savjeta za C++11/14; obavezno štivo nakon osnova.
- **C++ Core Guidelines** (isocpp.github.io) — službene smjernice za dobar moderni C++.
- **Compiler Explorer** (godbolt.org) — isprobaj kod u pregledniku, na više prevoditelja.

UKRATKO

- Moderni C++: `nullptr`, `constexpr`, `enum class`, `string`, `vector`, `make_unique`, range-for, algoritmi, `auto`, `optional`, strukturirano vezanje.
- Nauči kontrolni popis čestih grešaka — većina bugova je na njemu.
- Alati: upozorenja (`-Wall -Wextra`), sanitizatori (`-fsanitize=address,undefined`), debugger.
- Piši malo, prevodi i testiraj često.

DIO V

Vježba

Završni projekti koji povezuju sve naučeno, tipična ispitna pitanja s odgovorima i rješenja svih zadataka.

24. Završni projekti

Zadaci na kraju poglavlja vježbaju pojedine koncepte. Ali pravo razumijevanje dolazi tek kad ih moraš **spojiti** u jedan program: podijeliti problem na klase, odlučiti tko je vlasnik čega, obraditi greške, spremiti podatke. Sljedeći projekti su veći — za svaki predvidi nekoliko večeri. Nemaju jedno „točno“ rješenje; zato uz svaki dajem prijedlog strukture i popis zahtjeva po kojem možeš sam provjeriti jesi li gotov.

> **KAKO PRISTUPITI VEĆEM PROJEKTU**
>
> 1) **Prvo papir**: nacrtaj klase, njihove atribute i odnose („ima“, „je“). 2) **Najmanja verzija koja radi**: npr. samo dodavanje i ispis, bez datoteka i iznimaka. 3) **Dodaj jednu po jednu mogućnost**, i nakon svake prevedi i testiraj. 4) Tek na kraju — dotjerivanje ispisa. Nikad ne piši cijeli projekt odjednom.

Projekt 1: Knjižnica ★★☆

Cilj: program za vođenje knjižnice — knjige, članovi i posudbe.

Zahtjevi:

- klasa `Knjiga` (ISBN, naslov, autor, godina, je li posuđena) i klasa `Clan` (broj iskaznice, ime, popis posuđenih ISBN-ova, najviše 3 knjige);
- klasa `Knjiznica` koja posjeduje sve knjige i članove (`vector` ili `map` s ključem ISBN / broj iskaznice);
- izbornik: dodaj knjigu, dodaj člana, posudi, vrati, pretraži po autoru ili dijelu naslova, ispiši sve slobodne knjige, ispiši posudbe člana;
- neispravne operacije (nepostojeća knjiga, već posuđena knjiga, član s 3 knjige) javljaju se **iznimkama** koje izbornik hvata i ispisuje;
- podaci se pri izlasku spremaju u `knjige.csv` i `clanovi.csv`, a pri pokretanju učitavaju;
- sve metode koje ne mijenjaju objekt su `const`; nema ni jednog `new` / `delete`.

Prijedlog strukture:

```
class Knjiga {
    string isbn, naslov, autor;
    int godina;
    bool posudena = false;
public:
    Knjiga(string isbn, string naslov, string autor, int godina);
    const string& getIsbn() const;
    bool jePosudena() const;
    void posudi();                // baca logic_error ako je već posuđena
    void vrati();
    friend ostream& operator<<(ostream& os, const Knjiga& k);
};

class Knjiznica {
    map<string, Knjiga> knjige;    // ISBN -> knjiga
    map<int, Clan> clanovi;        // broj iskaznice -> član
public:
    void dodajKnjigu(const Knjiga& k);
    void posudi(int brojIskaznice, const string& isbn);
    vector<const Knjiga*> pretraziAutora(const string& dioImena) const;
    void spremi(const string& mapa) const;
    void ucitaj(const string& mapa);
};
```

Proširenja: datum posudbe i zakasnina; sortiranje rezultata pretrage (lambda); statistika — najposuđivanija knjiga.

Projekt 2: Bankovni sustav s polimorfizmom ★★★

Cilj: simulacija banke s različitim vrstama računa.

Zahtjevi:

- apstraktna klasa `Racun` (broj, vlasnik, stanje, povijest transakcija) s čistim virtualnim metodama `bool mozeIsplatiti(double) const` i `string vrsta() const`, te virtualnom metodom `void krajMjeseca()`;
- izvedene klase: `TekuciRacun` (dopušteni minus, mjesečna naknada), `StedniRacun` (kamata na kraju mjeseca, najviše 2 isplate mjesečno), `StudentskiRacun` (bez naknade, ograničen dnevni iznos isplate);
- `Banka` posjeduje račune kao `vector<unique_ptr<Racun>>` i nudi uplatu, isplatu, prijenos između računa i obradu kraja mjeseca **za sve račune jednom petljom**;
- svaka transakcija zapisuje se u povijest (struktura `Transakcija`: vrsta, iznos, stanje nakon);
- hijerarhija vlastitih iznimaka (`BankaGreska` → `NedovoljnoSredstava`, `NepostojeciRacun`, `PrekoracenLimit`);
- prijenos je „sve ili ništa“: ako isplata s izvora uspije, a uplata ne, stanje se vraća.

Provjera razumijevanja: koji destruktor mora biti virtualan i zašto? Gdje bi `dynamic_cast` bio znak lošeg dizajna? Kako dodati novu vrstu računa bez mijenjanja klase `Banka`?

Projekt 3: Analiza teksta ★★☆

Cilj: program koji čita tekstualnu datoteku (npr. poglavlje knjige) i ispisuje statistiku.

Zahtjevi:

- broj redova, riječi, rečenica i znakova; prosječna duljina riječi i rečenice;
- 20 najčešćih riječi (bez razlike velikih i malih slova, bez interpunkcije) s brojem pojavljivanja;
- najdulja riječ, sve riječi-palindromi duže od 3 slova, broj različitih riječi;
- histogram duljina riječi ispisan zvjezdicama;
- rezultat se ispisuje na ekran i sprema u `izvjestaj.txt`;
- koristi `map`/`unordered_map`, `set`, algoritme i lambde — **što manje ručnih petlji**.

Prijedlog: funkcija `string ocisti(string rijec)` (mala slova, bez interpunkcije), klasa `Statistika` s metodama `obradiRed(const string&)` i `ispisi(ostream&) const` — tako isti kod piše i na ekran (`cout`) i u datoteku (`ofstream`), jer oba nasljeđuju `ostream`.

Projekt 4: Igra života ★★☆

Cilj: simulacija Conwayeve „Igre života“ — stanični automat na mreži ćelija koje su žive ili mrtve.

Pravila (za svaku ćeliju, prema broju živih od 8 susjeda): živa ćelija s 2 ili 3 živa susjeda preživljava, inače umire; mrtva ćelija s točno 3 živa susjeda oživljava.

Zahtjevi:

- klasa `Mreza` s `vector<vector<bool>>` (ili jednim vektorom veličine `r*s`), metodama `postavi`, `brojZivihSusjeda(i, j)`, `korak()` i `ispisi()` (`#` za živu, `.` za mrtvu);
- `korak()` mora računati **novu** mrežu iz **stare** — zašto se ne smije mijenjati na mjestu?
- rubovi: odaberi i objasni — ćelije izvan mreže su mrtve, ili se mreža „omata“ (torus, `% r`);
- početno stanje učitava se iz datoteke; program ispisuje prvih N generacija;

- dodatno: prepoznaj kad je mreža stabilna ili se ponavlja (usporedi s prethodnim generacijama pomoću `set` stanja).

Isprobaj s poznatim uzorcima: „jedrilica“ (*glider*) putuje po mreži, „treptaljka“ (*blinker*) titra između dva stanja.

25. Tipična ispitna pitanja

Ovdje su skupljena pitanja koja se u nekom obliku pojavljuju na gotovo svakom kolokviju i ispitu iz C++-a. Pokušaj na svako odgovoriti **naglas, vlastitim riječima**, prije nego što pročitaš odgovor. Ako ne možeš objasniti u dvije-tri rečenice — vrati se na poglavlje.

25.1 Teorijska pitanja

1. Koja je razlika između deklaracije i definicije? Deklaracija uvodi ime i tip („ovo postoji“), npr. prototip funkcije `int f(int);`. Definicija daje sadržaj — tijelo funkcije ili rezervira memoriju za varijablu. Deklaracija smije biti više, definicija samo jedna (pravilo jedne definicije).

2. Što rade preprocesor, prevoditelj i povezivač? Preprocesor obrađuje `#` naredbe (kopira `#include` datoteke, zamjenjuje `#define`). Prevoditelj provjerava i prevodi svaku `.cpp` datoteku u objektni kod. Povezivač spaja objektne datoteke i biblioteke u izvršnu datoteku i razrješava pozive funkcija iz drugih datoteka.

3. Prijenos po vrijednosti, po referenci i po konstantnoj referenci? Po vrijednosti funkcija dobiva kopiju — promjene se ne vide izvana. Po referenci dobiva sam objekt — promjene se vide. Po `const` referenci dobiva objekt bez kopiranja, ali ga ne smije mijenjati; koristi se za velike objekte koji se samo čitaju.

4. Koja je razlika između pokazivača i reference? Pokazivač je varijabla koja sadrži adresu; može biti `nullptr`, može se preusmjeriti i podržava aritmetiku. Referenca je drugo ime za postojeći objekt; mora se inicijalizirati, ne može biti prazna i ne može se preusmjeriti.

5. Što je viseći pokazivač? Navedi dva načina kako nastaje. Pokazivač na memoriju koja više ne pripada programu. Nastaje npr. vraćanjem adrese lokalne varijable iz funkcije, korištenjem pokazivača nakon `delete`, ili zadržavanjem pokazivača na element vektora nakon što se vektor preselio (`push_back`).

6. Stog i gomila — razlike? Na stogu su lokalne varijable i parametri; memorijom upravlja prevoditelj automatski, brz je i malen, a životni vijek je do kraja bloka. Na gomili su objekti stvoreni s `new`; velika je, životni vijek određuje programer, a memorija se mora vratiti (`delete`) — ili, u modernom C++-u, to rade pametni pokazivači i kontejneri.

7. Što je curenje memorije i kako se sprječava? Dinamička memorija koja se nikad ne oslobodi. Sprječava se RAII-jem: resurse drže objekti (`vector`, `string`, `unique_ptr`) čiji destruktori ih automatski oslobađaju — i kod ranog `return`-a i kod iznimke.

8. Zašto `delete[]`, a ne `delete` za polja? `new[]` stvara više objekata i negdje pamti njihov broj. `delete[]` poziva destruktor za svaki element i oslobađa cijeli blok; `delete` očekuje jedan objekt. Miješanje oblika je nedefinirano ponašanje.

9. Što je enkapsulacija? Čemu služe `private`, `protected` i `public`? Skrivanje unutarnjeg stanja objekta i dopuštanje pristupa samo kroz kontrolirano sučelje, tako da objekt sam čuva svoju ispravnost. `private` — samo klasa; `protected` — klasa i izvedene klase; `public` — svi.

10. Kada prevoditelj generira zadani konstruktor? Samo ako klasa nema nijedan korisnički napisan konstruktor. Čim napišemo bilo koji konstruktor, automatski zadani nestaje (može se vratiti s `= default`).

11. Zašto koristiti inicijalizacijsku listu konstruktora? Lista inicijalizira članove odmah pravom vrijednošću, umjesto da ih prvo stvori pa prepisuje. Obavezna je za `const` članove, reference i članove bez zadanog konstruktora, kao i za poziv konstruktora bazne klase.

12. Kada se poziva destruktor i kojim redoslijedom? Kad objekt prestaje postojati: izlazak iz dosega (lokalni objekti), `delete` (dinamički), kraj programa (globalni i statički), uništenje objekta koji ga sadrži. Lokalni objekti uništavaju se obrnutim redom od stvaranja; kod nasljeđivanja prvo izvedeni, pa bazni dio.

13. Što znači `const` iza metode? Metoda obećava da neće mijenjati objekt (u njoj je `this` pokazivač na `const`). Samo takve metode smiju se pozivati na konstantnim objektima i preko `const&` referenci.

14. Što je `static` član klase? Član koji pripada klasi, a ne pojedinom objektu — postoji u jednoj kopiji koju dijele svi objekti. Statička metoda nema `this` i može pristupati samo statičkim članovima.

15. Plitka i duboka kopija? Što je pravilo tri? Plitka kopija kopira vrijednosti članova, pa za pokazivač kopira adresu — dva objekta dijele iste podatke (i oba ih brišu). Duboka kopija alocira novu memoriju i kopira sadržaj. Pravilo tri: ako klasa treba vlastiti destruktor, konstruktor kopiranja ili operator pridruživanja, treba sva tri (pravilo pet dodaje i dva premještanja).

16. Kada se poziva konstruktor kopiranja, a kada operator pridruživanja? Konstruktor kopiranja kad se **novi** objekt stvara iz postojećeg: `T b = a;`, `T b(a);`, prijenos po vrijednosti u funkciju, vraćanje po vrijednosti (ako se ne izostavi). Operator pridruživanja kad objekt s lijeve strane **već postoji**: `b = a;`.

17. Što je semantika premještanja i što radi `std::move`? Premještanje „krade“ resurse privremenog objekta umjesto skupog kopiranja (npr. preuzme pokazivač na polje). `std::move` sam ništa ne premješta — samo pretvara objekt u rvalue i time dopušta da se odabere premještanje.

18. Što je RAII? *Resource Acquisition Is Initialization*: resurs se uzima u konstruktoru objekta, a vraća u destruktoru. Budući da je poziv destruktora zajamčen (i pri iznimkama), resurs se ne može zaboraviti osloboditi.

19. Koja je razlika između preopterećenja (overloading) i nadjačavanja (overriding)? Preopterećenje: više funkcija istog imena s različitim parametrima, u istom dosegu; bira se pri prevođenju. Nadjačavanje: izvedena klasa daje novu implementaciju **virtualne** metode bazne klase s istim potpisom; bira se tijekom izvođenja.

20. Što je virtualna funkcija i kako radi dinamičko vezanje? Funkcija čija se verzija bira prema stvarnom tipu objekta, a ne prema tipu pokazivača/reference. Svaka klasa s virtualnim funkcijama ima tablicu virtualnih funkcija (vtable), a svaki objekt skriveni pokazivač (vptr) na tablicu svoje klase; poziv ide preko te tablice.

21. Što je apstraktna klasa? Klasa s barem jednom čistom virtualnom funkcijom (`= 0`). Ne može se instancirati; služi kao sučelje koje izvedene klase moraju implementirati.

22. Zašto polimorfna bazna klasa mora imati virtualni destruktor? Da bi `delete` preko pokazivača na baznu klasu pozvao destruktor stvarnog (izvedenog) objekta. Bez toga se poziva samo bazni destruktor — nedefinirano ponašanje, u praksi curenje resursa izvedenog dijela.

23. Što je rezanje objekta (slicing)? Kad se objekt izvedene klase kopira u objekt bazne klase (npr. prijenos po vrijednosti), kopira se samo bazni dio; izvedeni dio i polimorfno ponašanje se gube. Izbjegava se prijenosom po referenci ili pokazivaču.

24. Što je predložak i zašto se piše u zaglavlju? Recept po kojem prevoditelj generira funkcije ili klase za konkretne tipove (instanciranje pri prevođenju). Prevoditelj mora vidjeti cijelu definiciju na mjestu upotrebe, pa ona mora biti u zaglavlju koje se uključuje.

25. `vector` ili `list`? `map` ili `unordered_map`? `vector`: neprekinuta memorija, O(1) pristup indeksom, brz prolazak — zadani izbor. `list`: O(1) umetanje/brisanje na poznatoj poziciji, ali bez indeksa i sporiji prolazak. `map`: sortirano stablo, O(log n); `unordered_map`: hash tablica, prosječno O(1), bez redoslijeda.

26. Što vraća `end()` i smije li se dereferencirati? Iterator na poziciju **iza** zadnjeg elementa; služi kao oznaka kraja raspona `[begin, end)`. Ne smije se dereferencirati. Algoritmi poput `find` vraćaju `end()` kad ništa ne nađu.

27. `unique_ptr`, `shared_ptr` i `weak_ptr`? `unique_ptr` — jedini vlasnik, ne kopira se, može se premjestiti, bez dodatne cijene. `shared_ptr` — zajedničko vlasništvo s brojanjem referenci, objekt se briše kad nestane zadnji vlasnik. `weak_ptr` — promatra objekt u vlasništvu `shared_ptr`-a bez povećavanja brojača; služi za prekidanje ciklusa.

28. Zašto iznimke hvatamo po `const` referenci? Što je odmatanje stoga? Da se izbjegne kopiranje i rezanje (zadržava se stvarni tip i `what()` izvedene klase). Odmatanje stoga je izlazak iz funkcija nakon `throw`, pri čemu se za sve lokalne objekte pozivaju destruktori.

29. Što je nedefinirano ponašanje? Navedi primjere. Situacija za koju standard ne propisuje što se događa — program može raditi, srušiti se ili se ponašati nasumično. Primjeri: izlazak izvan granica polja, dereferenciranje `nullptr`-a ili visećeg pokazivača, čitanje neinicijalizirane varijable, preljev predznačenog cijelog broja, dvostruki `delete`, `delete` bez virtualnog destruktora.

30. Zašto je `vector::size() - 1` opasan izraz? `size()` vraća nepredznačen tip (`size_t`). Za prazan vektor `0 - 1` se „namota“ na najveći mogući broj, pa petlja s uvjetom `i < v.size() - 1` prolazi daleko izvan vektora.

25.2 Što ispisuje program?

Za svaki isječak odredi točan ispis. Pretpostavi da su uključena potrebna zaglavlja i `using namespace std;`. Odgovori su na kraju odjeljka.

Z1.

```
int a = 7, b = 2;
double c = a / b;
double d = static_cast<double>(a) / b;
cout << c << " " << d << " " << a % b << "\n";
int e = a++ + ++b;
cout << a << " " << b << " " << e << "\n";
```

Z2.

```
int s = 0;
for (int i = 1; i <= 10; i += 3) {
    if (i % 2 == 0) continue;
    s += i;
}
cout << s << "\n";
```

Z3.

```
void f(int x, int& y) { x *= 2; y *= 2; }
int g(int& x) { return ++x; }

int main() {
    int a = 3, b = 4;
    f(a, b);
    int c = g(a);
    c += g(a);
    cout << a << " " << b << " " << c << "\n";
}
```

Z4.

```
int arr[] = {1, 2, 3, 4, 5};
int* p = arr + 2;
*p += 10;
p--;
*(p + 3) = *p * 2;
cout << arr[1] << " " << arr[2] << " " << arr[4] << " " << p - arr << "\n";
```

Z5.

```
void r(int n) {
    if (n == 0) return;
    cout << n << " ";
    r(n - 1);
    cout << n << " ";
}
// u main-u: r(3);
```

Z6.

```
struct T {
    int id;
    T(int i) : id(i) { cout << "K" << id << " "; }
    ~T() { cout << "D" << id << " "; }
};
void f() { T a(2); static T s(3); }

int main() {
    T x(4);
    f();
    f();
    { T y(5); }
    cout << "| ";
}
```

Z7.

```
struct A {
    A() { cout << "A "; }
    virtual ~A() { cout << "~A "; }
    virtual void f() { cout << "A::f "; }
    void g() { cout << "A::g "; f(); }
};
struct B : A {
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
    void f() override { cout << "B::f "; }
    void g() { cout << "B::g "; }
};
int main() {
    A* p = new B;
    p->f();
    p->g();
    delete p;
}
```

Z8.

```
struct X {
    X() { cout << "d "; }
    X(const X&) { cout << "c "; }
    X& operator=(const X&) { cout << "= "; return *this; }
};
void f(X x) {}
void g(const X& x) {}

int main() {
    X a;
    X b = a;
    X c;
    c = a;
    f(a);
    g(a);
    X arr[2];
}
```

Z9.

```
vector<int> v = {5, 2, 8, 2, 9};
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
map<int, int> m;
for (int x : v) m[x % 3]++;
cout << v.size() << " " << m[2] << " " << m.size() << "\n";
```

Z10.

```
int f(int x) {
    if (x < 0) throw invalid_argument("neg");
    if (x == 0) throw 0;
    return 10 / x;
}
int main() {
    for (int x : {5, 0, -1}) {
        try { cout << f(x) << " "; }
        catch (const exception& e) { cout << e.what() << " "; }
        catch (int) { cout << "nula "; }
    }
}
```

Z11.

```
string s = "programiranje";
cout << s.substr(3, 4) << " " << s.find('r') << " " << s.rfind('r') << " " << s.size() << "\n";
```

Odgovori

	Ispis	Ključ
Z1	`3 3.5 1` i `8 3 10`	`7/2` je cjelobrojno (3), tek onda u `double`. `a++` daje 7, `++b` daje 3.
Z2	`8`	`i` = 1, 4, 7, 10; parni (4, 10) se preskaču: 1 + 7.
Z3	`5 8 9`	`x` je kopija, `y` referenca (b = 8). `g` povećava `a`: 4, pa 5; c = 4 + 5.
Z4	`2 13 4 1`	`arr[2]` = 13; `p` → `arr[1]`; `arr[4]` = 2 · 2; `p - arr` = 1.
Z5	`3 2 1 1 2 3`	Ispis prije poziva ide „prema dolje“, ispis nakon poziva pri povratku.
Z6	`K4 K2 K3 D2 K2 D2 K5 D5 | D4 D3`	`static` objekt se stvara samo pri prvom pozivu i uništava tek na kraju programa.
Z7	`A B B::f A::g B::f ~B ~A`	`g` nije virtualna → `A::g`; unutar nje `f()` je virtualna → `B::f`. Destruktor je virtualan.
Z8	`d c d = c d d`	`X b = a` je konstruktor kopiranja; `c = a` pridruživanje; `f(a)` kopira; `g(a)` ne.
Z9	`4 3 2`	Nakon `unique`: {2, 5, 8, 9}. Ostaci pri dijeljenju s 3: 2, 2, 2, 0.
Z10	`2 nula neg`	`throw 0` baca `int`, koji ne nasljeđuje `exception`, pa ga hvata `catch (int)`.
Z11	`gram 1 8 13`	Indeksi od 0: „pro**gram**iranje“; prvi `r` na 1, zadnji na 8.

26. Rješenja zadataka

Prije nego što pogledaš rješenje, stvarno se potrudi sam. A kad ga pogledaš, zapamti: **ovo je jedno od mogućih rješenja**, ne jedino točno. Ako tvoje radi, prolazi rubne slučajeve i čitljivo je — dobro je. Usporedi pristupe i razmisli koji ti se više sviđa i zašto.

Radi kratkoće, u rješenjima su izostavljeni `#include` i `using namespace std;` (pretpostavi da su uključena sva potrebna zaglavlja), a kod većih zadataka prikazan je samo ključni dio.

Poglavlje 1 — Kako C++ radi

Rješenje 1.1

```
int main() {
    cout << "Ana Horvat\n";
    cout << "Fakultet organizacije i informatike\n";
    cout << "Omiljeni broj: " << 6 * 7 << "\n";
}
```

Rješenje 1.2

(1) `using namespace std` — nedostaje `;` → greška pri prevođenju. (2) `Main` umjesto `main` — zanimljiv slučaj: prevođenje *prolazi* (`Main` je sasvim legalno ime funkcije), ali **povezivač** javlja grešku jer ne postoji funkcija `main` (`undefined reference to main`). (3) `4 + 5` umjesto `4 * 5` — logička greška, program radi, ali ispisuje 9. (4) `return 0` — nedostaje `;` → greška pri prevođenju.

Rješenje 1.3

```
int main() {
    cout << "    *\n";
    cout << "   ***\n";
    cout << "  *****\n";
    cout << " *******\n";
    cout << "    |\n";
}
```

Rješenje 1.4

Preprocesor na mjesto linije `#include <iostream>` doslovno ubacuje sadržaj datoteke `iostream`, u kojoj su deklarirani `cout`, `cin`, operator `<<` i ostalo. Bez te linije prevoditelj nigdje nije vidio deklaraciju `cout`-a, pa ne zna što je to ime. Grešku javlja **prevoditelj** (`'cout' was not declared in this scope`).

Poglavlje 2 — Varijable, tipovi i operatori

Rješenje 2.1

```
int main() {
    int a = 3, b = 8;
    cout << "prije: " << a << " " << b << "\n";
    int t = a;  a = b;  b = t;
    cout << "poslije: " << a << " " << b << "\n";
}
```

Rješenje 2.2

Izraz	Vrijednost	Tip
`17 / 5`	3	`int`
`17 % 5`	2	`int`
`17 / 5.0`	3.4	`double`
`(double)(17 / 5)`	3 (tj. 3.0)	`double` — dijeljenje je obavljeno prije pretvorbe
`17 / 5 * 5 + 17 % 5`	17	`int` — $(3 \cdot 5) + 2$; uvijek vrijedi `(a/b)*b + a%b == a`
`'a' + 1`	98	`int`
`char('a' + 1)`	`'b'`	`char`
`10 / 4 * 2.0`	4 (tj. 4.0)	`double` — prvo `10/4 = 2`, pa `2 * 2.0`
`-17 % 5`	−2	`int`

Rješenje 2.3

```
int main() {
    int s;
    cin >> s;
    int sati = s / 3600;
    int minute = s % 3600 / 60;
    int sekunde = s % 60;
    cout << sati << " h " << minute << " min " << sekunde << " s\n";
}
```

Rješenje 2.4

```
int main() {
    int n;
    cin >> n;                          // npr. 472
    int s = n / 100, d = n / 10 % 10, j = n % 10;
    cout << s + d + j << " " << j * 100 + d * 10 + s << "\n";
}
```

Rješenje 2.5

```
int main() {
    int a, b, c;
    cin >> a >> b >> c;
    double prosjek = (a + b + c) / 3.0;          // 3.0 -> pravo dijeljenje
    cout << prosjek << " " << round(prosjek) << "\n";
}
```

Rješenje 2.6

```
int main() {
    int a = 3, b = 8;
    a = a + b;      // a = 11
    b = a - b;      // b = 3
    a = a - b;      // a = 8
    cout << a << " " << b << "\n";
}
```

Ako je `a + b` izvan raspona `int`-a (npr. obje vrijednosti oko 2 milijarde), prvi korak uzrokuje preljev predznačenog tipa — nedefinirano ponašanje. Zato se u praksi koristi pomoćna varijabla (ili `std::swap`).

Poglavlje 3 — Ulaz i izlaz

Rješenje 3.1

```
int main() {
    const int OVA_GODINA = 2026;
    string ime;
    int godinaRodjenja;
    cout << "Ime: ";            cin >> ime;
    cout << "Godina rodjenja: "; cin >> godinaRodjenja;
    cout << "Bok, " << ime << "! Ove godine punis " << OVA_GODINA - godinaRodjenja << ".\n";
}
```

Rješenje 3.2

```
int main() {
    const double PDV = 0.25;
    double cijena;
    cin >> cijena;
    cout << fixed << setprecision(2);
    cout << "PDV: " << cijena * PDV << "\nUkupno: " << cijena * (1 + PDV) << "\n";
}
```

Rješenje 3.3

```
int main() {
    int godine;
    string ime;
    cout << "Godine: ";
    cin >> godine;
    cout << "Ime i prezime: ";
    getline(cin >> ws, ime);        // ws preskače zaostali '\n'
    cout << "Ime: " << ime << "\nGodine: " << godine << "\n";
}
```

Rješenje 3.4

```
int main() {
    cout << setw(6) << "n" << setw(6) << "n^2" << setw(6) << "n^3" << "\n";
    for (int n = 1; n <= 5; n++)
        cout << setw(6) << n << setw(6) << n * n << setw(6) << n * n * n << "\n";
}
```

Rješenje 3.5

```
int main() {
    int x;
    cout << "Unesi broj 1-10: ";
    while (!(cin >> x) || x < 1 || x > 10) {
        cout << "Neispravno. Ponovno: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "Unio si " << x << "\n";
}
```

Kratko spajanje je ključno: ako unos nije uspio, `||` ne provjerava raspon (ne bi imalo smisla).

Poglavlje 4 — Grananje

Rješenje 4.1

```
int main() {
    int n;
    cin >> n;
    if (n > 0) cout << "pozitivan";
    else if (n < 0) cout << "negativan";
    else cout << "nula";
```

```
    cout << (n % 2 == 0 ? ", paran\n" : ", neparan\n");   // n % 2 != 0, ne == 1 (negativni!)
}
```

Rješenje 4.2

```
bool prijestupna(int g) {
    return (g % 4 == 0 && g % 100 != 0) || g % 400 == 0;
}
```

Rješenje 4.3

```
int main() {
    int a, b, c;
    cin >> a >> b >> c;
    int mx = a;
    if (b > mx) mx = b;
    if (c > mx) mx = c;
    cout << "Najveci: " << mx << "\n";
    // sortiranje s tri usporedbe i zamjene
    if (a > b) swap(a, b);
    if (b > c) swap(b, c);      // sad je c najveći
    if (a > b) swap(a, b);
    cout << a << " " << b << " " << c << "\n";
}
```

Rješenje 4.4

```
int main() {
    double a, b;
    char op;
    cin >> a >> op >> b;
    switch (op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/':
            if (b == 0) cout << "Dijeljenje s nulom!";
            else cout << a / b;
            break;
        default: cout << "Nepoznat operator '" << op << "'";
    }
    cout << "\n";
}
```

Rješenje 4.5

```
int main() {
    double a, b, c;
    cin >> a >> b >> c;
    if (a <= 0 || b <= 0 || c <= 0 || a + b <= c || a + c <= b || b + c <= a) {
        cout << "Nije trokut\n";
    } else if (a == b && b == c) {
        cout << "Jednakostranican\n";
    } else if (a == b || b == c || a == c) {
        cout << "Jednakokracan\n";
    } else {
        cout << "Raznostranican\n";
    }
}
```

Rješenje 4.6

```
int main() {
    double a, b, c;
    cin >> a >> b >> c;
    if (a == 0) {                                  // linearna: bx + c = 0
        if (b != 0) cout << "x = " << -c / b << "\n";
        else if (c == 0) cout << "Beskonacno rjesenja\n";
        else cout << "Nema rjesenja\n";
        return 0;
    }
```

```
    double D = b * b - 4 * a * c;
    if (D > 0) {
        cout << "x1 = " << (-b + sqrt(D)) / (2 * a)
             << ", x2 = " << (-b - sqrt(D)) / (2 * a) << "\n";
    } else if (D == 0) {
        cout << "x = " << -b / (2 * a) << "\n";
    } else {
        double re = -b / (2 * a), im = sqrt(-D) / (2 * a);
        cout << "x = " << re << " +- " << fabs(im) << "i\n";
    }
}
```

(Strogo gledano, `D == 0` za `double` treba zamijeniti s `fabs(D) < 1e-12` — poglavlje 2.9.)

Poglavlje 5 — Petlje

Rješenje 5.1

```
int main() {
    int n, zbrojParnih = 0, djeljiviS3 = 0;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0) zbrojParnih += i;
        if (i % 3 == 0) djeljiviS3++;
    }
    cout << zbrojParnih << " " << djeljiviS3 << "\n";
}
```

Rješenje 5.2

```
int main() {
    int n;
    cin >> n;
    long long f = 1;                 // 20! ≈ 2.4 · 10^18 stane u long long; 13! ne stane u int
    for (int i = 2; i <= n; i++) f *= i;
    cout << f << "\n";
}
```

Za `n = 21` rezultat premašuje raspon `long long`-a — preljev predznačenog tipa, nedefinirano ponašanje (u praksi besmislen, često negativan broj).

Rješenje 5.3

```
int main() {
    int n;
    cin >> n;
    int k = n, obrnuto = 0;
    while (k > 0) {
        obrnuto = obrnuto * 10 + k % 10;
        k /= 10;
    }
    cout << (obrnuto == n ? "palindrom" : "nije palindrom") << "\n";
}
```

Rješenje 5.4

```
int main() {
    for (int n = 2; n < 10000; n++) {
        int zbroj = 1;                          // 1 je djelitelj svakog n > 1
        for (int d = 2; d * d <= n; d++) {
            if (n % d == 0) {
                zbroj += d;
                if (d != n / d) zbroj += n / d;  // par djelitelja, ali ne dvaput korijen
            }
        }
        if (zbroj == n) cout << n << " ";
    }
    cout << "\n";
}
```

```
6 28 496 8128
```
IZLAZ PROGRAMA

Rješenje 5.5

```cpp
int main() {
    const int TAJNI = 37;
    int pokusaj, broj = 0;
    do {
        cout << "Pogodi: ";
        cin >> pokusaj;
        broj++;
        if (pokusaj < TAJNI) cout << "vise\n";
        else if (pokusaj > TAJNI) cout << "manje\n";
    } while (pokusaj != TAJNI);
    cout << "Pogodak u " << broj << " pokusaja!\n";
}
```

`do-while` je prirodan izbor jer se mora pogađati barem jednom. (Za pravi slučajni broj: `srand(time(0)); int TAJNI = rand() % 100 + 1;` ili, moderno, `<random>`.)

Rješenje 5.6

```cpp
int main() {
    int n = 5;                          // neparan
    int pola = n / 2;
    for (int i = 0; i < n; i++) {
        int udaljenost = abs(pola - i);          // 2,1,0,1,2
        int zvjezdica = n - 2 * udaljenost;      // 1,3,5,3,1
        cout << string(udaljenost, ' ') << string(zvjezdica, '*') << "\n";
    }
}
```

```
  *
 ***
*****
 ***
  *
```
IZLAZ PROGRAMA

Trik: umjesto dvije polovice romba, jedna petlja s `abs` — udaljenost od srednjeg reda određuje sve.

Rješenje 5.7

```cpp
int main() {
    int najbolji = 1, najdulji = 1;
    for (int n = 1; n < 10000; n++) {
        long long x = n;              // međurezultati prelaze raspon int-a!
        int duljina = 1;
        while (x != 1) {
            x = (x % 2 == 0) ? x / 2 : 3 * x + 1;
            duljina++;
        }
        if (duljina > najdulji) { najdulji = duljina; najbolji = n; }
    }
    cout << najbolji << " " << najdulji << "\n";
}
```

```
6171 262
```
IZLAZ PROGRAMA

(Niz za 6171 ima 262 člana, tj. 261 korak.)

Poglavlje 6 — Funkcije

Rješenje 6.1

```cpp
int zbrojZnamenki(int n) {
    int s = 0;
    for (n = abs(n); n > 0; n /= 10) s += n % 10;
    return s;
```

```
}
bool jeParan(int n) { return n % 2 == 0; }

int main() {
    for (int i = 1; i <= 100; i++)
        if (jeParan(zbrojZnamenki(i))) cout << i << " ";
    cout << "\n";
}
```

Rješenje 6.2

```
void udvostruci(int& x) { x *= 2; }
int udvostruceno(int x) { return x * 2; }

int main() {
    int a = 5;
    udvostruci(a);              // a postaje 10 — mijenja se argument
    int b = udvostruceno(a);    // a ostaje 10, rezultat se sprema u b (20)
    cout << a << " " << b << "\n";
}
```

Prva verzija mijenja postojeću varijablu i ne može se pozvati s konstantom (`udvostruci(5)` se ne prevodi). Druga ne dira argument i rezultat vraća — može se koristiti u izrazima. Općenito je druga verzija čišća.

Rješenje 6.3

```
int nzd(int a, int b) {
    a = abs(a); b = abs(b);
    while (b != 0) { int r = a % b; a = b; b = r; }
    return a;
}
int nzv(int a, int b) { return a / nzd(a, b) * b; }   // prvo dijeli — manji rizik preljeva

void skratiRazlomak(int& brojnik, int& nazivnik) {
    int d = nzd(brojnik, nazivnik);
    if (d != 0) { brojnik /= d; nazivnik /= d; }
}

int main() {
    int b = 84, n = 36;
    skratiRazlomak(b, n);
    cout << b << "/" << n << "\n";
}
```

IZLAZ PROGRAMA

```
7/3
```

Rješenje 6.4

```
int zbrojZnamenki(int n) { return n < 10 ? n : n % 10 + zbrojZnamenki(n / 10); }

long long potencija(long long x, int n) { return n == 0 ? 1 : x * potencija(x, n - 1); }

void ispisiUnatrag(int n) {
    cout << n % 10;
    if (n >= 10) ispisiUnatrag(n / 10);
}

int main() {
    cout << zbrojZnamenki(9041) << " " << potencija(2, 10) << " ";
    ispisiUnatrag(1234);
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
14 1024 4321
```

Rješenje 6.5

Ispis je `1 12 23`. `a` je kopija od `x`: postaje 11, ali `x` ostaje 1. `b` je referenca na `y`, pa `y` postaje 12. Funkcija vraća 11 + 12 = 23.

Rješenje 6.6

```
double brzaPotencija(double x, long long n) {
    if (n == 0) return 1;
    if (n % 2 == 0) {
        double pola = brzaPotencija(x, n / 2);   // izračunaj JEDNOM, pa kvadriraj
        return pola * pola;
    }
    return x * brzaPotencija(x, n - 1);
}
```

Svaka dva koraka najmanje prepolove `n`, pa je broj poziva najviše oko $2 \cdot \log_2(n)$ — za milijun oko 40, naspram milijun poziva naivne verzije. Pazi: `brzaPotencija(x, n/2) * brzaPotencija(x, n/2)` bi opet bilo sporo — dva rekurzivna poziva poništavaju uštedu.

Rješenje 6.7

```
void hanoi(int n, char s, char c, char p) {
    if (n == 0) return;
    hanoi(n - 1, s, p, c);                // n-1 diskova sa s na pomoćni
    cout << s << " -> " << c << "\n";     // najveći disk na cilj
    hanoi(n - 1, p, c, s);                // n-1 diskova s pomoćnog na cilj
}
```

Broj poteza zadovoljava `T(n) = 2 · T(n-1) + 1`, `T(1) = 1`, pa je `T(n) = 2ⁿ - 1`. Za 64 diska to je oko $1{,}8 \cdot 10^{19}$ poteza.

Poglavlje 7 — Polja i vektori

Rješenje 7.1

```
int main() {
    int n;
    cin >> n;
    vector<int> v(n);
    for (int& x : v) cin >> x;
    for (int i = n - 1; i >= 0; i--) cout << v[i] << " ";
    cout << "\n";
    double prosjek = 0;
    for (int x : v) prosjek += x;
    prosjek /= n;
    for (int x : v) if (x > prosjek) cout << x << " ";
    cout << "\n";
}
```

Rješenje 7.2

```
// vraća false ako drugi najveći ne postoji (manje od 2 različite vrijednosti)
bool drugiNajveci(const vector<int>& v, int& rezultat) {
    bool imaPrvi = false, imaDrugi = false;
    int prvi = 0, drugi = 0;
    for (int x : v) {
        if (!imaPrvi || x > prvi) {
            if (imaPrvi) { drugi = prvi; imaDrugi = true; }
            prvi = x; imaPrvi = true;
        } else if (x != prvi && (!imaDrugi || x > drugi)) {
            drugi = x; imaDrugi = true;
        }
    }
    rezultat = drugi;
    return imaDrugi;
}
```

Ako su svi elementi jednaki, drugi najveći (različit od najvećeg) ne postoji — funkcija to javlja povratnom vrijednošću. (Moderno: vratiti `optional<int>`, poglavlje 23.)

Rješenje 7.3

```
void rotiraj(vector<int>& v, int k) {
    int n = v.size();
    if (n == 0) return;
    k %= n;                                  // rotacija za n je isto što i za 0
    vector<int> r(n);
    for (int i = 0; i < n; i++) r[(i + k) % n] = v[i];
    v = r;
}
```

Elegantno rješenje bez pomoćnog vektora: obrni cijeli vektor, pa obrni prvih `k` i ostatak (`reverse` iz poglavlja 19).

Rješenje 7.4

```
int main() {
    int n;
    cin >> n;
    vector<vector<int>> m(n, vector<int>(n));
    for (auto& red : m) for (int& x : red) cin >> x;

    int sporedna = 0;
    bool simetricna = true;
    for (int i = 0; i < n; i++) {
        int zr = 0, zs = 0;
        for (int j = 0; j < n; j++) {
            zr += m[i][j];
            zs += m[j][i];
            if (m[i][j] != m[j][i]) simetricna = false;
        }
        cout << "red " << i << ": " << zr << ", stupac " << i << ": " << zs << "\n";
        sporedna += m[i][n - 1 - i];
    }
    cout << "sporedna: " << sporedna << (simetricna ? ", simetricna\n" : ", nije simetricna\n");
}
```

Rješenje 7.5

```
void izbaciDuplikate(vector<int>& v) {
    if (v.empty()) return;
    size_t pisi = 1;                         // gdje upisujemo sljedeći jedinstveni
    for (size_t citaj = 1; citaj < v.size(); citaj++)
        if (v[citaj] != v[pisi - 1]) v[pisi++] = v[citaj];
    v.resize(pisi);
}
```

Obrazac s dva indeksa („čitaj“ i „piši“) iznimno je koristan — isto radi `std::unique`.

Rješenje 7.6

```
vector<int> spoji(const vector<int>& a, const vector<int>& b) {
    vector<int> r;
    r.reserve(a.size() + b.size());
    size_t i = 0, j = 0;
    while (i < a.size() && j < b.size())
        r.push_back(a[i] <= b[j] ? a[i++] : b[j++]);
    while (i < a.size()) r.push_back(a[i++]);   // ostatak prvog
    while (j < b.size()) r.push_back(b[j++]);   // ostatak drugog
    return r;
}
```

Poglavlje 8 — Stringovi

Rješenje 8.1

```
int main() {
    string s;
    getline(cin, s);
    int rijeci = 0, velika = 0;
    bool uRijeci = false;
```

```
    for (char& c : s) {
        if (isupper(c)) velika++;
        if (isspace(c)) { uRijeci = false; continue; }
        if (!uRijeci) { rijeci++; c = toupper(c); }   // prvo slovo riječi
        uRijeci = true;
    }
    cout << rijeci << " " << velika << "\n" << s << "\n";
}
```

Rješenje 8.2

```
bool jePalindrom(const string& s) {
    string t;
    for (char c : s)
        if (!isspace(c)) t += tolower(c);
    for (size_t i = 0, j = t.size(); i + 1 < j; i++, j--)
        if (t[i] != t[j - 1]) return false;
    return true;
}
```

Rješenje 8.3

```
string sifriraj(string s, int k) {
    k = ((k % 26) + 26) % 26;                // radi i za negativan k
    for (char& c : s) {
        if (islower(c)) c = 'a' + (c - 'a' + k) % 26;
        else if (isupper(c)) c = 'A' + (c - 'A' + k) % 26;
    }
    return s;
}
string desifriraj(string s, int k) { return sifriraj(s, -k); }
```

Rješenje 8.4

```
string zamijeniSve(string s, const string& staro, const string& novo) {
    if (staro.empty()) return s;
    size_t poz = s.find(staro);
    while (poz != string::npos) {
        s.replace(poz, staro.size(), novo);
        poz = s.find(staro, poz + novo.size());   // nastavi IZA umetnutog teksta
    }
    return s;
}
```

Da smo nastavljali od `poz + 1`, zamjena `"a"` → `"aa"` bi nalazila upravo umetnuta slova i nikad ne bi završila.

Rješenje 8.5

```
string komprimiraj(const string& s) {
    string r;
    for (size_t i = 0; i < s.size(); ) {
        size_t j = i;
        while (j < s.size() && s[j] == s[i]) j++;
        r += s[i];
        r += to_string(j - i);
        i = j;
    }
    return r.size() < s.size() ? r : s;
}

string dekomprimiraj(const string& s) {
    string r;
    for (size_t i = 0; i < s.size(); ) {
        char c = s[i++];
        int n = 0;
        while (i < s.size() && isdigit(s[i])) n = n * 10 + (s[i++] - '0');
        r += string(n, c);
    }
    return r;
}
```

Rješenje 8.6

```
bool anagramSort(string a, string b) {        // kopije, jer ih sortiramo
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    return a == b;
}

bool anagramBroj(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    int broj[26] = {};
    for (char c : a) broj[tolower(c) - 'a']++;
    for (char c : b) broj[tolower(c) - 'a']--;
    for (int x : broj) if (x != 0) return false;
    return true;
}
```

Brojanje je O(n), sortiranje O(n log n). Za kratke riječi razlika je nebitna, za duge tekstove brojanje je brže. (Verzija s brojanjem pretpostavlja samo slova a–z.)

Poglavlje 9 — Pokazivači i reference

Rješenje 9.1

Ispis: `11 0 0 0`. Korak po korak: `p → a`, `q → b`. `*p = *q + 1` upisuje 11 u `a`. `p = q` preusmjerava `p` na `b` (`a` ostaje 11). `*p = 0` upisuje 0 u `b`. Sada i `p` i `q` pokazuju na `b`, koji je 0.

Rješenje 9.2

```
void minMax(const int* a, int n, int* mn, int* mx) {
    *mn = *mx = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] < *mn) *mn = a[i];
        if (a[i] > *mx) *mx = a[i];
    }
}
void minMaxRef(const int* a, int n, int& mn, int& mx) {
    mn = mx = a[0];
    for (int i = 1; i < n; i++) { mn = min(mn, a[i]); mx = max(mx, a[i]); }
}
int main() {
    int a[] = {4, -2, 9, 7}, x, y;
    minMax(a, 4, &x, &y);        // šaljemo adrese
    minMaxRef(a, 4, x, y);       // šaljemo varijable
    cout << x << " " << y << "\n";
}
```

Verzija s referencama je čitljivija — nema `*` pri svakom korištenju ni `&` pri pozivu.

Rješenje 9.3

```
void obrni(int* a, int n) {
    int* lijevo = a;
    int* desno = a + n - 1;
    while (lijevo < desno) {
        int t = *lijevo; *lijevo = *desno; *desno = t;
        lijevo++;
        desno--;
    }
}
```

Rješenje 9.4

```
int duljina(const char* s) {
    const char* p = s;
    while (*p != '\0') p++;
    return p - s;                // razlika pokazivača = broj elemenata između njih
}
```

Rješenje 9.5

Ispis: `4 6 7 8 2`. Na početku `p → a[1]` (4). `*p++`: postfiks `++` ima veći prioritet od `*`, ali vraća staru adresu — ispisuje 4, a `p` prelazi na `a[2]`. `*p` je 6. `++*p` povećava *vrijednost* `a[2]` na 7 i ispisuje je. `*(p + 2) - *(p - 2)` = `a[4] - a[0]` = 10 − 2 = 8. `p - a` = 2 (`p` je na indeksu 2).

Poglavlje 10 — Dinamička memorija

Rješenje 10.1

```
int main() {
    int n;
    cin >> n;
    int* a = new int[n];
    double prosjek = 0;
    for (int i = 0; i < n; i++) { cin >> a[i]; prosjek += a[i]; }
    prosjek /= n;
    for (int i = 0; i < n; i++) if (a[i] > prosjek) cout << a[i] << " ";
    cout << "\n";
    delete[] a;
}
```

Rješenje 10.2

```
int* spoji(const int* a, int na, const int* b, int nb) {
    int* r = new int[na + nb];
    for (int i = 0; i < na; i++) r[i] = a[i];
    for (int i = 0; i < nb; i++) r[na + i] = b[i];
    return r;                    // POZIVATELJ mora pozvati delete[]
}
```

Odgovornost za brisanje prelazi na pozivatelja, ali to nigdje ne piše u tipu `int*` — lako ju je zaboraviti (ili pozvati `delete` umjesto `delete[]`). Zato se u modernom C++-u vraća `vector<int>` ili `unique_ptr<int[]>`, gdje je vlasništvo jasno.

Rješenje 10.3

1) `return b;` vraća adresu lokalnog polja `b` koje nestaje kad `f` završi — **viseći pokazivač**. 2) Polje `a` je alocirano, ali se nikad ne briše — **curenje** (ionako se vraća krivo polje). 3) `q = p;` gubi jedinu adresu dinamičkog `int(7)` — još jedno **curenje**. 4) `delete q;` briše memoriju koja nije dobivena s `new` (to je stog!) — nedefinirano ponašanje. 5) `delete[] p;` ponovno briše istu adresu — **dvostruko brisanje**. Ispravno: `f` vraća `a`, `main` koristi rezultat i poziva `delete[] p` točno jednom; `q` briše s `delete q` prije preusmjeravanja — ili, bolje, sve zamijeniti s `vector`.

Rješenje 10.4

```
struct DinPolje {
    int* podaci = nullptr;
    int velicina = 0;
    int kapacitet = 0;
};

void dodaj(DinPolje& p, int x) {
    if (p.velicina == p.kapacitet) {
        int novi = p.kapacitet == 0 ? 1 : 2 * p.kapacitet;
        int* n = new int[novi];
        for (int i = 0; i < p.velicina; i++) n[i] = p.podaci[i];
        delete[] p.podaci;
        p.podaci = n;
        p.kapacitet = novi;
    }
    p.podaci[p.velicina++] = x;
}
int dohvati(const DinPolje& p, int i) { return p.podaci[i]; }
void oslobodi(DinPolje& p) { delete[] p.podaci; p = DinPolje{}; }

int main() {
    DinPolje p;
    for (int i = 0; i < 20; i++) { dodaj(p, i); cout << p.kapacitet << " "; }
```

```
    cout << "\n" << dohvati(p, 19) << "\n";
    oslobodi(p);
}
```

IZLAZ PROGRAMA

```
1 2 4 4 8 8 8 8 16 16 16 16 16 16 16 16 32 32 32 32
19
```

Poglavlje 11 — Strukture i enumeracije

Rješenje 11.1

```
struct Pravokutnik { double sirina, visina; };

double povrsina(const Pravokutnik& p) { return p.sirina * p.visina; }
double opseg(const Pravokutnik& p)    { return 2 * (p.sirina + p.visina); }
bool jeKvadrat(const Pravokutnik& p)  { return p.sirina == p.visina; }

int main() {
    Pravokutnik naj{0, 0};
    for (int i = 0; i < 3; i++) {
        Pravokutnik p;
        cin >> p.sirina >> p.visina;
        if (povrsina(p) > povrsina(naj)) naj = p;
    }
    cout << naj.sirina << " x " << naj.visina << "\n";
}
```

Rješenje 11.2

```
struct Razlomak { long long b, n; };

Razlomak skrati(Razlomak r) {
    long long d = gcd(r.b, r.n);          // std::gcd iz <numeric>
    if (r.n < 0) d = -d;                  // nazivnik neka bude pozitivan
    return {r.b / d, r.n / d};
}
Razlomak zbroji(Razlomak x, Razlomak y)  { return skrati({x.b * y.n + y.b * x.n, x.n * y.n}); }
Razlomak pomnozi(Razlomak x, Razlomak y) { return skrati({x.b * y.b, x.n * y.n}); }

int main() {
    Razlomak r = zbroji(zbroji({1, 2}, {1, 3}), {1, 6});
    cout << r.b << "/" << r.n << "\n";
}
```

IZLAZ PROGRAMA

```
1/1
```

Rješenje 11.3

```
struct Student {
    string ime;
    vector<int> ocjene;
};

double prosjek(const Student& s) {
    if (s.ocjene.empty()) return 0;
    double z = 0;
    for (int o : s.ocjene) z += o;
    return z / s.ocjene.size();
}

void sortirajPoProsjeku(vector<Student>& v) {
    for (size_t i = 0; i + 1 < v.size(); i++)
        for (size_t j = 0; j + 1 < v.size() - i; j++)
            if (prosjek(v[j]) < prosjek(v[j + 1]))    // silazno
                swap(v[j], v[j + 1]);
}
```

Rješenje 11.4

```
enum class Dan { Pon, Uto, Sri, Cet, Pet, Sub, Ned };

Dan sutra(Dan d) { return static_cast<Dan>((static_cast<int>(d) + 1) % 7); }
bool jeVikend(Dan d) { return d == Dan::Sub || d == Dan::Ned; }
string naziv(Dan d) {
    const string imena[] = {"ponedjeljak", "utorak", "srijeda", "cetvrtak",
                            "petak", "subota", "nedjelja"};
    return imena[static_cast<int>(d)];
}

int main() {
    Dan d = Dan::Pon;
    for (int i = 0; i < 100; i++) d = sutra(d);   // ili: 100 % 7 = 2 dana dalje
    cout << naziv(d) << (jeVikend(d) ? " (vikend)" : "") << "\n";
}
```

IZLAZ PROGRAMA

```
srijeda
```

Rješenje 11.5

Kostur rješenja — izbornik s `do-while` i `switch`, podaci u `vector<Artikl>`:

```
struct Artikl { int sifra; string naziv; int kolicina; double cijena; };

Artikl* pronadji(vector<Artikl>& v, int sifra) {
    for (auto& a : v) if (a.sifra == sifra) return &a;
    return nullptr;                              // nije pronađen
}

int main() {
    vector<Artikl> skladiste;
    int izbor;
    do {
        cout << "1 dodaj  2 ispis  3 trazi  4 kolicina  5 vrijednost  0 kraj: ";
        cin >> izbor;
        switch (izbor) {
            case 1: { Artikl a; cin >> a.sifra >> a.naziv >> a.kolicina >> a.cijena;
                      skladiste.push_back(a); break; }
            case 2: for (const auto& a : skladiste)
                        cout << a.sifra << " " << a.naziv << " " << a.kolicina << "\n";
                    break;
            case 3: case 4: {
                int s; cin >> s;
                Artikl* a = pronadji(skladiste, s);
                if (!a) { cout << "Nema artikla\n"; break; }
                if (izbor == 3) cout << a->naziv << " " << a->kolicina << "\n";
                else cin >> a->kolicina;
                break;
            }
            case 5: { double z = 0;
                      for (const auto& a : skladiste) z += a.kolicina * a.cijena;
                      cout << z << "\n"; break; }
        }
    } while (izbor != 0);
}
```

Vitičaste zagrade oko `case` blokova potrebne su kad unutar njih deklariraš varijable.

Poglavlje 12 — Klase i objekti

Rješenje 12.1

```
class Pravokutnik {
    double sirina, visina;
public:
    Pravokutnik(double s, double v) : sirina(s < 0 ? 0 : s), visina(v < 0 ? 0 : v) {}
    double povrsina() const { return sirina * visina; }
    double opseg() const    { return 2 * (sirina + visina); }
```

```
    bool jeKvadrat() const  { return sirina == visina; }
};
```

Rješenje 12.2

Najlakše je interno čuvati **ukupan broj sekundi** — normalizacija i zbrajanje postaju trivijalni, a sate/minute računamo tek pri ispisu:

```
class Vrijeme {
    long long ukupno;                              // sekunde od ponoći
    void normaliziraj() { ukupno = ((ukupno % 86400) + 86400) % 86400; }
public:
    Vrijeme(int h, int m, int s) : ukupno(h * 3600LL + m * 60 + s) { normaliziraj(); }
    void dodajSekunde(int s) { ukupno += s; normaliziraj(); }
    void ispisi() const {
        cout << setfill('0') << setw(2) << ukupno / 3600 << ":"
             << setw(2) << ukupno % 3600 / 60 << ":" << setw(2) << ukupno % 60
             << setfill(' ') << "\n";
    }
};

int main() {
    Vrijeme t(0, 75, 80);
    t.ispisi();
    t.dodajSekunde(3600 * 23);
    t.ispisi();
}
```

IZLAZ PROGRAMA

```
01:16:20
00:16:20
```

Rješenje 12.3

```
class Stog {
    vector<int> v;
public:
    void push(int x) { v.push_back(x); }
    void pop() { if (!v.empty()) v.pop_back(); }        // ili baciti iznimku (pogl. 21)
    int top() const { return v.back(); }                // pozivatelj mora provjeriti empty()!
    bool empty() const { return v.empty(); }
    size_t size() const { return v.size(); }
};

bool zagradeIspravne(const string& s) {
    Stog st;
    const string otvorene = "([{", zatvorene = ")]}";
    for (char c : s) {
        if (otvorene.find(c) != string::npos) st.push(c);
        else if (size_t k = zatvorene.find(c); k != string::npos) {
            if (st.empty() || st.top() != otvorene[k]) return false;
            st.pop();
        }
    }
    return st.empty();                                  // ništa ne smije ostati otvoreno
}
```

Rješenje 12.4

Ključni dijelovi (`Racun.h`):

```
class Racun {
    static int brojOtvorenih;          // definicija u Racun.cpp: int Racun::brojOtvorenih = 0;
    int broj;
    std::string vlasnik;
    double stanje;
public:
    Racun(const std::string& v, double pocetno)
        : broj(++brojOtvorenih), vlasnik(v), stanje(pocetno >= 0 ? pocetno : 0) {}
    bool isplati(double iznos);
    void uplati(double iznos);
    bool prijenos(Racun& drugi, double iznos) {
```

```
        if (&drugi == this || !isplati(iznos)) return false;
        drugi.uplati(iznos);
        return true;
    }
    static int ukupnoOtvoreno() { return brojOtvorenih; }
};
```

Primijeti da `prijenos` smije pristupiti privatnim članovima objekta `drugi` — pristup je određen **klasom**, ne objektom.

Rješenje 12.5

```
class Polinom {
    vector<double> k;                  // k[i] je koeficijent uz x^i
public:
    Polinom(vector<double> koef) : k(std::move(koef)) {
        while (k.size() > 1 && k.back() == 0) k.pop_back();
    }
    int stupanj() const { return k.size() - 1; }
    double vrijednost(double x) const {        // Horner: (((a_n)x + a_(n-1))x + ...)
        double r = 0;
        for (int i = stupanj(); i >= 0; i--) r = r * x + k[i];
        return r;
    }
    Polinom derivacija() const {
        if (k.size() == 1) return Polinom({0});
        vector<double> d(k.size() - 1);
        for (size_t i = 1; i < k.size(); i++) d[i - 1] = k[i] * i;
        return Polinom(d);
    }
    void ispisi() const {
        bool prvi = true;
        for (int i = stupanj(); i >= 0; i--) {
            if (k[i] == 0 && stupanj() > 0) continue;
            double a = k[i];
            if (!prvi) { cout << (a < 0 ? " - " : " + "); a = fabs(a); }
            if (a != 1 || i == 0) cout << a;
            if (i > 0) cout << "x";
            if (i > 1) cout << "^" << i;
            prvi = false;
        }
        cout << "\n";
    }
};

int main() {
    Polinom p({1, -2, 3});             // 3x^2 - 2x + 1
    p.ispisi();
    cout << p.vrijednost(2) << "\n";
    p.derivacija().ispisi();
}
```

IZLAZ PROGRAMA

```
3x^2 - 2x + 1
9
6x - 2
```

Poglavlje 13 — Kopiranje i RAII

Rješenje 13.1

`Polje x(4);` — konstruktor. `Polje y(x);` — konstruktor kopiranja. `Polje z = x;` — konstruktor kopiranja (novi objekt!). `y = z;` — pridruživanje kopiranjem. `Polje w = Polje(2);` — samo običan konstruktor: od C++17 prevoditelj **mora** izostaviti kopiju (*guaranteed copy elision*) i gradi objekt izravno u `w`. `z = std::move(y);` — pridruživanje premještanjem.

Rješenje 13.2

```
class Tekst {
    char* znakovi;
```

```
    size_t n;
public:
    Tekst(const char* s = "") : n(strlen(s)) {
        znakovi = new char[n + 1];                // +1 za '\0'
        strcpy(znakovi, s);
    }
    ~Tekst() { delete[] znakovi; }
    Tekst(const Tekst& o) : znakovi(new char[o.n + 1]), n(o.n) {
        strcpy(znakovi, o.znakovi);
    }
    Tekst& operator=(const Tekst& o) {
        if (this != &o) {
            char* novo = new char[o.n + 1];   // prvo alociraj: ako baci, objekt ostaje ispravan
            strcpy(novo, o.znakovi);
            delete[] znakovi;
            znakovi = novo;
            n = o.n;
        }
        return *this;
    }
    size_t duljina() const { return n; }
    const char* c_str() const { return znakovi; }
};
```

Rješenje 13.3

Dva problema. (1) **Samopridruživanje**: kod `a = a` prvo se obriše `podaci`, a zatim se kopira iz `o.podaci` — iste, upravo obrisane memorije. (2) **Sigurnost kod iznimaka**: ako `new` baci `bad_alloc`, `podaci` već pokazuje na obrisanu memoriju, a `n` je promijenjen — objekt je pokvaren i destruktor će napraviti dvostruko brisanje. Ispravak: kao u 13.3 u poglavlju — prvo alociraj i kopiraj u novo polje, tek onda obriši staro; uz provjeru `this == &o`.

Rješenje 13.4

Verzija po pravilu nule — cijela klasa bez ijedne posebne funkcije:

```
class Matrica {
    int r, s;
    vector<double> p;                          // r*s elemenata, redak po redak
public:
    Matrica(int r, int s) : r(r), s(s), p(r * s, 0.0) {}
    double& at(int i, int j)       { return p[i * s + j]; }
    double  at(int i, int j) const { return p[i * s + j]; }
    Matrica pomnozi(const Matrica& b) const {
        if (s != b.r) throw invalid_argument("dimenzije se ne slazu");
        Matrica c(r, b.s);
        for (int i = 0; i < r; i++)
            for (int j = 0; j < b.s; j++)
                for (int k = 0; k < s; k++)
                    c.at(i, j) += at(i, k) * b.at(k, j);
        return c;
    }
};
```

Verzija s `double*` treba još destruktor, dva konstruktora i dva operatora pridruživanja — oko 40 redova koda u kojima se lako pogriješi. Vektor daje sve to besplatno i bez grešaka.

Poglavlje 14 — Preopterećenje operatora

Rješenje 14.1

Unutar klase `Razlomak` (uz pristup članovima `b` i `n`):

```
Razlomak& operator-=(const Razlomak& o) { return *this += -o; }
Razlomak& operator/=(const Razlomak& o) {
    if (o.b == 0) throw invalid_argument("dijeljenje s nulom");
    return *this *= Razlomak(o.n, o.b);          // množenje recipročnom vrijednošću
}
```

Izvan klase:

```
Razlomak operator-(Razlomak x, const Razlomak& y) { return x -= y; }
Razlomak operator/(Razlomak x, const Razlomak& y) { return x /= y; }
bool operator!=(const Razlomak& x, const Razlomak& y) { return !(x == y); }
bool operator>(const Razlomak& x, const Razlomak& y)  { return y < x; }
bool operator<=(const Razlomak& x, const Razlomak& y) { return !(y < x); }
bool operator>=(const Razlomak& x, const Razlomak& y) { return !(x < y); }
```

Rješenje 14.2

```
struct Vektor2D {
    double x = 0, y = 0;
    double duljina() const { return sqrt(x * x + y * y); }
};
Vektor2D operator+(Vektor2D a, Vektor2D b) { return {a.x + b.x, a.y + b.y}; }
Vektor2D operator-(Vektor2D a, Vektor2D b) { return {a.x - b.x, a.y - b.y}; }
Vektor2D operator*(Vektor2D a, double k)   { return {a.x * k, a.y * k}; }
Vektor2D operator*(double k, Vektor2D a)   { return a * k; }        // simetrija
double   operator*(Vektor2D a, Vektor2D b) { return a.x * b.x + a.y * b.y; }
bool operator==(Vektor2D a, Vektor2D b)    { return a.x == b.x && a.y == b.y; }
ostream& operator<<(ostream& os, Vektor2D a) { return os << "(" << a.x << ", " << a.y << ")"; }

int main() {
    Vektor2D u{3, 4}, v{1, 2};
    cout << u + v << " " << 2 * v << " " << u * v << " " << u.duljina() << "\n";
}
```

```
(4, 6) (2, 4) 11 5
```
IZLAZ PROGRAMA

Rješenje 14.3

```
class Novac {
    long long centi;
public:
    explicit Novac(long long c = 0) : centi(c) {}
    static Novac izEura(long long eur, int cent) { return Novac(eur * 100 + cent); }
    Novac& operator+=(Novac o) { centi += o.centi; return *this; }
    friend Novac operator+(Novac a, Novac b) { return a += b; }
    friend Novac operator-(Novac a, Novac b) { return Novac(a.centi - b.centi); }
    friend bool operator<(Novac a, Novac b)  { return a.centi < b.centi; }
    friend bool operator==(Novac a, Novac b) { return a.centi == b.centi; }
    friend ostream& operator<<(ostream& os, Novac n) {
        long long a = llabs(n.centi);
        return os << (n.centi < 0 ? "-" : "") << a / 100 << "."
                  << (a % 100 < 10 ? "0" : "") << a % 100 << " EUR";
    }
};

int main() {
    Novac a = Novac::izEura(10, 5), b = Novac::izEura(2, 0);
    cout << a + b << " " << b - a << "\n";
}
```

```
12.05 EUR -8.05 EUR
```
IZLAZ PROGRAMA

`double` ne može točno zapisati 0.10 ili 0.05 (poglavlje 2.9), pa se kroz tisuće operacija nakupljaju greške u centima. Cijeli brojevi centi su točni.

Rješenje 14.4

```
class VelikiBroj {
    vector<int> z;                          // znamenke, NAJMANJE značajna prva
public:
    VelikiBroj(const string& s = "0") {
        for (int i = s.size() - 1; i >= 0; i--) z.push_back(s[i] - '0');
    }
    friend VelikiBroj operator+(const VelikiBroj& a, const VelikiBroj& b) {
        VelikiBroj r; r.z.clear();
        int prijenos = 0;
        for (size_t i = 0; i < max(a.z.size(), b.z.size()) || prijenos; i++) {
            int s = prijenos + (i < a.z.size() ? a.z[i] : 0) + (i < b.z.size() ? b.z[i] : 0);
```

```
            r.z.push_back(s % 10);
            prijenos = s / 10;
        }
        return r;
    }
    friend VelikiBroj operator*(const VelikiBroj& a, const VelikiBroj& b) {
        VelikiBroj r; r.z.assign(a.z.size() + b.z.size(), 0);
        for (size_t i = 0; i < a.z.size(); i++)
            for (size_t j = 0; j < b.z.size(); j++)
                r.z[i + j] += a.z[i] * b.z[j];
        for (size_t k = 0; k + 1 < r.z.size(); k++) {   // prijenosi
            r.z[k + 1] += r.z[k] / 10;
            r.z[k] %= 10;
        }
        while (r.z.size() > 1 && r.z.back() == 0) r.z.pop_back();
        return r;
    }
    friend bool operator==(const VelikiBroj& a, const VelikiBroj& b) { return a.z == b.z; }
    friend bool operator<(const VelikiBroj& a, const VelikiBroj& b) {
        if (a.z.size() != b.z.size()) return a.z.size() < b.z.size();
        return lexicographical_compare(a.z.rbegin(), a.z.rend(), b.z.rbegin(), b.z.rend());
    }
    friend ostream& operator<<(ostream& os, const VelikiBroj& a) {
        for (auto it = a.z.rbegin(); it != a.z.rend(); ++it) os << *it;
        return os;
    }
};

int main() {
    VelikiBroj f("1");
    for (int i = 2; i <= 50; i++) f = f * VelikiBroj(to_string(i));
    cout << f << "\n";
}
```

```
30414093201713378043612608166064768844377641568960512000000000000
```
IZLAZ PROGRAMA

Poglavlje 15 — Nasljeđivanje

Rješenje 15.1

Uz klasu `Osoba` iz odjeljka 15.2:

```
class Profesor : public Osoba {
    string katedra;
    double placa;
public:
    Profesor(const string& i, int g, const string& k, double p)
        : Osoba(i, g), katedra(k), placa(p) {}
    void predstaviSe() const {
        Osoba::predstaviSe();
        cout << " Radim na katedri " << katedra << ".";
    }
    void povisica(double postotak) { if (postotak > 0) placa *= 1 + postotak / 100; }
};
```

Rješenje 15.2

```
struct Vozilo {
    string marka; int godina;
    Vozilo(string m, int g) : marka(m), godina(g) { cout << "Vozilo() "; }
    ~Vozilo() { cout << "~Vozilo() "; }
};
struct Automobil : Vozilo {
    int vrata;
    Automobil(string m, int g, int v) : Vozilo(m, g), vrata(v) { cout << "Automobil() "; }
    ~Automobil() { cout << "~Automobil() "; }
};
struct Motocikl : Vozilo {
    bool prikolica;
    Motocikl(string m, int g, bool p) : Vozilo(m, g), prikolica(p) { cout << "Motocikl() "; }
    ~Motocikl() { cout << "~Motocikl() "; }
```

```
};

int main() {
    {
        Automobil a("Skoda", 2020, 5);
        Motocikl m("Honda", 2018, false);
        cout << "| ";
    }
    cout << "\n";
}
```

```
Vozilo() Automobil() Vozilo() Motocikl() | ~Motocikl() ~Vozilo() ~Automobil() ~Vozilo()
```
IZLAZ PROGRAMA

Rješenje 15.3

Ako je `stanje` `private`, izvedena klasa ga ne može izravno mijenjati. Čist dizajn je da bazna klasa pita izvedenu *smije li* isplatiti — virtualnom metodom (to je već uvod u poglavlje 16):

```
class Racun {
    double stanje = 0;
protected:
    virtual double donjaGranica() const { return 0; }   // izvedene klase mogu promijeniti
public:
    virtual ~Racun() = default;
    double getStanje() const { return stanje; }
    void uplati(double x) { if (x > 0) stanje += x; }
    bool isplati(double x) {
        if (x <= 0 || stanje - x < donjaGranica()) return false;
        stanje -= x;
        return true;
    }
};
class TekuciRacun : public Racun {
    double dopusteniMinus;
protected:
    double donjaGranica() const override { return -dopusteniMinus; }
public:
    explicit TekuciRacun(double minus) : dopusteniMinus(minus) {}
};
class StedniRacun : public Racun {
    double stopa;
public:
    explicit StedniRacun(double s) : stopa(s) {}
    void pripisiKamate() { uplati(getStanje() * stopa); }
};
```

Rješenje 15.4

(a) **Kompozicija** — stog *koristi* vektor, ali nije vektor (inače bi naslijedio `insert` u sredinu i uništio LIFO pravilo). (b) **Nasljeđivanje** — pas *je* životinja. (c) **Kompozicija** — knjižnica *ima* knjige. (d) **Nasljeđivanje** — menadžer *je* zaposlenik (ako se ponaša kao zaposlenik u svim kontekstima). (e) **Ni jedno ni drugo naivno** — zbog `postaviSirinu` kvadrat se ne može koristiti svugdje gdje i pravokutnik; bolje su dvije zasebne klase (npr. obje izvedene iz `Oblik`). (f) **Kompozicija** — ekran *ima* piksele.

Poglavlje 16 — Polimorfizam

Rješenje 16.1

```
class Trokut : public Oblik {
    double a, b, c;
public:
    Trokut(double a, double b, double c) : a(a), b(b), c(c) {}
    double povrsina() const override {
        double s = (a + b + c) / 2;                    // Heronova formula
        return sqrt(s * (s - a) * (s - b) * (s - c));
    }
    string naziv() const override { return "trokut"; }
};
// u main-u samo: oblici.push_back(make_unique<Trokut>(3, 4, 5));
```

Petlja u `main`-u **ne mijenja se** — to je cijela poanta polimorfizma. (Za `opseg()` dodaš čistu virtualnu metodu u `Oblik` i implementaciju u svaku klasu.)

Rješenje 16.2

Ispis: `A::f B::f A::g B::f A::g B::f B::g`. `a.f()` — `a` je *kopija* (odrezani objekt), pravi `A` → `A::f`. `r.f()` — referenca, `f` virtualna → `B::f`. `r.g()` — `g` nije virtualna, tip reference je `A&` → `A::g`, a unutar nje `f()` je virtualna na objektu tipa `B` → `B::f`. `p->g()` — isto kao prethodno. `b.g()` — objekt tipa `B` → `B::g`.

Rješenje 16.3

```cpp
class Zaposlenik {
    string ime;
public:
    explicit Zaposlenik(string i) : ime(std::move(i)) {}
    virtual ~Zaposlenik() = default;
    virtual double placa() const = 0;
    const string& getIme() const { return ime; }
};
class StalniZaposlenik : public Zaposlenik {
    double mjesecna;
public:
    StalniZaposlenik(string i, double m) : Zaposlenik(std::move(i)), mjesecna(m) {}
    double placa() const override { return mjesecna; }
};
class SatnicaZaposlenik : public Zaposlenik {
    double satnica; int sati;
public:
    SatnicaZaposlenik(string i, double s, int h) : Zaposlenik(std::move(i)), satnica(s), sati(h) {}
    double placa() const override { return satnica * sati; }
};
class Prodavac : public Zaposlenik {
    double osnovica, promet;
public:
    Prodavac(string i, double o, double p) : Zaposlenik(std::move(i)), osnovica(o), promet(p) {}
    double placa() const override { return osnovica + 0.05 * promet; }
};

int main() {
    vector<unique_ptr<Zaposlenik>> z;
    z.push_back(make_unique<StalniZaposlenik>("Ana", 1500));
    z.push_back(make_unique<SatnicaZaposlenik>("Ivo", 10, 120));
    z.push_back(make_unique<Prodavac>("Eva", 900, 20000));
    double ukupno = 0;
    for (const auto& x : z) {
        cout << x->getIme() << ": " << x->placa() << "\n";
        ukupno += x->placa();
    }
    cout << "Ukupno: " << ukupno << "\n";
}
```

IZLAZ PROGRAMA

```
Ana: 1500
Ivo: 1200
Eva: 1900
Ukupno: 4600
```

Rješenje 16.4

Ključ je da **svaka životinja sama zna** što jede i koliko, a zbrajanje po vrsti hrane radi `map`:

```cpp
class Zivotinja {
public:
    virtual ~Zivotinja() = default;
    virtual string ime() const = 0;
    virtual string zvuk() const = 0;
    virtual string hrana() const = 0;
    virtual double dnevnaKolicinaHrane() const = 0;
};
// class Lav : public Zivotinja { ... hrana() vraća "meso", količina 7 ... }  itd.

map<string, double> potrebe;
```

```
for (const auto& z : zoo) {
    cout << z->ime() << ": " << z->zvuk() << "\n";
    potrebe[z->hrana()] += z->dnevnaKolicinaHrane();
}
for (const auto& [hrana, kg] : potrebe) cout << hrana << ": " << kg << " kg\n";
```

Poglavlje 17 — Predlošci

Rješenje 17.1

```
template <typename T>
void zamijeni(T& a, T& b) {
    T t = std::move(a);          // move: jeftino i za velike objekte
    a = std::move(b);
    b = std::move(t);
}
```

Rješenje 17.2

```
template <typename T>
int pronadji(const vector<T>& v, const T& x) {
    for (size_t i = 0; i < v.size(); i++)
        if (v[i] == x) return static_cast<int>(i);
    return -1;
}

template <typename T>
T najveciElement(const vector<T>& v) {
    if (v.empty()) throw invalid_argument("prazan vektor nema najveci element");
    T m = v[0];
    for (const T& x : v) if (m < x) m = x;      // zahtijeva samo operator <
    return m;
}
```

Za prazan vektor nema smislenog rezultata — bacanje iznimke (ili vraćanje `optional<T>`) je poštenije od vraćanja izmišljene vrijednosti.

Rješenje 17.3

```
template <typename T>
class Kutija {
    optional<T> sadrzaj;
public:
    void stavi(T x) {
        if (sadrzaj) throw logic_error("kutija je puna");
        sadrzaj = std::move(x);
    }
    T uzmi() {
        if (!sadrzaj) throw logic_error("kutija je prazna");
        T x = std::move(*sadrzaj);
        sadrzaj.reset();
        return x;
    }
    bool jePrazna() const { return !sadrzaj.has_value(); }
};

int main() {
    Kutija<vector<string>> k;
    k.stavi({"a", "b"});
    cout << k.uzmi().size() << " " << k.jePrazna() << "\n";
}
```

IZLAZ PROGRAMA

```
2 1
```

Rješenje 17.4

```
template <typename T, int N>
class Red {
    T podaci[N];
```

```
    int pocetak = 0, broj = 0;          // indeks prvog elementa i broj elemenata
public:
    bool jePrazan() const { return broj == 0; }
    bool jePun() const { return broj == N; }
    void dodaj(const T& x) {
        if (jePun()) throw overflow_error("red je pun");
        podaci[(pocetak + broj) % N] = x;    // mjesto iza zadnjeg, "u krug"
        broj++;
    }
    void ukloni() {
        if (jePrazan()) throw underflow_error("red je prazan");
        pocetak = (pocetak + 1) % N;
        broj--;
    }
    const T& prvi() const { return podaci[pocetak]; }
};

int main() {
    Red<int, 3> r;
    r.dodaj(1); r.dodaj(2); r.dodaj(3);
    r.ukloni();
    r.dodaj(4);                          // upisuje se na mjesto 0 — "omatanje"
    while (!r.jePrazan()) { cout << r.prvi() << " "; r.ukloni(); }
    cout << "\n";
}
```

```
2 3 4
```
IZLAZ PROGRAMA

Poglavlje 18 — STL kontejneri

Rješenje 18.1

```
int main() {
    set<string> razlicite;
    map<string, int> broj;
    string r;
    while (cin >> r) {
        razlicite.insert(r);
        broj[r]++;
    }
    cout << "Razlicitih: " << razlicite.size() << "\n";   // (isto je i broj.size())
    for (const auto& [rijec, n] : broj) cout << rijec << " " << n << "\n";
}
```

Rješenje 18.2

```
bool zagrade(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char o = st.top(); st.pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c == '}' && o != '{'))
                return false;
        }
    }
    return st.empty();
}

double postfiks(const string& izraz) {
    stack<double> st;
    stringstream ss(izraz);
    string t;
    while (ss >> t) {
        if (t == "+" || t == "-" || t == "*" || t == "/") {
            double b = st.top(); st.pop();         // PAZI: prvo se skida DRUGI operand
            double a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
```

```
            else if (t == "*") st.push(a * b);
            else st.push(a / b);
        } else {
            st.push(stod(t));
        }
    }
    return st.top();
}

int main() {
    cout << zagrade("(a+[b*c]-{d})") << zagrade("(]") << " " << postfiks("3 4 + 2 *") << "\n";
}
```

IZLAZ PROGRAMA

```
10 14
```

Rješenje 18.3

Ključne operacije:

```
map<string, vector<string>> imenik;
imenik[ime].push_back(broj);                    // dodaj broj (osoba se stvara ako ne postoji)
imenik.erase(ime);                              // obriši osobu
if (auto it = imenik.find(ime); it != imenik.end())
    for (const auto& b : it->second) cout << b << "\n";   // brojevi osobe
for (const auto& [osoba, brojevi] : imenik)     // map je već sortirana po imenu
    cout << osoba << ": " << brojevi.size() << " broj(eva)\n";
```

Rješenje 18.4

```
struct Kupac { string ime; int artikli; };

int main() {
    queue<Kupac> red;
    red.push({"Ana", 3});
    red.push({"Ivo", 1});
    red.push({"Eva", 2});
    int minuta = 0;
    while (!red.empty()) {
        Kupac k = red.front();
        red.pop();
        minuta += k.artikli;                    // blagajna obradi sve njegove artikle
        cout << k.ime << " odlazi u minuti " << minuta << "\n";
    }
}
```

IZLAZ PROGRAMA

```
Ana odlazi u minuti 3
Ivo odlazi u minuti 4
Eva odlazi u minuti 6
```

Rješenje 18.5

```
vector<int> kNajvecih(const vector<int>& v, int k) {
    priority_queue<int, vector<int>, greater<int>> minRed;   // na vrhu je NAJMANJI
    for (int x : v) {
        minRed.push(x);
        if ((int)minRed.size() > k) minRed.pop();            // izbaci najmanjeg
    }
    vector<int> r;
    while (!minRed.empty()) { r.push_back(minRed.top()); minRed.pop(); }
    return r;                                                 // uzlazno
}
```

Red nikad nema više od `k` elemenata, pa svaka operacija košta O(log k), ukupno O(n log k). Sortiranje cijelog vektora je O(n log n) i treba svu memoriju odjednom; za `n` = milijardu i `k` = 10 razlika je golema — a ovaj pristup radi čak i kad podaci stižu jedan po jedan (npr. iz datoteke).

Rješenje 18.6

```cpp
// vraća indekse dvaju elemenata ili {-1, -1}
pair<int, int> dvaZbroja(const vector<int>& v, int cilj) {
    unordered_map<int, int> vidjeni;          // vrijednost -> indeks
    for (int i = 0; i < (int)v.size(); i++) {
        auto it = vidjeni.find(cilj - v[i]);  // je li "par" već viđen?
        if (it != vidjeni.end()) return {it->second, i};
        vidjeni[v[i]] = i;
    }
    return {-1, -1};
}
```

Dvije ugniježđene petlje ispituju sve parove — $O(n^2)$. Ovdje za svaki element u prosječno $O(1)$ provjerimo je li njegov „komplement“ već viđen, pa je ukupno $O(n)$.

Poglavlje 19 — Iteratori, algoritmi i lambde

Rješenje 19.1

```cpp
int main() {
    vector<int> v = {4, -3, 14, 8, -1, 9};
    int zbroj = accumulate(v.begin(), v.end(), 0);
    cout << zbroj << " " << static_cast<double>(zbroj) / v.size() << " "
         << *max_element(v.begin(), v.end()) << " "
         << count_if(v.begin(), v.end(), [](int x) { return x < 0; }) << " "
         << boolalpha << any_of(v.begin(), v.end(), [](int x) { return x % 7 == 0; }) << "\n";
}
```

IZLAZ PROGRAMA

```
31 5.16667 14 2 true
```

Rješenje 19.2

```cpp
struct Student { string ime, prezime; int godina; double prosjek; };

void sortiraj(vector<Student>& v) {
    sort(v.begin(), v.end(), [](const Student& a, const Student& b) {
        if (a.godina != b.godina) return a.godina < b.godina;       // 1. godina uzlazno
        if (a.prosjek != b.prosjek) return a.prosjek > b.prosjek;   // 2. prosjek silazno
        return a.prezime < b.prezime;                               // 3. prezime
    });
}
```

Elegantna alternativa: usporedba n-torki, `return tie(a.godina, b.prosjek, a.prezime) < tie(b.godina, a.prosjek, b.prezime);` — silazni kriterij dobiva se zamjenom `a` i `b` na tom mjestu.

Rješenje 19.3

```cpp
int main() {
    vector<string> v = {"ana", "", "c", "Ivo", "ana", "ok", "eva"};
    v.erase(remove_if(v.begin(), v.end(), [](const string& s) { return s.size() < 3; }),
            v.end());
    for (string& s : v)
        transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return toupper(c); });
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    for (const auto& s : v) cout << s << " ";
    cout << "\n";
}
```

IZLAZ PROGRAMA

```
ANA EVA IVO
```

Rješenje 19.4

```cpp
template <typename F>
vector<int> filtriraj(const vector<int>& v, F uvjet) {
    vector<int> r;
    for (int x : v) if (uvjet(x)) r.push_back(x);
    return r;
}
```

```cpp
int main() {
    vector<int> v = {1, 5, 8, 12, 15, 20};
    int prag = 10;
    auto a = filtriraj(v, [](int x) { return x % 2 == 0; });
    auto b = filtriraj(v, [prag](int x) { return x > prag; });     // hvata prag
    auto c = filtriraj(v, [](int x) { return x % 5 == 0; });
    cout << a.size() << " " << b.size() << " " << c.size() << "\n";
}
```

IZLAZ PROGRAMA

```
3 3 3
```

Rješenje 19.5

```cpp
int main() {
    map<string, int> broj;
    string r;
    while (cin >> r) {
        string cista;
        for (char c : r) if (isalpha((unsigned char)c)) cista += tolower(c);
        if (!cista.empty()) broj[cista]++;
    }
    vector<pair<string, int>> v(broj.begin(), broj.end());
    sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
        if (a.second != b.second) return a.second > b.second;      // češće prvo
        return a.first < b.first;                                  // pa abecedno
    });
    for (size_t i = 0; i < v.size() && i < 10; i++)
        cout << v[i].first << " " << v[i].second << "\n";
}
```

Poglavlje 20 — Pametni pokazivači

Rješenje 20.1

```cpp
int main() {
    auto p = make_unique<int>(42);       // umjesto new int(42) ... delete p
    *p += 8;
    cout << *p << "\n";

    int n;
    cin >> n;
    vector<double> ocjene(n);            // umjesto new double[n] ... delete[]
    for (double& x : ocjene) cin >> x;
}
```

Rješenje 20.2

```cpp
struct Cvor {
    int vrijednost;
    unique_ptr<Cvor> sljedeci;
};

class Lista {
    unique_ptr<Cvor> glava;
public:
    void dodajNaPocetak(int x) {
        glava = make_unique<Cvor>(Cvor{x, std::move(glava)});   // novi čvor preuzima staru glavu
    }
    void ispisi() const {
        for (Cvor* c = glava.get(); c; c = c->sljedeci.get()) cout << c->vrijednost << " ";
        cout << "\n";
    }
    int zbroj() const {
        int s = 0;
        for (Cvor* c = glava.get(); c; c = c->sljedeci.get()) s += c->vrijednost;
        return s;
    }
};
```

```
int main() {
    Lista l;
    for (int i = 1; i <= 4; i++) l.dodajNaPocetak(i);
    l.ispisi();
    cout << l.zbroj() << "\n";
}
```

IZLAZ PROGRAMA

```
4 3 2 1
10
```

Destruktor ne treba: kad nestane `glava`, njen `unique_ptr` briše prvi čvor, čiji `sljedeci` briše drugi, i tako dalje. (Za jako duge liste ta lančana rekurzija može prepuniti stog — tada se piše destruktor koji briše petljom.) Za prolazak koristimo sirove pokazivače (`get()`), jer oni samo *gledaju*, ne posjeduju.

Rješenje 20.3

Ispis: `2 1 0 1`. Nakon `q = p` vlasnika su 2 (`weak_ptr` se ne broji). `p.reset()` — ostaje jedan vlasnik (`q`), objekt živi pa `w.expired()` daje `false` (0). `q.reset()` — zadnji vlasnik nestaje, objekt se briše i `w.expired()` postaje `true` (1).

Rješenje 20.4

```
class Mapa {
    string ime;
    Mapa* roditelj;                                  // ne posjeduje — samo pokazuje gore
    vector<unique_ptr<Mapa>> podmape;                // posjeduje djecu
public:
    Mapa(string i, Mapa* r = nullptr) : ime(std::move(i)), roditelj(r) {}
    Mapa* dodajPodmapu(const string& i) {
        podmape.push_back(make_unique<Mapa>(i, this));
        return podmape.back().get();
    }
    string punaPutanja() const {
        if (!roditelj) return ime.empty() ? "" : "/" + ime;
        return roditelj->punaPutanja() + "/" + ime;
    }
};

int main() {
    Mapa korijen("");
    Mapa* p = korijen.dodajPodmapu("home")->dodajPodmapu("ana")->dodajPodmapu("projekti");
    cout << p->punaPutanja() << "\n";
}
```

IZLAZ PROGRAMA

```
/home/ana/projekti
```

Roditelj posjeduje djecu (`unique_ptr`), a dijete na roditelja ima samo sirovi pokazivač. Da je i veza prema gore bila vlasnička (`shared_ptr` u oba smjera), nastao bi ciklus i ništa se ne bi obrisalo.

Poglavlje 21 — Iznimke

Rješenje 21.1

```
int ucitajBroj(const string& s, int min, int max) {
    int n = stoi(s);                                 // može baciti invalid_argument / out_of_range
    if (n < min || n > max)
        throw out_of_range("broj " + to_string(n) + " nije u ["
                           + to_string(min) + ", " + to_string(max) + "]");
    return n;
}

int main() {
    for (string s : {"5", "abc", "50", "99999999999"}) {
        try {
            int n = ucitajBroj(s, 1, 10);    // prvo izračunaj, pa ispiši
            cout << "OK: " << n << "\n";
        } catch (const invalid_argument&) {
            cout << "'" << s << "' nije broj\n";
        } catch (const out_of_range& e) {
```

```
            cout << "izvan raspona: " << e.what() << "\n";
        }
    }
}
```

IZLAZ PROGRAMA

```
OK: 5
'abc' nije broj
izvan raspona: broj 50 nije u [1, 10]
izvan raspona: stoi
```

Zadnji redak pokazuje da i `stoi` baca `out_of_range` (s kratkom porukom) — isti `catch` hvata obje situacije.

Rješenje 21.2

U konstruktoru `Razlomak`-a iz poglavlja 14:

```
Razlomak(long long brojnik = 0, long long nazivnik = 1) : b(brojnik), n(nazivnik) {
    if (n == 0) throw invalid_argument("nazivnik ne smije biti 0");
    skrati();
}
```

Ako konstruktor baci iznimku, objekt **uopće ne nastaje** — ne postoji varijabla s neispravnim stanjem koju bi netko mogao koristiti. Sve ostale operacije (`/=`, `/`) grade nove razlomke preko konstruktora ili provjeravaju nulu, pa neispravan razlomak ne može nastati nigdje.

Rješenje 21.3

```
struct BankaGreska : runtime_error { using runtime_error::runtime_error; };
struct NedovoljnoSredstava : BankaGreska { using BankaGreska::BankaGreska; };
struct RacunBlokiran : BankaGreska { using BankaGreska::BankaGreska; };
struct NepostojeciRacun : BankaGreska { using BankaGreska::BankaGreska; };

struct Racun { double stanje; bool blokiran; };

void prijenos(map<int, Racun>& racuni, int izvor, int odrediste, double iznos) {
    auto i = racuni.find(izvor), o = racuni.find(odrediste);
    if (i == racuni.end() || o == racuni.end()) throw NepostojeciRacun("nepostojeci racun");
    if (i->second.blokiran) throw RacunBlokiran("racun " + to_string(izvor) + " je blokiran");
    if (i->second.stanje < iznos) throw NedovoljnoSredstava("nedovoljno sredstava");
    i->second.stanje -= iznos;
    o->second.stanje += iznos;
}

int main() {
    map<int, Racun> r = {{1, {100, false}}, {2, {50, true}}};
    vector<tuple<int, int, double>> nalozi = {{1, 2, 30}, {2, 1, 10}, {1, 9, 5}, {1, 2, 500}};
    for (auto [iz, u, iznos] : nalozi) {
        try {
            prijenos(r, iz, u, iznos);
            cout << "uspjeh\n";
        } catch (const NedovoljnoSredstava& e) {
            cout << "Posebna obrada: " << e.what() << "\n";
        } catch (const BankaGreska& e) {              // sve ostale bankovne greške
            cout << "Greska banke: " << e.what() << "\n";
        }
    }
}
```

IZLAZ PROGRAMA

```
uspjeh
Greska banke: racun 2 je blokiran
Greska banke: nepostojeci racun
Posebna obrada: nedovoljno sredstava
```

`using runtime_error::runtime_error;` nasljeđuje konstruktore bazne klase — ne moramo ih ručno pisati.

Rješenje 21.4

Ispis: `X1 X2 X3 ~X3 ~X2 ~X1 uhvacen 3`. Pozivi `f(1)`, `f(2)`, `f(3)` stvaraju objekte X1, X2, X3. U `f(3)` baca se iznimka; odmatanjem stoga uništavaju se lokalni objekti svih funkcija kroz koje iznimka prolazi — obrnutim redom — i tek tada se ulazi u `catch`.

Poglavlje 22 — Datoteke

Rješenje 22.1

```
int main(int argc, char* argv[]) {
    if (argc < 2) { cerr << "Upotreba: wc datoteka\n"; return 1; }
    ifstream f(argv[1]);
    if (!f) { cerr << "Ne mogu otvoriti " << argv[1] << "\n"; return 1; }
    long long redova = 0, rijeci = 0, znakova = 0;
    string red;
    while (getline(f, red)) {
        redova++;
        znakova += red.size() + 1;            // +1 za '\n'
        stringstream ss(red);
        string r;
        while (ss >> r) rijeci++;
    }
    cout << redova << " " << rijeci << " " << znakova << "\n";
}
```

`argc` i `argv` su argumenti iz naredbenog retka: `./wc tekst.txt`.

Rješenje 22.2

```
int main() {
    ifstream ulaz("brojevi.txt");
    if (!ulaz) { cerr << "Nema datoteke\n"; return 1; }
    vector<int> parni, neparni;
    int x;
    while (ulaz >> x) (x % 2 == 0 ? parni : neparni).push_back(x);
    sort(parni.begin(), parni.end());
    sort(neparni.begin(), neparni.end());
    ofstream p("parni.txt"), n("neparni.txt");
    for (int v : parni) p << v << "\n";
    for (int v : neparni) n << v << "\n";
}
```

Rješenje 22.3

Ključna ideja: metode primaju `ostream&` / `istream&`, pa rade s bilo kojim tokom — datotekom, `cout`-om ili `stringstream`-om:

```
void Racun::spremi(ostream& os) const {
    os << broj << ' ' << stanje << ' ' << vlasnik << '\n';   // ime na kraju (može imati razmake)
}
optional<Racun> Racun::ucitaj(istream& is) {                  // static metoda
    int broj; double stanje; string vlasnik;
    if (!(is >> broj >> stanje)) return nullopt;
    getline(is >> ws, vlasnik);
    return Racun(broj, vlasnik, stanje);
}
// main: while (auto r = Racun::ucitaj(f)) racuni.push_back(*r);
```

Rješenje 22.4

```
struct Rezultat { string jmbag, ime, prezime; int bodovi[4]; int ukupno; int ocjena; };

int ocjena(int b) { return b >= 90 ? 5 : b >= 75 ? 4 : b >= 60 ? 3 : b >= 50 ? 2 : 1; }

int main() {
    ifstream ulaz("ispit.csv");
    ofstream greske("greske.txt");
    if (!ulaz) { cerr << "Nema ulazne datoteke\n"; return 1; }
    vector<Rezultat> svi;
    string red;
    getline(ulaz, red);                                       // zaglavlje
    for (int brojReda = 2; getline(ulaz, red); brojReda++) {
        stringstream ss(red);
        Rezultat r;
        string polje;
        try {
```

```
            getline(ss, r.jmbag, ';'); getline(ss, r.ime, ';'); getline(ss, r.prezime, ';');
            r.ukupno = 0;
            for (int& b : r.bodovi) {
                if (!getline(ss, polje, ';')) throw runtime_error("premalo stupaca");
                b = stoi(polje);
                r.ukupno += b;
            }
            r.ocjena = ocjena(r.ukupno);
            svi.push_back(r);
        } catch (const exception& e) {
            greske << "red " << brojReda << ": " << e.what() << "\n";
        }
    }
    sort(svi.begin(), svi.end(), [](const Rezultat& a, const Rezultat& b) {
        return a.ukupno > b.ukupno;
    });
    ofstream izlaz("rezultati.csv");
    izlaz << "jmbag;ime;prezime;z1;z2;z3;z4;ukupno;ocjena\n";
    for (const auto& r : svi) {
        izlaz << r.jmbag << ';' << r.ime << ';' << r.prezime;
        for (int b : r.bodovi) izlaz << ';' << b;
        izlaz << ';' << r.ukupno << ';' << r.ocjena << '\n';
    }
}
```

Kraj skripte. Sretno na ispitu — i, još važnije, uživaj u programiranju.